

Efficient and Consistent Bundle Adjustment on Lidar Point Clouds

Zheng Liu, Xiyuan Liu and Fu Zhang

Abstract—Simultaneous determination of sensor poses and scene geometry is a fundamental problem for robot vision that is often achieved by Bundle Adjustment (BA). This paper presents an efficient and consistent bundle adjustment method for lidar sensors. The method employs edge and plane features to represent the scene geometry, and directly minimizes the natural Euclidean distance from each raw point to the respective geometry feature. A nice property of this formulation is that the geometry features can be analytically solved, drastically reducing the dimension of the numerical optimization. To represent and solve the resultant optimization problem more efficiently, this paper then adopts and formalizes the concept of *point cluster*, which encodes all raw points associated to the same feature by a compact set of parameters, the *point cluster coordinates*. We derive the closed-form derivatives, up to the second order, of the BA optimization based on the point cluster coordinates and show their theoretical properties such as the null spaces and sparsity. Based on these theoretical results, this paper develops an efficient second-order BA solver. Besides estimating the lidar poses, the solver also exploits the second order information to estimate the pose uncertainty caused by measurement noises, leading to consistent estimates of lidar poses. Moreover, thanks to the use of point cluster, the developed solver fundamentally avoids the enumeration of each raw point in all steps of the optimization: cost evaluation, derivatives evaluation and uncertainty evaluation. The implementation of our method is open sourced to benefit the robotics community².

Index Terms—Bundle adjustment, lidar SLAM.

I. INTRODUCTION

LIGHT detection and ranging (lidar) has become an essential sensing technology for robots to achieve a high level of autonomy [1, 2]. Enabled by the direct, dense, active and accurate (DDAA) depth measurements, lidar sensors have the ability to build a dense and accurate 3D map of the environment in real-time and at a relatively low computation cost. These unique advantages have made lidar sensors essential to a variety of applications that require real-time, dense, and accurate 3D mapping of the environment, such as autonomous driving [3, 4], unmanned aerial vehicles navigation [5]–[7], and real-time mobile mapping [8]–[10]. This trend becomes even more evident with recent developments in lidar technologies which have enabled the commercialization and

mass production of lightweight and high-performance solid-state lidars at a significantly lower cost [11, 12].

The central task of many lidar-based techniques, such as lidar-based odometry, simultaneous localization and mapping (SLAM), and multi-lidar calibration, is to register multiple point clouds, each measured by the lidar at different poses, into a consistent global point cloud map. However, the predominant point cloud registration methods, such as iterative closest point (ICP) [13] and its variants (e.g. generalized-ICP [14]), normal distribution transformation (NDT) [15, 16], and surfel registration [17], allow registration of two point clouds only. Such a pairwise registration leads to an incremental scan registration process for an odometry system (e.g., [17]–[20]), which would rapidly accumulate drift, or a repeated pairwise registration process for 3D mapping [21] or multi-lidar calibration [22], which would bring dramatic computation cost. All these necessitate an efficient concurrent multiple scan registration technique.

Concurrent multiple lidar scan registration requires determining all lidar poses and the scene geometry simultaneously, a process referred to as *bundle adjustment (BA)* in computer vision. Compared to visual BA, which has been well-established in photogrammetry and played a fundamental role in various vital applications, including visual odometry (VO) [23]–[25], visual-inertial odometry (VIO) [26, 27], 3D visual reconstruction [28, 29] and multi-camera calibration [30, 31], lidar BA has a similarly fundamental role but is much less mature due to two major challenges. First, lidar has a long measuring range but low resolution between scanning lines. The measured point cloud are sparsely (sometimes even not repeatedly [12]) distributed in a large 3D space, making it difficult (almost impossible) to scan the same point feature in the space across different scans. This has fundamentally prevented the use of straightforward visual bundle adjustment formulation, which is largely based on point features benefiting from the high-resolution images accurately capturing individual point features. The second challenge lies in the large number of raw points (from tens of thousands to million points) collected by a practical lidar sensors. Processing all these points in the lidar BA is extremely computation intensive.

In this work, we propose an efficient and consistent BA framework specifically designed for lidar point clouds. The framework follows our previous work BALM [32], which formulates the lidar BA problem based on edge and plane features that are abundant in lidar scans. The BA formulation naturally minimizes the straightforward Euclidean distance of each point in a scan to the corresponding edge or plane, while the decision variables include the lidar poses and feature

Manuscript received September 15, 2022, revised January 20, 2023 and May 2, 2023, accepted August 28, 2023. This work is supported in part by the University Grants Committee of Hong Kong General Research Fund (project number 17206421) and in part by DJI under the grant number 200009538. (Corresponding author: Fu Zhang.)

The authors are with the Mechatronics and Robotic Systems (MaRS) Laboratory, Department of Mechanical Engineering, University of Hong Kong, Hong Kong SAR, China (e-mail: u3007335@connect.hku.hk; xli-uaa@connect.hku.hk; fuzhang@hku.hk).

²<https://github.com/hku-mars/BALM>

(edge and plane) parameters. Furthermore, it is shown that the geometry parameters (i.e., edge and plane) can be solved analytically, leading to an optimization that depends on the lidar poses only. Since the number of geometry features is often large, elimination of these geometry features from the optimization will drastically reduce the optimization dimension (hence time).

A key concept our proposed BA framework adopts and formalizes is the *point cluster* [33]–[35], which summarizes all points of a lidar scan associated to one feature by a compact set of parameters, *point cluster coordinates*. Based on the point cluster, we derive the closed-form derivatives (up to second order) of the BA optimization with respect to (w.r.t.) its decision variables (i.e., lidar poses). We prove that the formulated BA optimization and the closed-form derivatives can both be represented fully by the point cluster without enumerating the large number of individual points in a lidar scan. The removal of dependence on individual raw points drastically speeds up the evaluation of the cost function and derivatives, which further enables us to develop an efficient and consistent second-order solver, BALM2.0, which is also released on Github to benefit the community. Our experiment video is available on website³.

We conduct extensive evaluations on the proposed BA method. Simulation study shows that the BA method produces consistent lidar pose estimate. Exhaustive benchmark comparison on 19 real-world open sequences shows that the BA method produces consistently higher performance (pose estimation accuracy, mapping accuracy, and computation efficiency) than other counterparts. We finally integrate the BA method in three vital lidar applications: lidar-inertial odometry, multi-lidar calibration, and global mapping, and show how their accuracy and/or computation efficiency are improved by the proposed BA.

II. RELATED WORKS

A. Multi-view registration

The bundle adjustment problem is similar to the *multi-view registration* problem that has been previously researched [36]–[41]. These methods all adopt a two layer framework: the first layer estimates the relative poses of a selected set of scan pairs using the pairwise registration methods (e.g., ICP [13]); From the relative poses, the second layer constructs and solves a pose graph to obtain a maximum a posteriori estimate of all lidar poses. Such a two-layer framework decouples the raw point registration from the global pose estimation, so that each raw point registration only involves a small amount of local points contained in the two scans (instead of all scans sharing overlaps) and the pose graph optimization only involves a small amount of constraints arising from the relative poses (instead of raw points). The net effect is a significant saving of time, hence being largely used in online lidar SLAM systems [42, 43]. However, the advantage in computation efficiency comes with fundamental limitation in accuracy: the pairwise scan registration only considers the overlap among two scan at a time, while the overlap is really shared by all scans and

should be registered concurrently. Moreover, the pose graph optimization only considers constraints from the relative poses, while the mapping consistency indicated by the raw points are completely ignored. Consequently, it is usually difficult to produce (or even be aware of) a globally consistent map that is necessary for high-accuracy localization and mapping tasks.

Some early works in computer vision and computer graphics have proposed multi-view registration methods that directly optimize the mapping consistency from multiple range images, aiming for consistent surface modeling of 3D objects. [44] is a direct extension of the ICP method, it minimizes the Euclidean distance between a pre-known control point in one scan to all matched control points in the rest scans. Within this framework, [45] uses a quaternion representation in the optimization, and [46] extends the distance between control points to the distance between surfaces around the respective control points. More recently, Zhu *et al.* [47] proposes a two step registration method: the first step uses a K-means clustering to cluster points from all scans, and the second step estimates the scan poses by minimizing the Euclidean distance between each point in a cluster to the centroid. Since these methods rely on point features in the scan, they require densely populated point cloud (e.g., by depth camera) for extracting such salient point features. While this is not a problem for small object reconstruction for which these methods are designed, it is not the case for scene reconstruction where the LiDAR measurements are very sparse (sometimes even non-repetitive) as explained above.

B. Bundle or plane adjustment

In recent years, researchers in the robotics community have shown increasing interests to address the bundle adjustment problem on (lidar) point clouds more formally. Kaess [48] exploits the plane features in the bundle adjustment and minimizes the difference between the plane measured in a scan and the plane predicted from the optimization variables: scan poses and plane parameters. This formulation was later integrated into a key-frame-based online SLAM system [49]. Since the method minimizes the plane-to-plane distance, it requires to segment each scan and estimate the contained local planes in advance. Such plane segmentation and estimation usually require dense point clouds measured by RGB-D cameras on which the work were demonstrated.

A more formal bundle adjustment method on lidar point cloud, termed as the *plannar (bundle) adjustment*, was later proposed in [34] which minimizes the natural Euclidean distance between each point in a scan to the plane predicted from the scan poses and plane parameters (the optimization variables). Compared with plane-to-plane distance in [48], the point-to-plane metric is faster, more accurate, and more suitable for lidar sensors, where local plane segmentation or estimation are less reliable due to sparse point clouds. Moreover, the direct use of raw points in the point-to-plane metric could also lead to a more consistent estimate of the optimization variables by considering the measurement noises in the raw points. Then, the formulated non-linear least square problem is solved by a Levenberg-Marquardt (LM) algorithm.

³<https://youtu.be/MDrIAyhQ-9E>

To lower the computation load caused by the large number of points measurements associated to the same plane feature, [34] propose a reduction technique to eliminate the enumeration of individual points in the evaluation of the residual and Jacobian. Furthermore, due to the very similar structure to the visual bundle adjustment, the proposed bundle adjustment is also compatible with the Schur complement trick [50], which eliminates the plane parameters in each iteration of the LM algorithm. This plane adjustment method is largely used in many online lidar SLAM systems developed subsequently [51, 52] or before [53, 54].

On the other hand, Ferrer [33] exploits plane features similar to [34] and minimizes any deviation of each raw point from the plane equation. The resultant optimization cost then reduces to the minimum eigenvalue of a covariance matrix and is thus termed as the eigen-factor. The author further derived the closed-form gradient of the cost function w.r.t. to both the scan poses and plane parameters and employed a gradient-based method to solve the optimization iteratively. Due to the second order nature of the eigenvalue (as confirmed in [32], see below), the gradient method converges very slowly (requiring a few hundreds of iterations) [33, 52].

Our previous work BALM [32] takes another step towards more efficient bundle adjustment. Similar to [34], BALM minimizes the natural Euclidean distance between each point in a scan to the plane (i.e., point-to-plane metric). Based on this cost metric, BALM proved that all plane parameters can be analytically solved with closed-form solutions in advance, hence the large number of plane parameters can be completely removed from the resultant optimization. Such an elimination of plane parameters is analogous to the well-studied separable least-squares problem [55]–[58] in general, but is specifically designed for the LiDAR BA problem. A prominent advantage of the feature elimination is the significant reduction of optimization dimension, which poses a fundamental difference from all the previous planar adjustment methods [34, 51]–[53] and visual bundle adjustment methods [50]. The feature elimination also removed the various issues caused by plane representation in the optimization, such as the normal constraints in the Hesse normal representation $(\mathbf{n}, d)$ [34], singularity issue in the closest-point (CP) representation $\mathbf{n}d$ [51]–[53] and over-parameterization issue in the quaternion representation [48]. With the feature elimination, BALM further proved that the point-to-plane (or edge) distance is essentially the eigenvalues of the covariance matrix used in [33], thus unifying the two metrics in [34] and [33]. While both BALM and [33] eliminate the feature from the BA optimization, [33] uses a gradient method to solve this optimization, which leads to very slow convergence as reviewed above. In contrast, BALM [32] derived the second order derivatives of the cost function and developed a LM-like second-order solver. The developed solver requires significantly less iterations to converge, achieving real-time sliding window optimization when integrated to LOAM [18]. A further advantage of BALM against previous method [33, 34, 53] is that the whole framework is naturally extendable to edge features besides plane features.

A major drawback of the BALM [32] is that the evaluation of the second-order derivatives including Jacobian and Hessian

requires to enumerate each individual lidar point, leading to a computational complexity of $O(N^2)$ where N is the number of points [52]. Consequently, the method is hard to be used in large-scale problems where the lidar points are huge in number. This problem is partially addressed in [35], which aggregates all points associated to the same plane feature in a scan in the scan local frame. However, to ensure convergence, [35] modifies the cost function by including an extra heuristic penalty term, which is not a true representation of the map consistency. Moreover, the cost function in [35] still involves the plane feature similar to [33, 34, 51]–[53]. To lower the computation load caused by optimizing the large number of feature parameters, the method further fixes the feature parameters in the optimisation, which could slow down the optimisation speed.

Our BA formulation in this paper is based on BALM [32], hence inheriting the fundamental feature elimination advantage when compared to [34, 51]–[53] and the fast convergence advantage when compared to [33]. To address the computational complexity of $O(N^2)$ in BALM, we adopt and formalize the concept of *point cluster*, which fundamentally eliminates the enumeration of each individual point in the evaluation of the cost function, Jacobian and Hessian matrix. Consequently, the computational complexity is irrelevant to both the feature dimension (similar to BALM [32]) and the point number (similar to [34, 51]). The point cluster in our method is similar to the point aggregation used in [35] (and also used in [33, 34]), but the overall BA formulation is fundamentally different: (1) it minimizes the true map consistency (the point-to-plane distance) without trading off with any other heuristic penalty; and (2) it performs exact feature elimination with rigorous proof instead of empirical fixation. Based on these nice theoretical results, we develop an efficient second-order solver, termed as BALM2.0. Besides solving the nominal lidar poses, the solver also estimates the uncertainty of the estimated lidar pose by leveraging the second-order derivative information, which is another new contribution compared with existing works.

III. BUNDLE ADJUSTMENT FORMULATION AND OPTIMIZATION

In this chapter, we derive our BA formulation and optimisation. First, following [32], we formulate the BA as minimizing the the point-to-plane (or point-to-edge) distance (Sec. III-A) and show that the feature parameters can be eliminated from the formulated optimisation (Sec. III-B). Then, we introduce the point cluster in Sec. III-C, based on which the first and second order derivatives are derived in Sec. III-D. Based on these theoretical results, we present our second-order solver in Sec. III-E. Finally, in Sec. III-F, we show how to estimate the uncertainty of the BA solution. Throughout this paper, we use notations summarized in Table I or otherwise specified in the context.

A. BA formulation

Shown in Fig. 1, assume there are M_f features, each denoted by parameter π_i ($i = 1, \dots, M_f$), observed by M_p

TABLE I
NOMENCLATURES

Notation	Explanation
$\mathbb{R}^{m \times n}$	The set of $m \times n$ real matrices.
$\mathbb{S}^{m \times m}$	The set of $m \times m$ symmetric matrices.
$\boxplus$	The encapsulated “boxplus” operations on manifold.
$(\cdot)_f$	The value of $(\cdot)$ expressed in lidar local frame.
$(\cdot)$	The value of $(\cdot)$ expressed in global frame.
$[\cdot]$	The skew symmetric matrix of $(\cdot)$.
$\exp(\cdot)$	Exponential of $(\cdot)$, which could be a matrix.
$\mathbb{I}_{i=j}$	Indicator function which is equal to “1” if $i = j$, otherwise equal to “0”.
M_f, M_p	The number of features and poses, respectively.
i, j, k	The indexes of features, poses and points, respectively.
l	The index of eigenvalue and eigenvector of a matrix.
p, q	The indexes of (block) row and column in a matrix.
$\mathbf{e}_l$	The vector in $\mathbb{R}^4$ with all elements being zeros except the l -th element being one ($l \in \{1, 2, 3, 4\}$).
$\mathbf{S}_p$	$\mathbf{S}_p = [\mathbf{I}_{3 \times 3} \quad \mathbf{0}_{3 \times 1}] \in \mathbb{R}^{3 \times 4}$.
$\mathbf{S}_v$	$\mathbf{S}_v = [\mathbf{0}_{1 \times 3} \quad 1] \in \mathbb{R}^{1 \times 4}$.
$\mathbf{E}_{kl}$	$\mathbf{E}_{kl} = \mathbf{e}_k \mathbf{e}_l^T + \mathbf{e}_l \mathbf{e}_k^T \in \mathbb{S}^{4 \times 4}$, $k, l \in \{1, 2, 3, 4\}$.

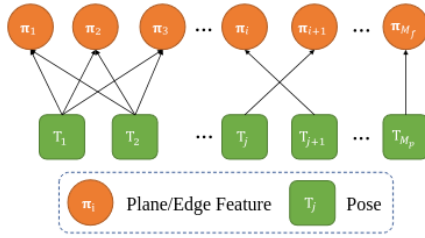


Fig. 1. Factor graph representation of the bundle adjustment formulation.

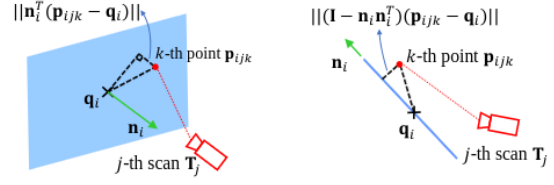
lidar poses, each denoted by $\mathbf{T}_j = (\mathbf{R}_j, \mathbf{t}_j)$ ($j = 1, \dots, M_p$), the bundle adjustment refers to simultaneously determining all the lidar poses (denoted by $\mathbf{T} = (\mathbf{T}_1, \dots, \mathbf{T}_{M_p})$) and feature parameters (denoted by $\boldsymbol{\pi} = (\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_{M_f})$), such that reconstructed map agrees with the lidar measurements to the best extent. Denote $c(\boldsymbol{\pi}_i, \mathbf{T})$ the map consistency due to the i -th feature, a straightforward BA formulation is

$$\min_{\mathbf{T}, \boldsymbol{\pi}} \left(\sum_{i=1}^{M_f} c(\boldsymbol{\pi}_i, \mathbf{T}) \right). \quad (1)$$

In our BA formulation, we make use of plane and edge features that are often abundant in lidar point cloud and minimize the natural Euclidean distance between each measured raw lidar point and its corresponding plane or edge feature. Specifically, assume a total number of N_{ij} lidar points are measured on the i -th feature at the j -th lidar pose, each denoted by $\mathbf{p}_{f_{ijk}}$ ($k = 1, \dots, N_{ij}$). Its predicted location in the global frame is

$$\mathbf{p}_{ijk} = \mathbf{R}_j \mathbf{p}_{f_{ijk}} + \mathbf{t}_j. \quad (2)$$

For a plane feature, it is parameterized by $\boldsymbol{\pi}_i = (\mathbf{n}_i, \mathbf{q}_i)$ with $\mathbf{n}_i$ the plane normal vector and $\mathbf{q}_i$ an arbitrary point on the plane, both in the global frame (see Fig. 2 (a)). Then, the Euclidean distance between a measured point $\mathbf{p}_{f_{ijk}}$ to the plane is $\|\mathbf{n}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2$. Aggregating the distance for all



(a) The i -th plane feature $\boldsymbol{\pi}_i = (\mathbf{n}_i, \mathbf{q}_i)$ (b) The i -th edge feature $\boldsymbol{\pi}_i = (\mathbf{n}_i, \mathbf{q}_i)$

Fig. 2. Plane and edge features used in the lidar BA. (a) The plane formulation. $\mathbf{q}_i$ is a point in the plane and $\mathbf{n}_i$ is the plane normal. (b) The line formulation. $\mathbf{q}_i$ is a point on the edge and $\mathbf{n}_i$ is the edge direction.

points observed in all poses leads to the total map consistency corresponding to this plane feature:

$$c(\boldsymbol{\pi}_i, \mathbf{T}) = \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|\mathbf{n}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2^2 \quad (3)$$

where $N_i = \sum_{j=1}^{M_p} N_{ij}$ is the total number of lidar points observed on the plane feature by all poses.

For an edge feature, it is parameterized by $\boldsymbol{\pi}_i = (\mathbf{n}_i, \mathbf{q}_i)$ with $\mathbf{n}_i$ the edge direction vector and $\mathbf{q}_i$ an arbitrary point on the edge, both in the global frame (see Fig. 2 (b)). Then, the Euclidean distance between a measured point $\mathbf{p}_{f_{ijk}}$ to the edge is $\|(\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T) (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2$. Aggregating the distance for all points observed in all poses leads to the total map consistency corresponding to this edge feature:

$$c(\boldsymbol{\pi}_i, \mathbf{T}) = \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|(\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T) (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2^2 \quad (4)$$

where $N_i = \sum_{j=1}^{M_p} N_{ij}$ is the total number of lidar points observed on the edge feature by all poses.

B. Elimination of feature parameters

In this section, we show that in the BA optimization (1), the feature parameter $\boldsymbol{\pi}$ can really be solved with a closed-form solution. The key observation is that one cost item $c(\boldsymbol{\pi}_i, \mathbf{T})$ depends solely on one feature parameter, so that the feature parameter can be optimized independently. Concretely,

$$\begin{aligned} \min_{\mathbf{T}, \boldsymbol{\pi}} \left(\sum_{i=1}^{M_f} c(\boldsymbol{\pi}_i, \mathbf{T}) \right) &= \min_{\mathbf{T}} \left(\min_{\boldsymbol{\pi}} \left(\sum_{i=1}^{M_f} c(\boldsymbol{\pi}_i, \mathbf{T}) \right) \right) \\ &= \min_{\mathbf{T}} \left(\sum_{i=1}^{M_f} \min_{\boldsymbol{\pi}_i} c(\boldsymbol{\pi}_i, \mathbf{T}) \right). \end{aligned} \quad (5)$$

In case of a plane feature, we substitute (3) into $c(\boldsymbol{\pi}_i, \mathbf{T})$:

$$\begin{aligned} \min_{\boldsymbol{\pi}_i} c(\boldsymbol{\pi}_i, \mathbf{T}) &= \min_{\boldsymbol{\pi}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|\mathbf{n}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2^2 \right) \\ &= \lambda_3(\mathbf{A}_i), \text{ when } \mathbf{n}_i^* = \mathbf{u}_3(\mathbf{A}_i), \mathbf{q}_i^* = \bar{\mathbf{p}}_i \end{aligned} \quad (6)$$

where $\lambda_l(\mathbf{A}_i)$ denotes the l -th largest eigenvalue of matrix $\mathbf{A}_i$, $\mathbf{u}_l(\mathbf{A}_i)$ denotes the corresponding eigenvector, the matrix $\mathbf{A}_i$ and vector $\bar{\mathbf{p}}_i$ are defined as:

$$\begin{aligned}\mathbf{A}_i &\triangleq \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)(\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)^T, \\ \bar{\mathbf{p}}_i &\triangleq \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \mathbf{p}_{ijk}.\end{aligned}\quad (7)$$

The proof will be given in Supplementary III-A [59]. Note that the optimal solution $\mathbf{q}_i^*$ in (6) is not unique, any deviation from $\mathbf{q}_i^*$ along a direction perpendicular to $\mathbf{n}_i^*$ will equally serve the optimal solution. However, these equivalent optimal solution will not change the plane nor the optimal cost (hence the results that follow next). Indeed, the point $\mathbf{q}_i^*$ could be an arbitrary point on the plane as it is defined to be.

In case of an edge feature, we substitute (4) into $c(\pi_i, \mathbf{T})$:

$$\begin{aligned}\min_{\pi_i} c(\pi_i, \mathbf{T}) &= \min_{\pi_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|(\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T)(\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2^2 \right) \\ &= \lambda_2(\mathbf{A}_i) + \lambda_3(\mathbf{A}_i); \text{ when } \mathbf{n}_i^* = \mathbf{u}_1(\mathbf{A}_i), \mathbf{q}_i^* = \bar{\mathbf{p}}_i.\end{aligned}\quad (8)$$

Again, the optimal solution $\mathbf{q}_i^*$ in (8) is not unique, any deviation from $\mathbf{q}_i^*$ along the direction $\mathbf{n}_i^*$ will equally serve the optimal solution. However, these equivalent optimal solution will not change the edge nor the optimal cost (hence the results that follow next).

As can be seen from (6) and (8), the parameter π_i for each feature, either it is a plane or edge, can be analytically solved and hence removed from the BA optimization process. Consequently, the original BA optimization in (1) reduces to

$$\min_{\mathbf{T}} \left(\sum_{i=1}^{M_f} \lambda_l(\mathbf{A}_i) \right) \quad (9)$$

where $l \in \{2, 3\}$ and we omitted the exact number of eigenvalues in the cost function for brevity.

Note that the matrix $\mathbf{A}_i$ in (9) depends on the lidar pose $\mathbf{T}$ since each involved point $\mathbf{p}_{ijk}$ depends on the pose (see (7) and (2)). Hence the decision variables of the resultant optimization in (9) involve the lidar pose $\mathbf{T}$ only, which dramatically reduces the optimization dimension (hence computation time).

C. Point cluster

With the feature parameters eliminated, another difficulty remaining in the BA optimization (9) is that the evaluation of matrix $\mathbf{A}_i$ (and its Jacobian or Hessian necessary for developing a numerical solver) requires to enumerate every point observed at each lidar pose. Such an enumeration is extremely computationally expensive due to the large number of points in a lidar scan. In this section, we show such point enumeration can be avoided by *point cluster*, which is detailed as follows.

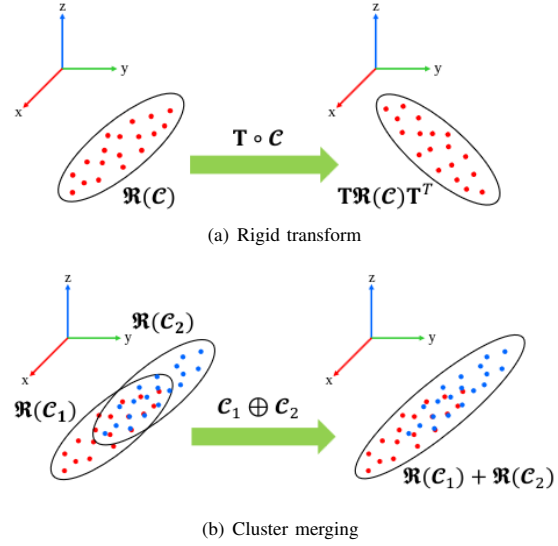


Fig. 3. Two operations on point cluster (a) Rigid transform (b) Cluster merging.

A *point cluster* is a finite point set denoted by set $\mathcal{C} = \{\mathbf{p}_k \in \mathbb{R}^3 | k = 1, \dots, n\}$, the corresponding *point cluster coordinate*, denoted as $\mathcal{R}(\mathcal{C})$, is defined as:

$$\begin{aligned}\mathcal{R}(\mathcal{C}) &\triangleq \sum_{k=1}^n \begin{bmatrix} \mathbf{p}_k \\ 1 \end{bmatrix} \begin{bmatrix} \mathbf{p}_k^T & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & n \end{bmatrix} \in \mathbb{S}^{4 \times 4} \\ \mathbf{P} &= \sum_{k=1}^n \mathbf{p}_k \mathbf{p}_k^T, \quad \mathbf{v} = \sum_{k=1}^n \mathbf{p}_k,\end{aligned}\quad (10)$$

where $\mathbb{S}^{4 \times 4}$ denotes the set of 4×4 symmetric matrix.

A *point cluster* can be thought as a generalized point, for which a rigid transformation could be applied. Similarly, we can define rigid transformation on a point cluster as follows.

Definition 1. (Rigid transform) Given a point cluster with point collection $\mathcal{C} = \{\mathbf{p}_k \in \mathbb{R}^3 | k = 1, \dots, n\}$ and a pose $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \in SE(3)$. The rigid transformation of the point cluster $\mathcal{C}$, denoted by $\mathbf{T} \circ \mathcal{C}$, is defined as

$$\mathbf{T} \circ \mathcal{C} \triangleq \{\mathbf{R} \mathbf{p}_k + \mathbf{t} \in \mathbb{R}^3 | k = 1, \dots, n\}. \quad (11)$$

Besides rigid transformation, we also define cluster merging operation, as follows.

Definition 2. (Cluster merging) Given two point clusters with point collections $\mathcal{C}_1 = \{\mathbf{p}_k^1 \in \mathbb{R}^3 | k = 1, \dots, n_1\}$ and $\mathcal{C}_2 = \{\mathbf{p}_k^2 \in \mathbb{R}^3 | k = 1, \dots, n_2\}$ in the same reference frame, respectively. The merged cluster, denoted by $\mathcal{C}_1 \oplus \mathcal{C}_2$, is defined as

$$\mathcal{C}_1 \oplus \mathcal{C}_2 \triangleq \{\mathbf{p}_k^l \in \mathbb{R}^3 | l = 1, 2; k = 1, \dots, n_i\}. \quad (12)$$

Next we will show that the two operations defined above can be fully represented by their point cluster coordinates.

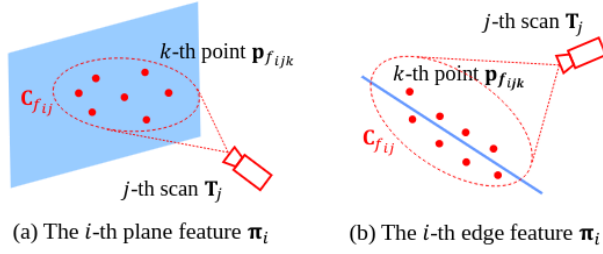


Fig. 4. For the i -th feature (either plane or edge), all points observed at the j -th pose are clustered as a point cluster and is represented by $\mathcal{C}_{f_{ij}}$ in its local frame.

Theorem 1. Given a point cluster $\mathcal{C}$ and a pose $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \in SE(3)$. The rigid transformation of the point cluster satisfies

$$\mathfrak{R}(\mathbf{T} \circ \mathcal{C}) = \mathbf{T} \mathfrak{R}(\mathcal{C}) \mathbf{T}^T \quad (13)$$

Proof. See Supplementary III-B [59]. $\square$

Theorem 2. Given two point clusters $\mathcal{C}_1$ and $\mathcal{C}_2$ in the same reference frame. The merged cluster satisfies

$$\mathfrak{R}(\mathcal{C}_1 \oplus \mathcal{C}_2) = \mathfrak{R}(\mathcal{C}_1) + \mathfrak{R}(\mathcal{C}_2) \quad (14)$$

Proof. See Supplementary III-C [59]. $\square$

As can be seen, rigid transformation and cluster merging operations on point clusters can be represented by usual matrix multiplication and addition on the point cluster coordinates. A visual illustration of the two operations and their coordinate representations are shown in Fig. 3. These results are crucially important: Theorem 1 indicates that the point cluster can be constructed in one frame (e.g., local lidar frame) and transformed to another (e.g., the global frame) without enumerating each individual points; Theorem 2 indicates that two (and by induction more) point clusters can be further merged to form a new point cluster. A particular case of Theorem 2 is when the second point cluster contains a single point, indicating that the point cluster can be constructed incrementally as lidar points arrives sequentially.

Remark 1. The concept of point cluster and its two operations are not new and have been used in previous works such as [33]–[35]. In this paper, we formalized this concept by 1) introducing the point cluster coordinate composing of $\mathbf{P}$, $\mathbf{v}$, and n as in (10), 2) formalizing the two operations: rigid transform and cluster merging, and 3) explicitly showing the relation between point cluster operations and their coordinates.

Remark 2. A point set and its coordinate is not a one-to-one mapping. While it is obvious that the coordinate is uniquely determined from the point set as shown in (10), the reverse way does not hold: the point set cannot be recovered from its coordinate uniquely. Since different point sets may lead to the same coordinate, the raw points must be saved if a re-clustering is needed.

Now, we apply the point cluster to the BA problem concerned in this paper. To start with, we group all points on

the same feature as a point cluster. For example, the point cluster for the i -th feature is $\mathcal{C}_i \triangleq \{\mathbf{p}_{ijk} | j = 1, \dots, M_p, k = 1, \dots, N_{ij}\}$. Denote $\mathbf{C}_i$ the coordinate of the point cluster, following (10), we obtain

$$\mathbf{C}_i = \mathfrak{R}(\mathcal{C}_i) \triangleq \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \begin{bmatrix} \mathbf{p}_{ijk} \\ 1 \end{bmatrix} \begin{bmatrix} \mathbf{p}_{ijk}^T & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{P}_i & \mathbf{v}_i \\ \mathbf{v}_i^T & N_i \end{bmatrix}$$

$$\mathbf{P}_i = \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \mathbf{p}_{ijk} \mathbf{p}_{ijk}^T, \quad \mathbf{v}_i = \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \mathbf{p}_{ijk} \quad (15)$$

A key result we show now is that this point cluster coordinate $\mathbf{C}_i$ is completely sufficient to represent the matrix $\mathbf{A}_i$ required in the BA optimization (9). According to (7), we have

$$\mathbf{A}_i = \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)(\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)^T$$

$$= \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} \mathbf{p}_{ijk}^T) - \bar{\mathbf{p}}_i \bar{\mathbf{p}}_i^T$$

$$= \frac{1}{N_i} \mathbf{P}_i - \frac{1}{N_i^2} \mathbf{v}_i \mathbf{v}_i^T \triangleq \mathbf{A}(\mathbf{C}_i) \quad (16)$$

where we denote $\mathbf{A}_i$ as a function of $\mathbf{C}_i$ since the $\mathbf{A}_i$ is fully represented (and uniquely determined) by $\mathbf{C}_i$. We slightly abuse the notation here by denoting the function as $\mathbf{A}(\cdot)$.

On the other hand, since the point set $\mathcal{C}_i$ is defined on points in the global frame, the coordinate $\mathbf{C}_i$ is dependent on the lidar pose, which remains to be optimized. To explicitly parameterize the lidar pose, we note that

$$\mathcal{C}_i \triangleq \{\mathbf{p}_{ijk} | j = 1, \dots, M_p, k = 1, \dots, N_{ij}\} \quad (17)$$

$$= \bigcup_{j=1}^{M_p} \{\mathbf{p}_{ijk} | k = 1, \dots, N_{ij}\} \quad (18)$$

$$\stackrel{\text{Def.2}}{=} \bigoplus_{j=1}^{M_p} \mathcal{C}_{ij}, \quad \mathcal{C}_{ij} \triangleq \{\mathbf{p}_{ijk} | k = 1, \dots, N_{ij}\} \quad (19)$$

$$\stackrel{\text{Def.1}}{=} \bigoplus_{j=1}^{M_p} (\mathbf{T}_j \circ \mathcal{C}_{f_{ij}}), \quad \mathcal{C}_{f_{ij}} \triangleq \{\mathbf{p}_{f_{ijk}} | k = 1, \dots, N_{ij}\} \quad (20)$$

where $\mathbf{p}_{f_{ijk}}$ is a point represented in the lidar local frame (see (2)), $\mathcal{C}_{ij}$ is the point set that is composed of all points on the i -th feature (either plane or edge) observed at the j -th lidar pose, and $\mathcal{C}_{f_{ij}}$ is the same point set as $\mathcal{C}_{ij}$, but represented in the lidar local frame (see Fig. 4).

The relation between the point cluster $\mathcal{C}_i$ and $\mathcal{C}_{f_{ij}}$ shown in (20) will lead to a relation between their coordinates $\mathbf{C}_i$ and $\mathbf{C}_{f_{ij}}$ as below:

$$\mathbf{C}_i \stackrel{\text{Thm.2}}{=} \sum_{j=1}^{M_p} \mathbf{C}_{ij} \stackrel{\text{Thm.1}}{=} \sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_{f_{ij}} \mathbf{T}_j^T \quad (21)$$

As a result, the BA optimization in (9) further reduces to

$$\min_{\mathbf{T}_j \in SE(3), \forall j} \underbrace{\left(\sum_{i=1}^{M_f} \lambda_i \left(\mathbf{A} \left(\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_{f_{ij}} \mathbf{T}_j^T \right) \right) \right)}_{c(\mathbf{T})} \quad (22)$$

where the function $\mathbf{A}(\cdot)$ is defined in (16). Note that the cost function in (22) only requires the knowledge of point cluster

coordinate $\mathbf{C}_{f_{ij}}$ without enumerating each individual points. The coordinate $\mathbf{C}_{f_{ij}}$ is computed as (following (10)):

$$\mathbf{C}_{f_{ij}} = \begin{bmatrix} \mathbf{P}_{f_{ij}} & \mathbf{v}_{f_{ij}} \\ \mathbf{v}_{f_{ij}}^T & N_{ij} \end{bmatrix}$$

$$\mathbf{P}_{f_{ij}} = \sum_{k=1}^{N_{ij}} \mathbf{p}_{f_{ijk}} \mathbf{p}_{f_{ijk}}^T, \quad \mathbf{v}_{f_{ij}} = \sum_{k=1}^{N_{ij}} \mathbf{p}_{f_{ijk}} \quad (23)$$

which can be constructed during the feature association stage before the optimization. In particular, if the j -th pose observes no point on the i -th feature, $\mathbf{C}_{f_{ij}} = \mathbf{0}_{4 \times 4}$, which will naturally remove the dependence on the j -th pose for the i -th cost item as shown in (22).

Theorem 3. Given a matrix function $\mathbf{A}(\mathbf{C}) \triangleq \frac{1}{N} \mathbf{P} - \frac{1}{N^2} \mathbf{v} \mathbf{v}^T$ with $\mathbf{C} = \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} \in \mathbb{S}^{4 \times 4}$, $\lambda_l(\mathbf{A})$ denotes the l -th largest eigenvalue of $\mathbf{A}$, then $\lambda_l(\mathbf{A}(\mathbf{C}))$ is invariant to any rigid transformation $\mathbf{T}_0 \in SE(3)$. That is,

$$\lambda_l(\mathbf{A}(\mathbf{T}_0 \mathbf{C} \mathbf{T}_0^T)) = \lambda_l(\mathbf{A}(\mathbf{C})), \forall \mathbf{T}_0 \in SE(3). \quad (24)$$

Proof. See Supplementary III-D [59]. $\square$

Theorem 3 implies that left multiplying all poses $\mathbf{T}_j, \forall j$, by the same transform $\mathbf{T}_0$ does not change the optimization at all. That is, the BA optimization is invariant to the change of the global reference frame, which is the well-known gauge freedom in a bundle adjustment problem.

D. First and Second Order Derivatives

As shown in the previous section, the BA optimization problem in (22) is completely equivalent to the original formulation (1), where each cost item standards for the squared Euclidean distance from a point to a plane (or edge). This squared distance is essentially a quadratic optimization, which requires the knowledge of the second-order information of the cost function for efficient solving. In this section, we derive such first and second order derivatives. Without loss of generality, we only discuss the i -th feature, which contributes a cost item in the form of

$$c_i(\mathbf{T}) = \lambda_l \left(\mathbf{A} \left(\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_{f_{ij}} \mathbf{T}_j^T \right) \right) \quad (25)$$

with $\mathbf{C}_{f_{ij}} \in \mathbb{R}^{4 \times 4}$ being a pre-computed matrix (see (23)).

To derive the derivative of the cost item (25) w.r.t. the pose $\mathbf{T}_j$, which is an element of the Special Euclidean group $SE(3)$, we parameterize its perturbation by a special addition, called boxplus ($\boxplus$ -operation). For the pose vector $\mathbf{T} = (\cdots, \mathbf{T}_j, \cdots)$, we define the $\boxplus$ operation as below

$$\mathbf{T} \boxplus \delta \mathbf{T} \triangleq (\cdots, \mathbf{T}_j \boxplus \delta \mathbf{T}_j, \cdots), \quad (26)$$

$$\mathbf{T}_j \boxplus \delta \mathbf{T}_j \triangleq (\exp([\delta \phi_j]) \mathbf{R}_j, \delta \mathbf{t}_j + \exp([\delta \phi_j]) \mathbf{t}_j), \quad (27)$$

where $\delta \mathbf{T} \triangleq (\cdots, \delta \mathbf{T}_j, \cdots) \in \mathbb{R}^{6M_p}$ with $\delta \mathbf{T}_j \triangleq (\delta \phi_j, \delta \mathbf{t}_j) \in \mathbb{R}^6, \forall j \in 1, \cdots, M_p$, is the perturbation on the pose vector.

For a scalar function $f(\mathbf{T}) : SE(3) \times \cdots \times SE(3) \mapsto \mathbb{R}$, denote $\left(\frac{\partial f(\mathbf{T})}{\partial \mathbf{T}} \right) (\mathbf{T}_0)$ its first-order derivative and

$\left(\frac{\partial^2 f(\mathbf{T})}{\partial \mathbf{T}^2} \right) (\mathbf{T}_0)$ its second-order derivative, both at a chosen point of the input $\mathbf{T}_0$. The $\boxplus$ operation enables us to parameterize the input of function $f(\cdot)$, $\mathbf{T}$, by its perturbation $\delta \mathbf{T}$ from a given point $\mathbf{T}_0$: $\mathbf{T} = \mathbf{T}_0 \boxplus \delta \mathbf{T}$. Since the map between $\mathbf{T}$ and $\delta \mathbf{T}$ is bijective if $\|\delta \phi_j\| < \pi, \forall j$, the scalar function $f(\mathbf{T})$ in terms of $\mathbf{T}$ can be equivalently written as a function $f(\mathbf{T}_0 \boxplus \delta \mathbf{T})$ in terms of $\delta \mathbf{T}$. As a consequence, the derivatives of $f(\mathbf{T})$ w.r.t. $\mathbf{T}$ at the point $\mathbf{T}_0$ can be defined as the derivatives of $f(\mathbf{T}_0 \boxplus \delta \mathbf{T})$ w.r.t. $\delta \mathbf{T}$ at zero, where the latter is a normal derivative w.r.t. Euclidean vectors:

$$\left(\frac{\partial f(\mathbf{T})}{\partial \mathbf{T}} \right) (\mathbf{T}_0) \triangleq \left(\frac{\partial f(\mathbf{T}_0 \boxplus \delta \mathbf{T})}{\partial \delta \mathbf{T}} \right) (\mathbf{0}) \quad (28)$$

$$\left(\frac{\partial^2 f(\mathbf{T})}{\partial \mathbf{T}^2} \right) (\mathbf{T}_0) \triangleq \left(\frac{\partial}{\partial \delta \mathbf{T}} \left(\frac{\partial f(\mathbf{T}_0 \boxplus \delta \mathbf{T})}{\partial \delta \mathbf{T}} \right) \right) (\mathbf{0}) \quad (29)$$

$$\forall \mathbf{T}_0 \in SE(3) \times \cdots \times SE(3).$$

In the following discussion, we use $\mathbf{T}$ as the reference point to replace $\mathbf{T}_0$ in the derivatives and omit it for the sake of notation simplification.

Based on the derivatives defined in (28) and (29), we have the following results for the first and second-order derivatives of the cost item (25).

Theorem 4. Given

- (1) Matrices $\mathbf{C}_j = \begin{bmatrix} \mathbf{P}_j & \mathbf{v}_j \\ \mathbf{v}_j^T & N_j \end{bmatrix} \in \mathbb{S}^{4 \times 4}, j = 1, \cdots, M_p$;
- (2) Poses $\mathbf{T}_j \in SE(3), j = 1, \cdots, M_p$;
- (3) A matrix $\mathbf{C} = \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} \triangleq \sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \in \mathbb{S}^{4 \times 4}$, which is the aggregation of $\mathbf{C}_j$, and a matrix function $\mathbf{A}(\mathbf{C}) \triangleq \frac{1}{N} \mathbf{P} - \frac{1}{N^2} \mathbf{v} \mathbf{v}^T \in \mathbb{S}^{3 \times 3}$; and
- (4) A function $\lambda_l(\mathbf{A}(\mathbf{C}))$, $\lambda_l(\mathbf{A})$ denotes the l -th ($l = 1, 2, 3$) largest eigenvalue of $\mathbf{A}$ with corresponding eigenvector $\mathbf{u}_l$;

The Jacobian matrix $\mathbf{J}_l$ and Hessian matrix $\mathbf{H}_l$ of the function $\lambda_l(\mathbf{A}(\mathbf{C}))$ with respect to the poses $\mathbf{T}$ are

$$\mathbf{J}_l = \mathbf{g}_{ll} \in \mathbb{R}^{1 \times 6M_p}, \quad (30)$$

$$\mathbf{H}_l = \mathbf{W}_l + \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \mathbf{g}_{kl}^T \mathbf{g}_{kl} \in \mathbb{R}^{6M_p \times 6M_p}, \quad (31)$$

where $\mathbf{g}_{ll}$ is $\mathbf{g}_{kl}$ with $k = l$. $\mathbf{g}_{kl}$ and $\mathbf{W}_l$ are matrices partitioned as

$$\mathbf{g}_{kl} = [\cdots, \mathbf{g}_{kl}^j, \cdots] \in \mathbb{R}^{1 \times 6M_p}, \quad (32)$$

$$\mathbf{W}_l = \begin{bmatrix} \vdots & & \\ \cdots & \mathbf{W}_l^{ij} & \cdots \\ \vdots & & \end{bmatrix} \in \mathbb{R}^{6M_p \times 6M_p}, \quad (33)$$

with block elements $\mathbf{g}_{kl}^j \in \mathbb{R}^{1 \times 6}, \mathbf{W}_l^{ij} \in \mathbb{R}^{6 \times 6}, \forall i, j \in$

$\{1, \dots, M_p\}$ defined as

$$\mathbf{g}_{kl}^j = \frac{1}{N} \mathbf{u}_l^T \mathbf{S}_p (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_k^T + \frac{1}{N} \mathbf{u}_k^T \mathbf{S}_p (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T, \quad (34)$$

$$\mathbf{W}_l^{ij} = -\frac{2}{N^2} \mathbf{V}_l \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \mathbb{1}_{i=j} \cdot \left(\frac{2}{N} \mathbf{V}_l \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \begin{bmatrix} \mathbf{K}_l^j & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \right), \quad (35)$$

$$\mathbf{K}_l^j = \frac{1}{N} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l] [\mathbf{u}_l] + \frac{1}{N} [\mathbf{u}_l] [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l], \quad (36)$$

$$\mathbf{V}_l = \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_l \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}, \quad (37)$$

$$\mathbf{S}_p = [\mathbf{I}_{3 \times 3} \quad \mathbf{0}_{3 \times 1}] \quad \mathbb{1}_{i=j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}. \quad (38)$$

Proof. See Supplementary III-E [59]. $\square$

Corollary 4.1. *The Jacobian matrix $\mathbf{J}_l$ and Hessian matrix $\mathbf{H}_l$ in Theorem 4 satisfy that, for any $l = 1, 2, 3$,*

$$\mathbf{J}_l \cdot \delta \mathbf{T} = 0, \quad \delta \mathbf{T}^T \cdot \mathbf{H}_l \cdot \delta \mathbf{T} = 0, \quad \forall \delta \mathbf{T} \in \mathcal{W} \triangleq \left\{ \begin{bmatrix} \mathbf{w} \\ \vdots \\ \mathbf{w} \end{bmatrix} \mid \forall \mathbf{w} \in \mathbb{R}^6 \right\}, \quad (39)$$

$$\mathbf{J}_l^j = \mathbf{0}_{1 \times 6}, \quad \text{if } \mathbf{C}_j = \mathbf{0}, \quad (40)$$

$$\mathbf{H}_l^{ij} = \mathbf{0}_{6 \times 6}, \quad \text{if } \mathbf{C}_i = \mathbf{0} \text{ or } \mathbf{C}_j = \mathbf{0}, \quad (41)$$

where $\mathbf{J}_l^j$ is the j -th column block of $\mathbf{J}_l$ and $\mathbf{H}_l^{ij}$ is the i -th row, j -th column block of $\mathbf{H}_l$.

Proof. See Supplementary III-F [59]. $\square$

Remark 3. (39) implies that the Jacobian and Hessian matrices have null space containing the space spanned by $\mathcal{W}$. This essentially means that the cost function (25) in the BA optimization does not change along the direction where all the poses are perturbed by the same quantity $\mathbf{w}$, which agrees with gauge freedom stated in Theorem 3.

Remark 4. The results in (40, 41) imply that the blocks in Jacobian and Hessian matrices are zeros and hence their computation can be saved if any of the related poses does not observe the current feature (i.e., $\mathbf{C}_i = \mathbf{0}$ or $\mathbf{C}_j = \mathbf{0}$). This sparse structure could save much computation time if a feature is observed only by a sparse set of poses.

Remark 5. The derivatives in Theorem 4 are obtained based on the pose perturbation defined in (26), which multiplies the perturbation $\delta \mathbf{T}$ on the left of the current pose (i.e., a perturbation in the global frame). If other perturbation (denoted by $\delta \check{\mathbf{T}}$) is preferred (e.g., a perturbation in the local frame to integrate with other measurements such as IMU pre-integration), where $\delta \mathbf{T} = \mathbf{L} \delta \check{\mathbf{T}}$ with $\mathbf{L}$ the Jacobian between the two perturbation parameterization, its first and second order derivatives can be computed as $\check{\mathbf{J}} = \mathbf{J} \cdot \mathbf{L}$ and $\check{\mathbf{H}} = \mathbf{L}^T \cdot \mathbf{H} \cdot \mathbf{L}$, respectively. It can be seen that $\check{\mathbf{J}}$ and $\check{\mathbf{H}}$ preserves a nullspace of $\mathbf{L} \cdot \mathcal{W}$ with $\mathcal{W}$ defined in (39).

E. Second Order Solver

The Jacobian and Hessian matrix from Theorem 4 are computed for one cost item (25) that corresponds to one feature in the space. Denote $\mathbf{J}_i, \mathbf{H}_i$ the Jacobian and Hessian matrix for the i -th feature (or cost item), to determine the incremental update $\Delta \mathbf{T}$, we make use of the second order approximation of the total cost function $c(\mathbf{T})$ in (22):

$$c(\mathbf{T} \boxplus \Delta \mathbf{T}) \approx c(\mathbf{T}) + \mathbf{J} \Delta \mathbf{T} + \frac{1}{2} \Delta \mathbf{T}^T \mathbf{H} \Delta \mathbf{T} \quad (42)$$

where $\mathbf{J} = \sum_{i=1}^{M_f} \mathbf{J}_i, \mathbf{H} = \sum_{i=1}^{M_f} \mathbf{H}_i$. For any $\mathbf{d} \in \mathcal{W}$, since $\mathbf{J}_i \mathbf{d} = 0, \mathbf{d}^T \mathbf{H}_i \mathbf{d} = 0, \forall i$, we have $\mathbf{J} \mathbf{d} = 0$ and $\mathbf{d}^T \mathbf{H} \mathbf{d} = 0, \forall i$, which means that any additional update along $\mathbf{d} \in \mathcal{W}$ does not change the approximation at all. One way to resolve the gauge freedom is fixing the first pose at its initial value throughout the optimization. That is, setting $\Delta \mathbf{T}_1 = \mathbf{0}$ in (42). Then, setting the differentiation of the cost approximation in (42) w.r.t. $\Delta \mathbf{T}$ (excluding $\Delta \mathbf{T}_1$) to zero leads to the optimal update $\Delta \mathbf{T}^*$:

$$\Delta \mathbf{T}^* = -(\mathbf{H} + \mu \mathbf{I})^{-1} \mathbf{J}^T, \quad (43)$$

where we used a Levenberg-Marquardt (LM) algorithm-like method to re-weight the gradient and Newton's direction by the damping parameter μ . The complete algorithm is summarized in Supplementary (Algorithm 1) with time analyses detailed in Supplementary (Section IV). Overall, the solver has a complexity of $O(M_f M_p + M_f M_p^2 + M_p^3)$, which is linear to the number of feature M_f , irrelevant to the number of points N , and cubic to the number of pose M_p . The term $M_f M_p$ and $M_f M_p^2$ are due to the calculation of Jacobian and Hessian, respectively and the term M_p^3 is due to (43).

F. Covariance Estimation

Assume the solver converges to an optimal pose $\mathbf{T}^*$, it is often useful to estimate the confidence level of the estimated pose. Let $\mathbf{T}^{\text{gt}}$ be the ground-true pose, which is unknown, and $\delta \mathbf{T}^*$ be the difference between the optimal estimate $\mathbf{T}^*$ and the ground-true $\mathbf{T}^{\text{gt}}$, where $\mathbf{T}^{\text{gt}} = \mathbf{T}^* \boxplus \delta \mathbf{T}^*$. The aim is to estimate the covariance of the error $\delta \mathbf{T}^*$, denoted by $\Sigma_{\delta \mathbf{T}^*}$.

Ultimately, the estimation error $\delta \mathbf{T}^*$ is caused by the measurement noise in each raw point. Denote $\mathbf{p}_{f_{ijk}}^{\text{gt}}$ the ground-true location of the k -th point observed on the i -th feature at the j -th lidar pose, with measurement noise $\delta \mathbf{p}_{f_{ijk}} \in \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{p}_{f_{ijk}}})$, the measured point location, denoted by $\mathbf{p}_{f_{ijk}}$, is

$$\mathbf{p}_{f_{ijk}} = \mathbf{p}_{f_{ijk}}^{\text{gt}} + \delta \mathbf{p}_{f_{ijk}}. \quad (44)$$

Aggregating the ground-true points and the measured ones lead to the ground-true point cluster, denoted by $\mathbf{C}_{f_{ij}}^{\text{gt}}$, and the measured point cluster, denoted by $\mathbf{C}_{f_{ij}}$, respectively (see (23)):

$$\mathbf{C}_{f_{ij}}^{\text{gt}} = \begin{bmatrix} \sum_{k=1}^{N_{ij}} \mathbf{p}_{f_{ijk}}^{\text{gt}} (\mathbf{p}_{f_{ijk}}^{\text{gt}})^T & \sum_{k=1}^{N_{ij}} \mathbf{p}_{f_{ijk}}^{\text{gt}} \\ \left(\sum_{k=1}^{N_{ij}} \mathbf{p}_{f_{ijk}}^{\text{gt}} \right)^T & N_{ij} \end{bmatrix} \quad (45)$$

$$\approx \mathbf{C}_{f_{ij}} - \delta \mathbf{C}_{f_{ij}}, \quad (46)$$

where

$$\delta \mathbf{C}_{f_{ij}} = \sum_{k=1}^{N_{ij}} \mathbf{B}_{f_{ijk}} \delta \mathbf{p}_{f_{ijk}}, \quad \text{see Supplementary III-G, (47)}$$

which can be constructed in advance along with the point cluster $\mathbf{C}_{f_{ij}}$ during the feature associations stage.

In the following discussion, to simplify the notation, we denote $\mathbf{C}_f^{\text{gt}} = \{\mathbf{C}_{f_{ij}}^{\text{gt}}\}$, $\mathbf{C}_f = \{\mathbf{C}_{f_{ij}}\}$, $\delta \mathbf{C}_f = \{\delta \mathbf{C}_{f_{ij}}\}$ the ground-truth, measurements, and noises of all point clusters observed on any features at any lidar poses.

Although the ground-truth pose $\mathbf{T}_f^{\text{gt}}$ and point cluster $\mathbf{C}_f^{\text{gt}}$ are unknown, they are genuinely the optimal solution of (22) and hence the Jacobian evaluated there should be zero, i.e.,

$$\mathbf{J}^T(\mathbf{T}_f^{\text{gt}}, \mathbf{C}_f^{\text{gt}}) = \mathbf{0} \quad (48)$$

where we wrote the Jacobian as an explicit function of the pose and point clusters. Now, we approximate the left hand side of (48) by its first order approximation:

$$\begin{aligned} \mathbf{J}^T(\mathbf{T}_f^{\text{gt}}, \mathbf{C}_f^{\text{gt}}) &= \mathbf{J}^T(\mathbf{T}^* \boxplus \delta \mathbf{T}^*, \mathbf{C}_f - \delta \mathbf{C}_f) = \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f) \\ &+ \frac{\partial \mathbf{J}^T(\mathbf{T}^* \boxplus \delta \mathbf{T}, \mathbf{C}_f)}{\partial \delta \mathbf{T}} \delta \mathbf{T}^* - \frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f. \end{aligned} \quad (49)$$

Noticing that $\frac{\mathbf{J}^T(\mathbf{T}^* \boxplus \delta \mathbf{T}, \mathbf{C}_f)}{\partial \delta \mathbf{T}} = \mathbf{H}(\mathbf{T}^*, \mathbf{C}_f)$, the Hessian matrix of (22) evaluated at $\mathbf{T}^*$ (also see (42)), we have

$$\begin{aligned} \mathbf{0} &= \mathbf{J}^T(\mathbf{T}_f^{\text{gt}}, \mathbf{C}_f^{\text{gt}}) = \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f) \\ &+ \mathbf{H} \cdot \delta \mathbf{T}^* - \frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f, \end{aligned} \quad (50)$$

which implies

$$\delta \mathbf{T}^* = -\mathbf{H}^{-1} \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f) + \mathbf{H}^{-1} \frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f \quad (51)$$

Since $\mathbf{T}^*$ is the converged solution using the measured cluster $\mathbf{C}_f$, they should lead to zero update, i.e., $\mathbf{H}^{-1} \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f) = \mathbf{0}$ (see (43) with zero μ at convergence). Therefore,

$$\delta \mathbf{T}^* = \mathbf{H}^{-1} \frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f \sim \mathcal{N}(\mathbf{0}, \Sigma_{\delta \mathbf{T}^*}), \quad (52)$$

$$\Sigma_{\delta \mathbf{T}^*} = \mathbf{H}^{-1} \frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \Sigma_{\delta \mathbf{C}_f} \frac{\mathbf{J}(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \mathbf{H}^{-T}. \quad (53)$$

We defer the exact derivation and results of $\frac{\mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f$, $\Sigma_{\delta \mathbf{C}_f}$ and $\Sigma_{\delta \mathbf{T}^*}$ to Supplementary III-G [59]. Note that the evaluation of $\Sigma_{\delta \mathbf{T}^*}$ only requires the covariance $\delta \mathbf{C}_{f_{ij}}$, which has been constructed in advance according to (47), avoiding the enumeration of each raw point during the optimization.

IV. IMPLEMENTATIONS

We implemented our proposed method in C++ and tested it in Ubuntu 20.04 running on a desktop equipped with Intel i7-10750H CPU and 16Gb RAM. Since the reduced optimization problem (22) is not in a standard least square problem, which existing solvers (e.g., Google Ceres [60]) applies to, we implemented the optimization algorithm with steps and parameters described in Supplementary (Algorithm 1). When

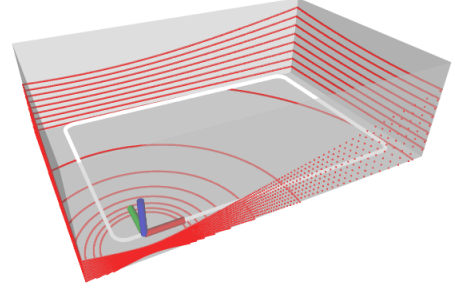


Fig. 5. Simulation setup: A 16-channel lidar moves along a rectangular trajectory in a cuboid semi-closed space. The white line is the trajectory and the red lines are the laser points.

solving the linear equation on Line 9 at each iteration, we use the LDLT Cholesky decomposition method implemented in Eigen library 3.3.7. The termination conditions on Line 19 are iteration number below 50 (i.e., $j_{\max} = 50$), rotation update below 10^{-6} rad, and translation update below 10^{-6} m.

V. CONSISTENCY EVALUATION

This study aims to verify the consistency of the proposed BA method. That is, whether the estimated covariance $\Sigma_{\delta \mathbf{T}^*}$ from (53) agrees with the ground-true covariance of the pose estimation error $\delta \mathbf{T}^*$. As the ground-true covariance is unknown, we refer to a standard measure of consistency, the normalized estimation error squared (NEES) [61, 62], which is defined below:

$$\eta = (\delta \mathbf{T}^*)^T \Sigma_{\delta \mathbf{T}^*}^{-1} \delta \mathbf{T}^*,$$

where $\delta \mathbf{T}_j^*$ is the estimation error of the pose defined according to (27):

$$\delta \mathbf{T}^* \triangleq (\dots, \delta \mathbf{T}_j^*, \dots) \in \mathbb{R}^{6M_p},$$

$$\delta \mathbf{T}_j^* = \left[\text{Log} \left(\mathbf{R}_j^{\text{gt}} (\mathbf{R}_j^*)^T \right), \mathbf{t}_j^{\text{gt}} - \mathbf{R}_j^{\text{gt}} (\mathbf{R}_j^*)^T \mathbf{t}_j^* \right]^T,$$

where the superscript “gt” denotes the ground-true poses and $(\mathbf{R}_j^*, \mathbf{t}_j^*)$ denotes the estimated pose for the j -th scan. Assume the pose estimate $(\mathbf{R}_j^*, \mathbf{t}_j^*)$ is unbiased (i.e., $E(\delta \mathbf{T}_j^*) = \mathbf{0}$), if the computed covariance $\Sigma_{\delta \mathbf{T}^*}$ is the ground-truth, we can obtain the expectation

$$\begin{aligned} E(\eta) &= E((\delta \mathbf{T}^*)^T \Sigma_{\delta \mathbf{T}^*}^{-1} \delta \mathbf{T}^*) = \text{trace}(E(\Sigma_{\delta \mathbf{T}^*}^{-1} \delta \mathbf{T}^* (\delta \mathbf{T}^*)^T)) \\ &= \text{trace}(\Sigma_{\delta \mathbf{T}^*}^{-1} E(\delta \mathbf{T}^* (\delta \mathbf{T}^*)^T)) = \text{trace}(\mathbf{I}) = \dim(\delta \mathbf{T}^*). \end{aligned} \quad (54)$$

That is, if the solver is consistent, the expectation of NEES should be equal to the dimension of the optimization variable. If the expectation of NEES is far higher than the dimension, the estimator is over-confident (i.e., the computed covariance is less than the ground-truth). Conversely, it is conservative.

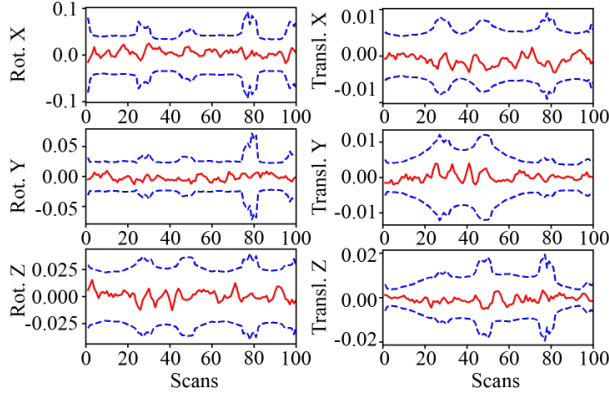


Fig. 6. The error (red) of rotation (deg) and position (m) with 3σ bounds (blue) for one simulation run.

In practice, the expectation of NEES is evaluated by Monte Carlo method, where the NEES is computed for many runs and then averaged to produce the empirical expectation.

$$\bar{\eta} = \frac{1}{N} \sum_{i=1}^N \eta^{(i)}$$

where $\eta^{(i)}$ is the NEES computed at the i -th Monte Carlo run.

To conduct the Monte Carlo evaluation, we simulate a 16-channel lidar along a rectangular trajectory in a cuboid semi-closed space shown in Fig. 5. The size of the space is 30 m $\times$ 20 m $\times$ 8 m and the length of trajectory is about 92 m. 100 scans are equally sampled on the trajectory and the number of points in each scan is 28,800. To simulate realistic measurements, each point is corrupted with an independent isotropic Gaussian noise with multiple standard deviations $\sigma_p \in \{0.05, 0.10, \dots, 1.00\}$ m and for each value of the standard deviation σ_p , we performed 100 Monte Carlo experiments, leading to a total number of 2000 experiments. In each run, we compute the optimal pose estimate from the Supplementary (Algorithm 1) with the same parameters specified in Section IV and the covariance matrix from (53). The initial trajectory required by the algorithm is obtained by perturbing the ground-true trajectory with a Gaussian noise with standard deviation $\delta\phi = 2$ deg and $\delta t = 0.1$ m on each pose. To avoid unnecessary errors, we use the ground-true plane association across different scans and ignore the in-frame motion distortion in the simulation.

Fig. 6 shows orientation and position errors with the corresponding 3σ bounds in one Monte Carlo experiment with $\sigma_p = 0.05$ m. As can be seen, the pose estimation errors are very small and they all remain within the 3σ bounds very well, which suggests that our new method is consistent.

Furthermore, we test the consistency of our BA method under different levels of point noise, where the standard deviation of a point noise ranges from $\sigma_p = 0.05$ m to 1 m. The results are shown in Fig. 7(a) for the NEES averaged over 100 runs for each noise level and in Fig. 7(b) for the average pose error. For better visualization, the average NEES is normalized by the pose dimension (i.e., 600 for 100 poses

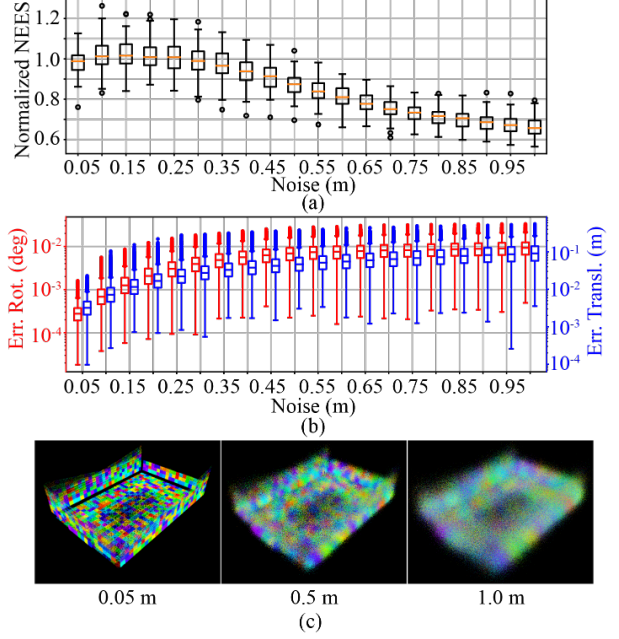


Fig. 7. (a) The normalized NEES averaged over 100 Monte Carlo runs at different point noise levels. The NEES is normalized by the pose dimension (i.e., 600) for better visualization. (b) The rotation (red) and translation (blue) errors at different point noise levels. (c) The point cloud map with ground-true poses at noise levels $\sigma_p = 0.05$ m, 0.5 m and 1 m, respectively.

on the trajectory) in Fig. 7(a). As can be seen, the normalized average NEES is very close to one, which suggests that our method is consistent, when the point noise is up to 0.3 m. Beyond this noise level, the first order approximation in (50) no longer holds, which undermines the accuracy of the computed covariance. We should note that this noise level rarely occurs in actual lidar sensors, which are well below 0.1 m. Moreover, from Fig. 7(b), we can see that our method produces accurate pose estimation even when the point noise are unrealistically large (up to 1 m, see Fig. 7(c) for the point cloud map at this point noise level).

VI. BENCHMARK EVALUATION

In this section, we compare our method with other multi-view registration methods for lidar point clouds. The experiment will be divided into two parts: Section VI-A evaluates all methods with known feature association on synthetic point clouds, and Section VI-B evaluates the overall BA pipeline including both optimization solver and feature association on various real-world open datasets.

To verify the effectiveness of our method, we compare it with four state-of-the-art methods that focus on the lidar bundle adjustment (or similar) problem: Eigen-Factor (EF) [33], BALM [32], Plane Adjustment (PA) [51], and BAREG [35]. Among them, EF⁴, BALM⁵, BAREG⁶ are open sourced, so we use the available implementation on Github. PA is

⁴<https://gitlab.com/gferrer/eigen-factors-iros2019>

⁵<https://github.com/hku-mars/BALM>

⁶<https://hyhuang1995.github.io/bareg/>

not available anywhere, so we re-implemented it in C++. To reduce the time cost of PA, we used the reduced Jacobian and residual technique in [34] (we derived it based on the cost function in Equation (10) of [51]), which avoids the enumeration of each individual point. The re-implemented PA is solved by the Ceres solver with “DENSE_SCHUR” [60], which leverages the Schur complement trick to reduce the linear equation dimension at each optimization iteration. To better exploit the separable structure reducing the solving time, we also compare with PA with inner iterations enabled in Ceres (denoted as “PA (inner)”).

For the solver parameters, our method and the re-implemented PA (and its variant PA (inner)) use the parameters specified in Section IV and [34], respectively, while EF, BALM and BAREG use their default parameters as available on their open source implementation. All methods use the same termination condition shown in Sec. IV (i.e. maximal iteration number below 200, rotation update below 10^{-6} rad, and translation update below 10^{-6} m), except for EF, which we found it converges too slowly and hence set the maximal iteration steps to 2000. In addition to the open source version of BALM (denoted by BALM), which samples only three points from each plane to lower the computation load, we also evaluated another vision (denoted by BALM (full)) which keeps all the points on a plane and use the same default parameters as its open sourced version. All solvers use the same initial pose trajectories detailed later.

A. Synthetic point cloud

To verify the effectiveness of the optimization solvers and their scalability to the number of pose M_p , number of feature M_f , and number of points N per feature, we design a point-cloud generator which generates M_f random planes and M_p lidar scans at random poses. Each pose corresponds to one group of point-cloud whose number of points on each plane is N . Hence, there are totally NM_f points at each scan. We use the ground-true plane association provided by the simulator. To mimic the real lidar point noises, we also corrupt the points sampled on each plane by an isotropic Gaussian noise with standard deviation $\sigma_p = 0.05\text{m}$, the typical noise level for existing lidar sensors. The initial poses are perturbed from the ground-true poses with errors randomly sampled from a Gaussian distribution. The base standard deviation of the Gaussian distribution is $\|\delta\phi\| = 0.1$ deg for rotation and $\|\delta\mathbf{t}\| = 1$ cm for translation. In the nominal settings, $M_f = M_p = N = 100$ and the initial pose error standard deviation is $10\times$ the base value. From the nominal settings, we enumerate each of the M_f, M_p and N at values $\{10, 30, 100, 300, 1000, 3000\}$ and the initial pose error standard deviation at values $1\times, 5\times, 10\times, 15\times, 20\times, 25\times$ of the base value to investigate the performance of each solver at different scales. This makes a total number of 21 scenes. In each scene, the experiment is repeated for 10 times with separately sampled poses, planes, and point noises, leading to a total 210 experiments.

1) *Convergence*: First we investigate the convergence performance of all methods. Fig. 8(a) and (b) respectively shows the convergence of cost and point-to-plane distance in one

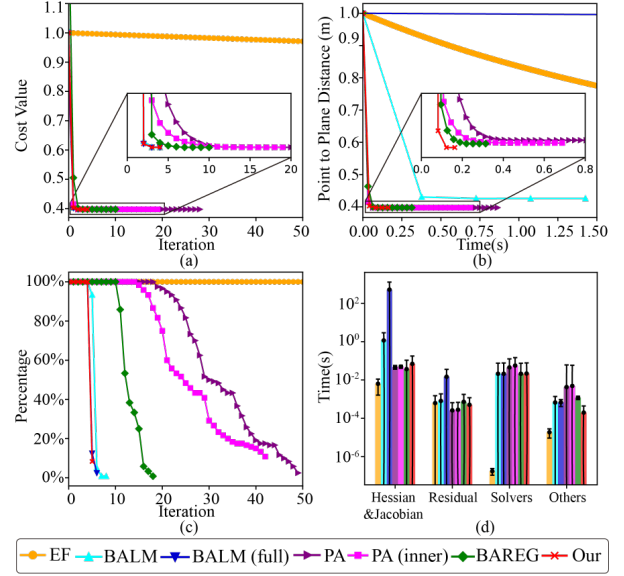


Fig. 8. Convergence of different methods for BA optimization. (a) Cost value versus iterations in one repeat experiment with the nominal settings $M_f = 100$, $M_p = 100$, $N = 100$ and initial pose error $10\times$. (b) Point-to-plane distance versus optimization time in one repeat experiment. (c) Iteration steps experienced by each method in all repeat experiments (i.e., 10) of all scenes (i.e., 21). The y-axis value represents how many experiments out of the 210 total experiments has experienced the iteration number indicated by the x-axis. (d) Breakdown of time spent on each iteration of all BA methods. The time is averaged among all experiments that all methods have participated.

repeat experiment with the nominal settings (i.e., $M_f = M_p = N = 100$ and initial pose error $10\times$). Since different method uses different cost function, to compare them in one figure, the cost value of each method is normalized by its initial cost and then it is re-based such that the converged cost value of all methods are aligned at the same value. Similarly, we normalize the point-to-plane distance by its initial value as well, which is valid to do because all methods have the same initial pose leading to the same initial point-to-plane distance. As can be seen, EF converges rather slowly and requires the most number of iterations. This is because EF optimizes a cost function similar to ours in (9), which is essentially a quadratic function, but uses only the gradient information for optimization. Indeed, slow convergence of the gradient descent method on a quadratic cost function is a very typical phenomenon [63]. PA, PA (inner), and BAREG converge fast at the beginning but slowly when approaching the final convergence value. This is because PA optimizes both the plane parameters and scan poses, leading to a very large number of optimization variables that significantly slow down the speed at convergence. PA (inner) converges faster than the original PA due to the inner iteration, but still slower than our method. For BAREG, the empirical fixation of plane parameters also causes the optimization to slow down. In contrast, BALM, BALM (full) and our method eliminates the plane parameters exactly and the resultant optimization problem is only in dimension of the pose number. Further leveraging the exact Hessian information in their optimization

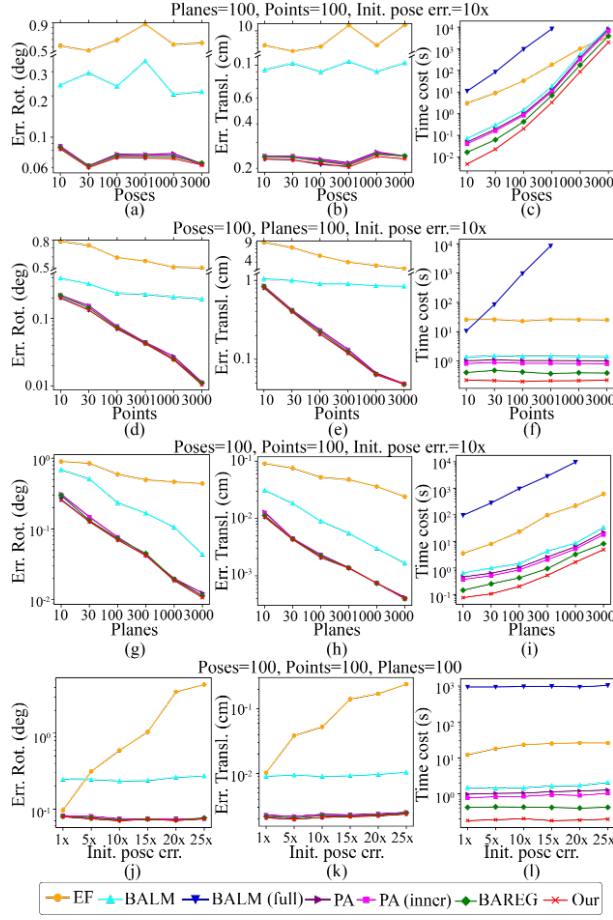


Fig. 9. Benchmark results on synthetic point cloud.

update, BALM, BALM (full) and our method converge in a few iterations, which often represent the fastest convergence.

Fig. 8(c) shows the iteration experienced by each BA optimization method, where for each data point, the y-axis value represents how many out of the total experiments experienced the iteration number indicated by the x-axis. As can be seen, the overall trend agrees with the results in Fig. 8 (a) very well: our proposed method and BALM (full) require only four or five iterations, while BAREG, PA, and PA (inner) require up to 20, 40, and 50 iterations, respectively. EF requires even more iterations beyond 100.

Fig. 8(d) shows the computation time in each iteration. As can be seen, EF consumes the least time for each iteration due to the lack of Hessian computation and linear equation solving. BALM (full) consumes the most time since the computation of Jacobian, Hessian and residuals require to enumerate each individual point, leading to a complexity of $O(N^2)$. The other methods, PA, PA (inner), BAREG, and ours, consume similar time for each iteration.

2) *Accuracy*: Fig. 9(a,b,d,e,g,h,j,k) shows the statistic values of the pose estimation accuracy in terms of RMSE. In each subplot, we fix three parameters of M_f , M_p , N and initial pose

error at the nominal values and change the fourth parameter to investigate its effect on the pose accuracy. Since the error of EF is much larger than the others, we used a broken y-axis to better display all the RMSE. As can be seen, overall the accuracy increases with the points per plane N (in (d) and (e)) or number of plane features M_f (in (g) and (h)) since both increases the number of pose constraints. In contrast, no such monotonic accuracy improvement is found for the number of poses (in (a) and (b)) as the pose number increases because increasing the pose number itself does not gives more pose constraints. Likewise, the accuracy also remains similar for different initial poses error for all methods except EF, which did not converge at the maximum iteration number. Relatively speaking, our proposed method and BALM (full) achieves the same highest accuracy, since they essentially optimizes the same cost using the same exact Hessian information. The next best methods are BAREG, PA, and PA (inner). While optimizing the same point to plane distance with our method (and BALM (full)), PA has significantly more optimization variables, which cause a much slower convergence where the solution is still slightly premature at the preset iteration number. Although PA (inner) has used inner iterations to alleviate this problem, its iterations are still larger than our methods and BAREG. The next accurate method is BALM, which samples only three instead of all points (as in our method, BALM (full), BAREG, PA, and PA (inner)) and hence has higher RMSE. Finally, EF has the highest RMSE due to the very slow convergence, the solution is much premature even at the preset iteration number.

3) *Computation time*: Finally, we show the total computation time of different solvers at different feature number M_f , pose number M_p , point number N and initial pose error. The results are shown in Fig. 9(c,f,i,l). As can be seen, the time consumption of all methods increases with the number of poses M_p (see (c)) and plane features M_f (see (i)), which is reasonable since more poses or planes lead to a higher optimization dimension or more number of cost items, respectively. On the other hand, as the point number N increases (see (f)), the method BALM (full) increases rapidly since its time complexity involves $O(N^2)$ while the rest methods (including ours) do not increase notably since they do not need to evaluate every raw point. For the effect of initial pose errors in (see (l)), they do not affect the solving time significantly.

Relatively speaking, our method achieves the lowest total computation time in all cases due to the small number of iteration numbers (Fig. 8(c)) and low time-complexity per iteration (Fig. 8(d)). The next efficient method is BAREG, which has very low time-complexity per iteration due to the empirical feature parameter fixation but significantly more iteration numbers due to the same reason. Compared with our method, PA (and PA (inner)) has similar time complexity per iteration as discussed in Supplementary (Section IV), but requires more iterations to converge. Hence their time costs are a little higher than ours and BAREG. BALM requires more iterations than our method and more time in each iteration due to the enumeration of the sampled points. Collectively, it leads to a computation time higher than our method and

also BAREG and PA (and PA (inner)). The slow convergence problem is more severe in EF, leading to an even higher computation time. Finally, the most time-consuming method is BALM (full), which, although has very small iteration numbers, consumes large time in each iteration.

B. Real-world datasets

In this experiment, we conduct benchmark comparison on three real-world datasets. The first dataset is “*Hilti*” [64] which is a handheld SLAM dataset including indoor and outdoor environments. We use the lidar data collected by Ouster OS0-64 in the dataset. The ground-true lidar pose trajectory is captured by a total station or motion capture system. The second dataset “*VIRAL*” [65] is collected on an unmanned aerial vehicle (UAV) equipped with two 16-channel OS1 lidars. One lidar is horizontal and the other is vertical. We will use the horizontal one in this experiment. The ground-true positions are provided by a Leica Nova MS60 MultiStation tracking a crystal prism on the UAV. The last dataset “*UrbanLoco*” [66] is collected by a car driving on urban streets. The lidar is a Velodyne HDL 32E and the ground truth is given by the Novatel SPAN-CPT, a navigation system incorporating Real Time Kinematic (RTK) and precisional IMU measurements.

Two preprocessing are performed for all sequences: motion compensation and scan downsample. To compensate the points distortion caused by continuous lidar movements within a scan, we run a tightly-coupled lidar-inertial odometry, FAST-LIO2 [19], which estimates the IMU bias (and other state variables) and compensates the point motion distortion in real-time. We kept all points in a scan whose distortion has been compensated by FAST-LIO2 and discard the odometry output. The processed data are then downsampled from the original 10 Hz to 2 Hz for all sequences. This is because the BA methods need to process all scans at once, a 10Hz scan rate causes prohibitively high computation load for all BA methods. The downsampling is also similar to the keyframe selection in common SLAM frameworks.

We compare our method with EF, BALM, PA, PA (inner) and BAREG. Noticing that the computation time of BALM (full) is prohibitively high due to the extremely large number of lidar points, we hence remove it from the benchmark comparison. For the rest methods, their solver parameters are kept the same for all sequences with values detailed in previous sections.

For feature association, we use the adaptive voxelization proposed in BALM [32], which registers all points in the world frame (using an initial trajectory) and recursively cuts the space into smaller sub-voxels until the sub-voxel contains only one feature (either plane or edge) that associates points from different scans. EF did not address the feature association problem and PA did not open relevant codes, so we use this method for them too. BAREG used a similar adaptive voxelization method but has its own implementation, so we retain its own implementation. All feature associations have the same set of parameters: the root voxel size $L = 1$ m for “*Hilti*” and $L = 2$ m for “*VIRAL*” and “*UrbanLoco*”, the maximum voxelization layer $l_{\max} = 3$, the minimum number

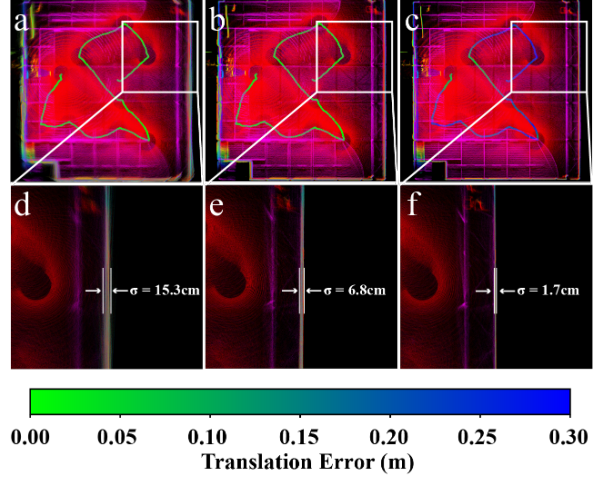


Fig. 10. Point cloud map of the UzhArea2 sequence in “*Hilti*”. (a) registered by ground-true pose trajectory. (b) registered by ground-true position with rotation optimized by our BA method. (c) registered by poses fully optimized by our BA method. (d), (e) and (f) points on one side wall in (a), (b) and (c), respectively.

of points $n_{\min} = 20$ for a feature test, and the feature test thresholds $\gamma = \frac{1}{25}$.

The above feature association method is able to extract and associate both plane and edge features. Since the other BA methods, including EF, PA (and PA (inner)), and BAREG, are only designed for plane features, we use only plane features for them. For our method, it is applicable to both plane and edge features, so we test two variants: the one with only plane features, denoted as “Ours”, for comparison with other BA methods, and the one with both plane and edge features, denoted as “Ours (edge)”. Moreover, besides the default implementation of our method with double-precision numbers, we test the stability of our method with single-precision floating number implementation, denoted as “Ours(float)”. Note that all other BA methods were implemented with double-precision.

In addition to the multi-view registration methods, we also compare with classic pairwise registration methods, including ICP, GICP, and NDT offered in PCL library. We run the pairwise registration methods in an incremental manner, where each new scan is registered and merged to previous scans incrementally. To constrain the computation time, in each new scan registration, only the last 20 scans are used. The pose estimation from the ICP is then used as the initial trajectory for feature association and optimization of the BA methods, including EF, BALM, PA, PA (inner), BAREG, and ours.

1) *Accuracy*: Table II shows the ATE results. As can be seen, our method consistently achieves the best results in all 19 sequences even with single-precision. The next accurate method is PA (inner), PA and BAREG, followed by BALM and EF. This trend is in great agreement with the results on synthetic point cloud shown in Section VI-A-2. In particular, our method achieves an accuracy within a few centimeters in all sequences of “*Hilti*” and “*VIRAL*”, with only one exception (i.e., UzhArea2), which will be analyzed later. The centimeter level accuracy achieved by our method is at the

TABLE II
ABSOLUTE TRAJECTORY ERROR (RMSE,METERS) FOR DIFFERENT METHODS.

Datasets	Sequence	ICP	GICP	NDT	EF	BALM	PA	PA (inner)	BAREG	Ours (float)	Ours (edge)	Ours
Hilti	Basement1	0.058	0.063	0.076	0.047	0.042	0.038	0.036	0.040	0.0359	0.0361	0.0353
	Basement4	0.084	0.089	0.098	0.071	0.058	0.048	0.045	0.054	0.0444	0.0448	0.0443
	Campus2	0.105	0.109	0.124	0.080	0.066	0.058	0.054	0.063	0.0535	0.0530	0.0531
	Construction2	0.108	0.104	0.113	0.086	0.068	0.060	0.059	0.063	0.0563	0.0577	0.0553
	LabSurvey2	0.066	0.069	0.072	0.046	0.025	0.019	0.019	0.023	0.0185	0.0189	0.0181
	UzhArea2	0.182	0.191	0.211	0.161	0.141	0.122	0.121	0.127	0.1205	0.1102	0.1171
VIRAL	eee01	0.159	0.163	0.172	0.102	0.073	0.052	0.040	0.061	0.0390	0.0401	0.0382
	eee02	0.153	0.154	0.163	0.092	0.062	0.043	0.037	0.057	0.0362	0.0378	0.0356
	eee03	0.171	0.175	0.180	0.113	0.081	0.056	0.053	0.068	0.0522	0.0548	0.0517
	nya01	0.139	0.136	0.163	0.107	0.082	0.042	0.038	0.054	0.0368	0.0372	0.0362
	nya02	0.160	0.159	0.124	0.097	0.067	0.050	0.048	0.061	0.0474	0.0472	0.0468
	nya03	0.142	0.143	0.146	0.085	0.074	0.044	0.042	0.067	0.0418	0.0425	0.0413
	sbs01	0.133	0.142	0.147	0.083	0.077	0.052	0.043	0.068	0.0397	0.0404	0.0385
	sbs02	0.127	0.127	0.121	0.094	0.062	0.040	0.039	0.059	0.0378	0.0393	0.0377
	sbs03	0.146	0.149	0.150	0.108	0.072	0.051	0.046	0.068	0.0440	0.0432	0.0427
UrbanLoco	0117	1.382	1.364	1.372	0.728	0.625	0.525	0.506	0.594	0.4964	0.5324	0.4956
	0317	1.384	1.299	1.289	0.878	0.732	0.661	0.657	0.682	0.6491	0.6449	0.6488
	0426-1	1.436	1.457	1.566	1.014	0.875	0.708	0.689	0.733	0.6891	0.7135	0.6886
	0426-2	1.676	1.693	1.543	1.113	0.924	0.864	0.837	0.905	0.8322	0.8536	0.8223
Average		0.411	0.410	0.412	0.268	0.221	0.186	0.179	0.203	0.1775	0.1826	0.1763

TABLE III
OCCUPIED CELLS OF POINT-CLOUD MAP FOR DIFFERENT METHODS.

Datasets	Sequence	ICP	GICP	NDT	EF	BALM	PA	PA (inner)	BAREG	Ours (float)	Ours (edge)	Ours
		(inc.)	(inc.)	(inc.)	(inc.)	(inc.)	(inc.)	(inc.)	(inc.)	(inc.)	(inc.)	(base)
Hilti	Basement1	+20300	+20954	+21354	+16692	+6285	+963	+332	+5864	+132	+257	391962
	Basement4	+7826	+7283	+8178	+6683	+4762	+1028	+223	+3752	+112	+197	558823
	Campus2	+14459	+15511	+21146	+8028	+2863	+977	+248	+2862	+68	-97	1319482
	Construction2	+6235	+9371	+10032	+6397	+1789	+1047	+394	+986	+95	+181	979614
	LabSurvey2	+1680	+3141	+6331	+5043	+1375	+410	+210	+1228	+83	+204	139682
	UzhArea2	+9490	+9623	+10832	+6371	+2688	+734	+484	+2785	+102	+344	628951
VIRAL	eee01	+43185	+43439	+44578	+22731	+2564	+996	+392	+1321	+85	+289	1166482
	eee02	+10339	+14573	+15848	+6985	+5938	+1538	+177	+5635	+91	+181	892168
	eee03	+8584	+9419	+7418	+5720	+2016	+1823	+286	+1060	+193	+630	594921
	nya01	+53004	+56370	+48669	+26087	+7368	+1717	+457	+4246	+46	+585	571365
	nya02	+38056	+37718	+38435	+24752	+4710	+1980	+692	+3902	+238	+232	572960
	nya03	+14282	+13896	+16325	+10688	+5922	+2178	+308	+2614	+172	+446	562583
	sbs01	+10069	+12196	+16597	+9635	+4224	+2056	+1064	+3691	+319	+717	794228
	sbs02	+16573	+16446	+21046	+10577	+9278	+3451	+488	+5238	+95	+502	808235
	sbs03	+12257	+11154	+8974	+4682	+877	+1492	+687	+763	+481	+332	867174
UrbanLoco	0117	+46718	+47572	+50969	+16327	+7237	+3420	+1016	+5412	+98	+1987	1743775
	0317	+37635	+33676	+41367	+20072	+13102	+4783	+1453	+8521	+103	+1011	1709823
	0426-1	+9165	+10242	+13695	+9539	+2331	+1364	+525	+1026	+33	+1413	1632662
	0426-2	+31870	+30461	+29568	+13827	+3428	+2021	+799	+4451	+472	+1252	2176302
Average		+21617	+21002	+22703	+12146	+4671	+1788	+539	+3439	+159	+561	953215

same level of lidar point noises. Moreover, using only lidar measurements, our method achieved an average accuracy of 4.2 cm on all VIRAL dataset sequences, which outperforms the accuracy 4.7 cm reported in VIRAL-SLAM [65] that fuses all data from stereo camera, IMU, lidar, and UWB. The accuracy on “UrbanLoco” is lower (analyzed later) than other datasets, but still outperforms the other BA methods. Finally, we can notice that the BA methods (i.e., EF, BALM, PA, PA (inner), BAREG, and ours) generally outperforms the pairwise registration methods (i.e., ICP, GICP and NDT) due to the full consideration of multi-view constraints.

When comparing among different variants of our method,

the single-precision implementation has a lower accuracy than double-precision as expected, but it offers significant time savings as discussed later. The incorporation of edge features leads to no noticeable accuracy improvement. The accuracy difference with and without edge features are as small as 6 mm. This is because in real-world point clouds, edge features extracted based on local smoothness (e.g., [18]) are very noisy because the laser pulse emitted by lidars can barely hit an edge exactly due to the limited angular resolution. The situation is further exacerbated when the edge is located at far or when the lidar has increased laser beam divergence, which

creates many bleeding points behind an edge and degrades the edge points extraction more [67]. On the other hand, in real-world environments, edge features are often created by depth discontinuity at the edge of a foreground object, which meanwhile makes a good plane feature, so adding the edge feature does contribute many new effective constraints.

It is noted that BAREG has an accuracy obviously lower than other methods (e.g., PA, PA (inner) and our method), which disagrees with results obtained previously from the synthetic data. The reason is that BAREG first extracts eigenvectors $\mathbf{u}_1$ and $\mathbf{u}_2$ ($\lambda_1 > \lambda_2 > \lambda_3$) of points corresponding to a plane feature in each local lidar scan. The two eigenvectors were assumed to be normal to the true plane normal and hence used to construct a cost item $\lambda_1 \|\mathbf{R}\mathbf{u}_1 \cdot \mathbf{n}\|^2 + \lambda_2 \|\mathbf{R}\mathbf{u}_2 \cdot \mathbf{n}\|^2$ in addition to the point to plane residual. The additional cost item could bias the optimization results if the extracted eigenvectors $\mathbf{u}_1$ and $\mathbf{u}_2$ are not accurate (i.e., they are not really perpendicular to the true plane normal), a presumption for the optimality of BAREG. Unfortunately, such optimality presumption did not hold well in real-world datasets, where the points density varies considerably: points on planes further from the sensor exhibit sparser distributions compared to those closer. This sparsity in distant planes leads to significant errors in the calculation of $\mathbf{u}_1$ and $\mathbf{u}_2$. Moreover, in real-world datasets, due to the imperfections of plane extraction, the extracted planes utilized for BA optimization may not be perfect planes (e.g., slightly curved walls or ground), and the point noise cannot be guaranteed isotropic Gaussian noise. All these factors contribute to errors in the extracted $\mathbf{u}_1$ and $\mathbf{u}_2$ and bias the optimization results.

Now we investigate the performance degradation on “UrbanLoco” and the sequence UzhArea2 in “Hilti” more closely. For the “UrbanLoco” dataset, we found that the RTK ground-truth had some false sudden jumps, which contributes the large ATEs. This sudden jump may be caused by tall buildings in the crowded urban area which lowers the quality of the ground-truth. For the sequence UzhArea2, we register the point cloud with the ground-true pose trajectory and compare it with the point cloud registered with our BA method in Fig. 10. As can be seen, with the ground-true pose, points on the side wall are very blurry and points on the wall form a plane with standard deviation up to 15.3 cm (Fig. 10(d)); with the ground-true translation but with rotations optimized by our BA method, the points on the side wall are much thinner and form an apparent plane of standard deviation 6.8 cm (Fig. 10(e)); with poses fully optimized by our method, the points are even more consistent and the standard deviation is 1.7 cm (Fig. 10(f)). From these results, we suspect that the ground-truth may be affected by some unknown errors (e.g., marker position change during the data collection). Indeed, we found similar problem on this sequence also occurred in other works [68]. Moreover, the standard deviation of 1.7cm achieved by our method is exactly the ranging accuracy of the lidar sensor, which confirms that our method achieves a mapping accuracy at the lidar noise level as if the sensor had no motion.

2) *Mapping quality*: A significant advantage of the BA method is the direct optimization of the map consistency (i.e., point-to-plane residuals). To evaluate the map quality

without a ground-true map, we adopt a method proposed by Anton *et al.* [69]. The method cuts the space into small cells and then counts the number of cells that lidar points occupy. The less the occupied cells, the higher the map quality. This indicator is intuitive: if points from different scans are registered accurately, they should agree with each other to the best extent, hence occupying the minimum possible number of cells. Based on this method, Table III presents the number of occupied cell with size 0.1 m. To better show the difference among different methods, the number of occupied cells are subtracted by our method for each sequence. We show the number of occupied cells by our method and the difference value of other methods. As can be seen, our methods consistently achieved the best performance in all sequences and the next best is PA (inner), PA, and BAREG. This trend also agrees with the ATE results very well.

3) *Computation time*: Finally, we compare the computation time. Since the pairwise registration methods, including ICP, GICP, and NDT, perform repetitive incremental registration at each scan reception, its computation time is very different from the BA methods that perform batch optimization on all scans at once. Therefore, we only compare the computation time of BA methods. Fig. 11 shows the convergence of all methods and Table IV shows the total optimization time. As can be seen, when all using double-precision, our method consumes the least computation time, about one fourth of the BAREG, one sixth of PA and PA (inner), one eighth of BALM, and one twentieth of EF. The overall trend agrees well with the results on synthetic point cloud in Section VI-A-3 with explanations detailed therein. Besides, our single-precision implementation reduces 40% further optimization time while still outperforming the other BA methods as detailed in previous section. Finally, the inclusion of extra edge features increases the number of cost items, resulting in an increased optimization time.

4) *Plane merging*: We further evaluate the performance of all BA methods at different number of plane features. To change the number of planes in real-world datasets, we develop a merging procedure in addition to the adaptive voxelization introduced in the experiment setup above. Starting from the root voxels, the adaptive voxelization recursively cuts the space into smaller sub-voxels until the sub-voxel contains only one plane feature. Then the merging process merges planes in small sub-voxels into larger planes. The merging proceeds at different degrees denoted by i (see Fig. 12), where a plane is merged with planes within up to $i - 1$ layers of neighboring root voxels. In the merging process, the two candidate planes $\mathcal{P}_i$ and $\mathcal{P}_j$ must satisfy

$$|\langle \mathbf{n}_i, \mathbf{n}_j \rangle| < \epsilon_1 \quad (55)$$

$$d \left| \langle \mathbf{c}_i - \mathbf{c}_j, \mathbf{n}_i \rangle - \frac{\pi}{2} \right| < \epsilon_2 \quad \left| \langle \mathbf{c}_i - \mathbf{c}_j, \mathbf{n}_j \rangle - \frac{\pi}{2} \right| < \epsilon_2 \quad (56)$$

where $\mathbf{n}$ and $\mathbf{c}$ are the normal vector and center of a plane respectively, symbol $\langle \cdot \rangle$ denotes the angle of two vectors, $\epsilon_1 = \epsilon_2 = 10^\circ$ are two constants. If the condition is not satisfied, the neighboring plane will not be merged.

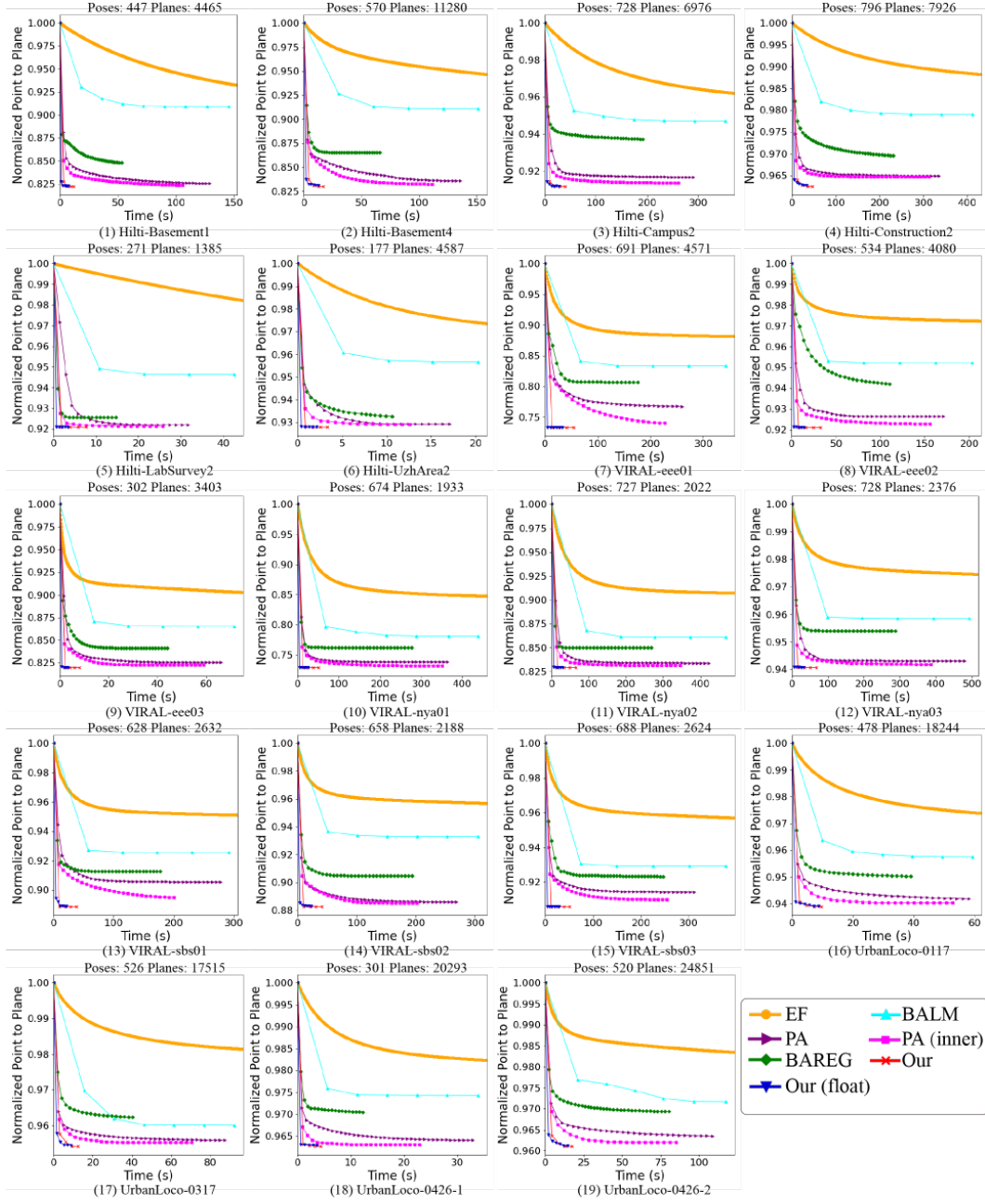


Fig. 11. Point-to-plane distance versus optimization time in real-world datasets including Hilti, VIRAL, and UrbanLoco. All methods have the same initial pose (hence the same initial point-to-plane distance) and have their point-to-plane distance all normalized by the initial values.

Given a merging degree i , we repeatedly merge planes starting from a seed plane randomly selected from the plane list. A merged plane will be removed from the list to avoid duplicate merging. Such procedure produces a new list of planes whose size are at most $i \cdot L$ with $L = 1$ or 2 m being the root voxel size. Larger merging degree i will lead to fewer number of planes but each with larger sizes.

Since the experimental results in the Sec. VI-A and VI-B have proved the PA with inner iteration outperforms the origi-

nal PA, we use the PA with inner iteration by default. The accuracy and computation time of all BA methods (including EF, BLAM, PA (inner), BAREG, and ours) at different merging degree i are shown in Fig. 13. The plane merging at different degrees leads to different computation time, so the time cost in the plot is the total time including adaptive voxelization, plane merging (if applicable), and BA optimization. As can be seen, our method consistently exhibits the highest accuracy and lowest time cost for all numbers of planes. Moreover,

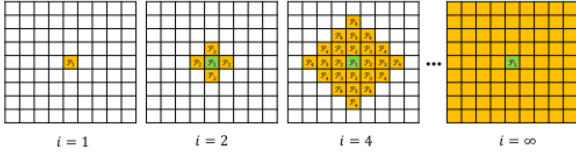


Fig. 12. Plane merging at degree i : “ $i = 1$ ” indicates only merging the planes in the same root voxel. “ $i = 2$ ” means merging maximum layers to $\mathcal{P}_2$ and “ $i = 4$ ” means merging maximum layers to $\mathcal{P}_4$. “ $i = \infty$ ” means no boundary layer is specified, the merging can go as far as possible.

as the merging degree i increases, the number of planes is decreased accordingly, leading to fewer planes that also reduce the optimization time of all BA methods. The reduction in optimization time is often larger than the time increment for merging, hence the total computation time still decreases with the merging degree. On the other hand, the pose RMSE of all methods all increase with the merging degree. This is because a larger merging degree introduces more bias to the optimization by merging planes not exactly on the same plane (e.g., slightly curved ground).

VII. APPLICATIONS

Bundle adjustment is the central technique of many lidar-based applications. In this section, we show how our bundle adjustment method can effectively improve the accuracy or computation efficiency of three vital applications: lidar-inertial odometry, multi-lidar calibration, and global mapping. Constrained by the page limit, details about the incorporation of bundle adjustment method in these applications and its effectiveness in real-world experiments are presented in Section I of the supplementary materials [59].

VIII. DISCUSSION

Here we discuss the efficiency, accuracy, and extendability of the proposed bundle adjustment method.

A. Efficiency

Our method achieved lower computation time than other state-of-the-art counterparts. The efficiency of our method are attributed to three inter-related and rigorously-proved techniques that make fully use of the problem nature and lidar point cloud property. The first technique is the solving of feature parameters in a closed-form before the BA optimization. It allows the feature parameters to be removed from the optimization, which fundamentally reduces the optimization dimension to the dimension of the pose only, a phenomenon that did not exist before in visual bundle adjustment problem. The second technique is a second-order solver which fits the quadratic cost function naturally and leads to fast convergence in the iterative optimization. This is enabled by the analytical derivation of the closed-form Jacobian and Hessian matrices of the cost function. The third technique is the point cluster, which enables the aggregation of all raw points without enumerating each individual point in neither of the cost evaluation, derivatives evaluation, or uncertainty evaluation. Collectively, these three techniques lead to an BA optimization with much lower dimension and time complexity.

B. Accuracy

Benefiting from the point cluster technique, our proposed method is able to exploit the information of all raw point measurements, achieving high pose estimation accuracy (a few centimeters) at the level of lidar measurement noise. Optimization from the raw lidar points also enables the developed method to estimate the uncertainty level of the estimated pose, which may be useful when this information is further fused with measurements from other sensors (e.g., IMU sensors). Moreover, by minimizing the Euclidean distance from each raw point to the corresponding feature, our method can reinforce the map consistency in a more direct manner than conventional pose graph optimization. While at a higher computation cost (due to the more complete consideration of features co-visible in multiple scans), it considerably improves the mapping accuracy which is important for mapping applications. Due to this reason, our method is particularly useful for accuracy refinements from a baseline pose trajectory that can be obtained by an odometry or a pose graph optimization module. The second order optimization provides very fast convergence when the solution is near to the optimal value, preventing premature solutions.

C. Extendability

As a basic technique for multiple scan registration, our proposed method can be easily be integrated with other formality of data, such as images and IMU measurements, by incorporating visual bundle adjustment factors and IMU pre-integration factors [62] in the optimization. Moreover, besides the frame-based pose trajectory, which attaches each frame an independent pose to estimate, our method can also work with other forms of pose trajectories, such as continuous-time trajectories based on Splines [9, 70] or Gaussian Process models [71, 72], which have the capability to compensate the in-frame motion distortion. According to the chain rules, the derivatives of the BA cost with respect to the trajectory parameters will consist of two parts: the first is the derivative of the BA cost with respect to the pose of each point cluster as derived in this paper, and the second part is the derivatives of the pose with respect to the trajectory parameters, which depends on the specific trajectories being used.

IX. CONCLUSION

This paper proposed a novel bundle adjustment method for lidar point cloud. The central of the proposed method is a point cluster technique, which aggregates all raw points into a compact set of parameters without enumerating each individual point. The paper showed how the bundle adjustment problem can be represented by the point cluster and also derived the analytical form of the Jacobian and Hessian matrices based on the point cluster. Based on these derivations, the paper developed a second-order solver, which estimates both the pose and the pose uncertainty. The developed BA method is open sourced to benefit the community.

Besides the technical developments, this paper also made some theoretical contributions, including the formalization of the point cluster and its operations, revealing of the invariance property of the formulated BA optimization, the proof of

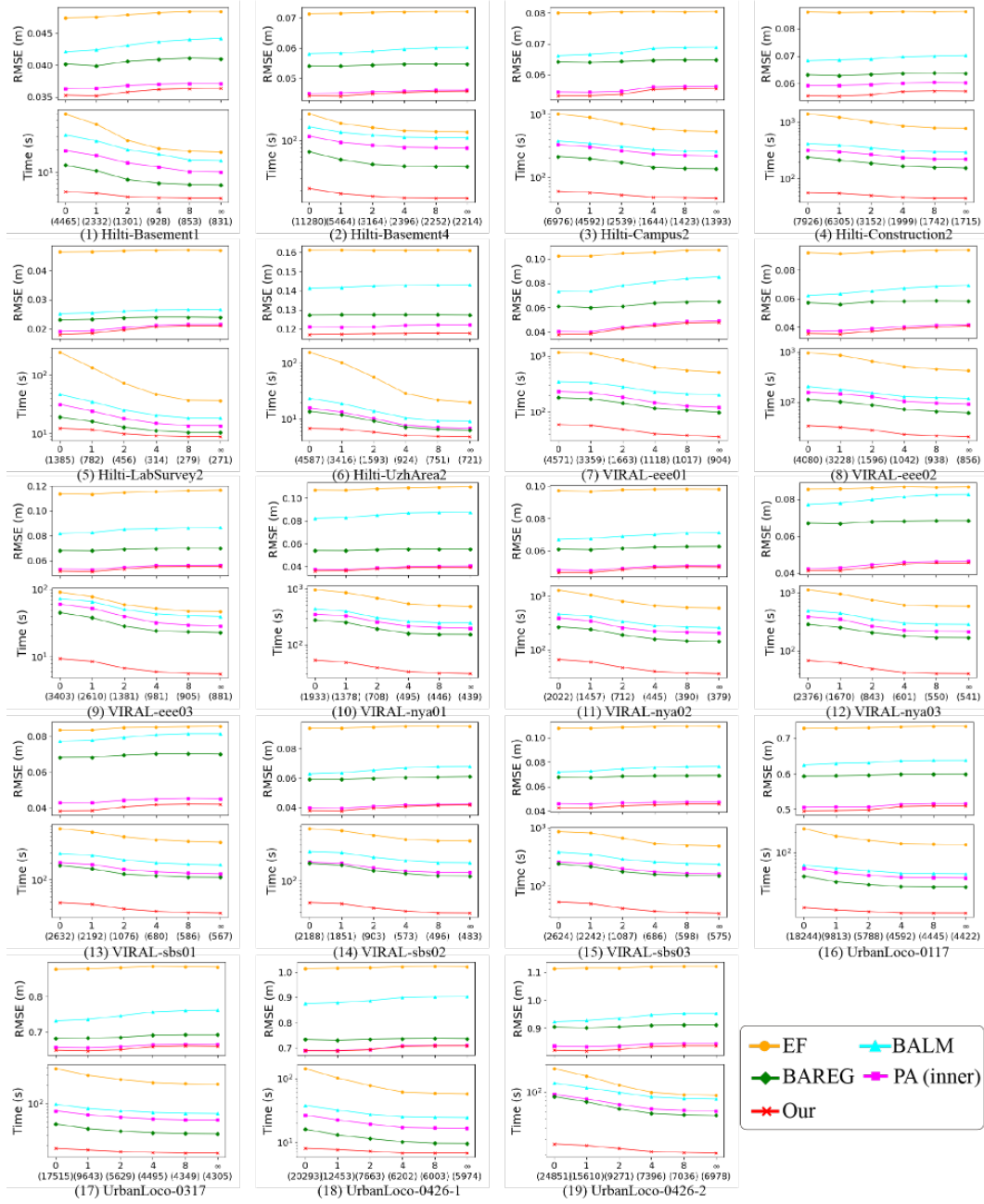


Fig. 13. The ATE and time cost of our method when merging planes at different degrees. The number “0”, “1”, “2”, “4”, “∞” on the X axis denotes the merging degree. The larger number in the parentheses below them are the number of planes corresponding to the merging degree.

null space and sparsity of the derived Jacobian and Hessian matrices, and the time complexity analysis of the proposed BA method and its comparison with others. These theoretical results serve the foundation of our developed BA techniques.

The proposed methods and implementations were extensively verified in both simulation and real-world experiments, in terms of consistency, efficiency, accuracy, and robustness. In all evaluations, the proposed method achieved consistently higher accuracy while consuming significantly lower computation time. This paper further demonstrated three applications

of the BA techniques, including lidar-inertial odometry, multi-lidar calibration, and high-accuracy mapping. In all applications, the adoption of BA method could effectively improve the accuracy or the efficiency.

In the future, we would like to incorporate the BA method more tightly to the above applications and beyond. This would require more thorough considerations of many practical issues, such as point cloud motion compensation, removal of dynamic objects, tightly-fusion with other formality of sensor data (e.g.,

TABLE IV
OPTIMIZATION TIME FOR DIFFERENT METHODS

Datasets	Sequence	EF	BALM	PA	PA (inner)	BAREG	Ours (float)	Ours (edge)	Ours
Hilti	Basement1	297.68	145.72	129.39	106.08	52.99	7.20	12.07	<i>11.94</i>
	Basement4	231.37	151.45	135.88	111.39	65.67	12.72	17.25	<i>17.01</i>
	Campus2	989.37	352.72	290.78	261.39	191.87	27.09	40.02	<i>39.95</i>
	Construction2	1415.18	412.00	335.70	313.23	231.48	33.04	47.34	<i>47.12</i>
	LabSurvey2	244.86	42.47	31.63	25.67	14.59	3.39	7.89	<i>7.64</i>
	UzhArea2	153.43	20.25	17.10	12.60	10.60	2.16	4.32	<i>4.08</i>
VIRAL	eee01	1162.25	342.60	259.95	227.86	175.83	33.01	56.21	<i>55.22</i>
	eee02	968.90	202.98	171.41	155.86	110.31	14.06	33.21	<i>32.34</i>
	eee03	89.45	71.56	66.11	59.02	44.10	3.27	8.17	<i>7.89</i>
	nya01	972.81	438.09	364.18	351.14	276.37	31.19	51.78	<i>51.01</i>
	nya02	1307.53	468.30	422.34	394.28	268.28	30.04	65.69	<i>65.19</i>
	nya03	1134.21	493.29	479.64	385.79	287.19	39.26	67.73	<i>67.26</i>
	sbs01	818.50	291.77	278.02	200.21	177.46	21.82	38.93	<i>37.80</i>
	sbs02	738.91	304.65	268.42	201.61	193.68	27.45	42.54	<i>41.35</i>
	sbs03	855.22	377.82	312.31	254.52	237.45	23.45	52.05	<i>51.55</i>
Urbanloco	0117	224.73	59.28	58.18	52.80	39.08	8.73	9.92	<i>9.60</i>
	0317	380.98	92.11	87.48	70.35	40.25	8.89	13.20	<i>12.38</i>
	0426-1	138.75	33.91	32.92	22.90	12.29	3.70	5.77	<i>4.31</i>
	0426-2	174.40	117.36	108.79	85.26	80.44	14.47	17.92	<i>17.34</i>
Average		647.29	232.54	202.64	171.11	132.10	18.15	31.16	<i>30.58</i>

IMU, camera) and module (e.g., loop closure).

REFERENCES

- [1] S. Thrun, M. Montemerlo, H. Dahlkamp, D. Stavens, A. Aron, J. Diebel, P. Fong, J. Gale, M. Halpenny, G. Hoffmann *et al.*, “Stanley: The robot that won the darpa grand challenge,” *Journal of field Robotics*, vol. 23, no. 9, pp. 661–692, 2006.
- [2] C. Urmson, J. Anhalt, D. Bagnell, C. Baker, R. Bittner, M. Clark, J. Dolan, D. Duggins, T. Galatali, C. Geyer *et al.*, “Autonomous driving in urban environments: Boss and the urban challenge,” *Journal of field Robotics*, vol. 25, no. 8, pp. 425–466, 2008.
- [3] J. Levinson, J. Askeland, J. Becker, J. Dolson, D. Held, S. Kammel, J. Z. Kolter, D. Langer, O. Pink, V. Pratt *et al.*, “Towards fully autonomous driving: Systems and algorithms,” in *2011 IEEE intelligent vehicles symposium (IV)*. IEEE, 2011, pp. 163–168.
- [4] Y. Li and J. Ibanez-Guzman, “Lidar for autonomous driving: The principles, challenges, and trends for automotive lidar and perception systems,” *IEEE Signal Processing Magazine*, vol. 37, no. 4, pp. 50–61, 2020.
- [5] F. Gao, W. Wu, W. Gao, and S. Shen, “Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments,” *Journal of Field Robotics*, vol. 36, no. 4, pp. 710–733, 2019.
- [6] F. Kong, W. Xu, Y. Cai, and F. Zhang, “Avoiding dynamic small obstacles with onboard sensing and computation on aerial robots,” *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7869–7876, 2021.
- [7] Y. Ren, F. Zhu, W. Liu, Z. Wang, Y. Lin, F. Gao, and F. Zhang, “Bubble planner: Planning high-speed smooth quadrotor trajectories using receding corridors,” *arXiv preprint arXiv:2202.12177*, 2022.
- [8] B. Schwarz, “Mapping the world in 3d,” *Nature Photonics*, vol. 4, no. 7, pp. 429–430, 2010.
- [9] M. Bosse, R. Zlot, and P. Fflick, “Zebedee: Design of a spring-mounted 3-d range sensor with application to mobile mapping,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1104–1119, 2012.
- [10] M. Helmberger, K. Morin, B. Berner, N. Kumar, G. Cioffi, and D. Scaramuzza, “The hilti slam challenge dataset,” *IEEE Robotics and Automation Letters*, 2022.
- [11] D. Wang, C. Watkins, and H. Xie, “Mems mirrors for lidar: a review,” *Micromachines*, vol. 11, no. 5, p. 456, 2020.
- [12] Z. Liu, F. Zhang, and X. Hong, “Low-cost retina-like robotic lidars based on incommensurable scanning,” *IEEE/ASME Transactions on Mechatronics*, vol. 27, no. 1, pp. 58–68, 2021.
- [13] P. J. Besl and N. D. McKay, “Method for registration of 3-d shapes,” in *Sensor fusion IV: control paradigms and data structures*, vol. 1611. Spie, 1992, pp. 586–606.
- [14] A. Segal, D. Haehnel, and S. Thrun, “Generalized-icp,” in *Robotics: science and systems*, vol. 2, no. 4. Seattle, WA, 2009, p. 435.
- [15] P. Biber and W. Straßer, “The normal distributions transform: A new approach to laser scan matching,” in *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)(Cat. No. 03CH37453)*, vol. 3. IEEE, 2003, pp. 2743–2748.
- [16] M. Magnusson, “The three-dimensional normal-distributions transform: an efficient representation for registration, surface analysis, and loop detection,” Ph.D. dissertation, Örebro universitet, 2009.
- [17] J. Behley and C. Stachniss, “Efficient surfel-based slam using 3d laser range data in urban environments,” in *Proc. of Robotics: Science and Systems (RSS)*, 2018.
- [18] J. Zhang and S. Singh, “Loam: Lidar odometry and mapping in real-time,” in *Robotics: Science and Systems*, vol. 2, no. 9. Berkeley, CA, 2014, pp. 1–9.
- [19] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “Fast-lio2: Fast direct lidar-inertial odometry,” *IEEE Transactions on Robotics*, pp. 1–21, 2022.
- [20] M. Yokozuka, K. Koide, S. Oishi, and A. Banno, “Litamin2: Ultra light lidar-based slam using geometric approximation applied with kl-divergence,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 11 619–11 625.
- [21] H. Surmann, A. Nüchter, and J. Hertzberg, “An autonomous mobile robot with a 3d laser range finder for 3d exploration and digitalization of indoor environments,” *Robotics and Autonomous Systems*, vol. 45, no. 3-4, pp. 181–198, 2003.
- [22] X. Liu and F. Zhang, “Extrinsic calibration of multiple lidars of small fov in targetless environments,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2036–2043, 2021.
- [23] G. Klein and D. Murray, “Parallel tracking and mapping for small ar workspaces,” in *2007 6th IEEE and ACM international symposium on mixed and augmented reality*. IEEE, 2007, pp. 225–234.
- [24] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, “Orb-slam: a versatile and accurate monocular slam system,” *IEEE transactions on robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [25] R. Mur-Artal and J. D. Tardós, “Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras,” *IEEE transactions on robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [26] T. Qin, P. Li, and S. Shen, “Vins-mono: A robust and versatile monocular visual-inertial state estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [27] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, “Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [28] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, “Pixel-wise view selection for unstructured multi-view stereo,” in *European Conference on Computer Vision (ECCV)*, 2016.

- [29] P. Moulon, P. Monasse, R. Perrot, and R. Marlet, "Openmvg: Open multiple view geometry," in *International Workshop on Reproducible Research in Pattern Recognition*. Springer, 2016, pp. 60–74.
- [30] B. Li, L. Heng, K. Koser, and M. Pollefeys, "A multiple-camera system calibration toolbox using a feature descriptor-based calibration pattern," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2013, pp. 1301–1307.
- [31] A. Zaharescu, R. Horaud, R. Ronfard, and L. Lefort, "Multiple camera calibration using robust perspective factorization," in *Third International Symposium on 3D Data Processing, Visualization, and Transmission (3DPVT'06)*. IEEE, 2006, pp. 504–511.
- [32] Z. Liu and F. Zhang, "Balm: Bundle adjustment for lidar mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3184–3191, 2021.
- [33] G. Ferrer, "Eigen-factors: Plane estimation for multi-frame and time-continuous point cloud alignment," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 1278–1284.
- [34] L. Zhou, D. Koppel, H. Ju, F. Steinbruecker, and M. Kaess, "An efficient planar bundle adjustment algorithm," in *2020 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 2020, pp. 136–145.
- [35] H. Huang, Y. Sun, J. Wu, J. Jiao, X. Hu, L. Zheng, L. Wang, and M. Liu, "On bundle adjustment for multiview point cloud registration," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 8269–8276, 2021.
- [36] R. Bergevin, M. Soucy, H. Gagnon, and D. Laurendeau, "Towards a general multi-view registration technique," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 18, no. 5, pp. 540–547, 1996.
- [37] F. Lu and E. Milios, "Globally consistent range scan alignment for environment mapping," *Autonomous robots*, vol. 4, no. 4, pp. 333–349, 1997.
- [38] K. Pulli, "Multiview registration for large data sets," in *Second international conference on 3-d digital imaging and modeling (cat. no. pr00062)*. IEEE, 1999, pp. 160–168.
- [39] D. F. Huber and M. Hebert, "Fully automatic registration of multiple 3d data sets," *Image and Vision Computing*, vol. 21, no. 7, pp. 637–650, 2003.
- [40] D. Borrmann, J. Elseberg, K. Lingemann, A. Nüchter, and J. Hertzberg, "Globally consistent 3d mapping with scan matching," *Robotics and Autonomous Systems*, vol. 56, no. 2, pp. 130–142, 2008.
- [41] V. M. Govindu and A. Pooja, "On averaging multiview relations for 3d scan registration," *IEEE Transactions on Image Processing*, vol. 23, no. 3, pp. 1289–1302, 2013.
- [42] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 5135–5142.
- [43] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Globally consistent 3d lidar mapping with gpu-accelerated gicp matching cost factors," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 8591–8598, 2021.
- [44] G. Blais and M. D. Levine, "Registering multiview range data to create 3d computer objects," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 17, no. 8, pp. 820–824, 1995.
- [45] R. Benjema and F. Schmitt, "A solution for the registration of multiple 3d point sets using unit quaternions," in *European conference on computer vision*. Springer, 1998, pp. 34–50.
- [46] P. J. Neugebauer, "Reconstruction of real-world objects via simultaneous registration and robust combination of multiple range images," *International journal of shape modeling*, vol. 3, no. 01n02, pp. 71–90, 1997.
- [47] J. Zhu, Z. Jiang, G. D. Evangelidis, C. Zhang, S. Pang, and Z. Li, "Efficient registration of multi-view point sets by k-means clustering," *Information Sciences*, vol. 488, pp. 205–218, 2019.
- [48] M. Kaess, "Simultaneous localization and mapping with infinite planes," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 4605–4611.
- [49] M. Hsiao, E. Westman, G. Zhang, and M. Kaess, "Keyframe-based dense planar slam," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. Ieee, 2017, pp. 5110–5117.
- [50] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment—a modern synthesis," in *International workshop on vision algorithms*. Springer, 1999, pp. 298–372.
- [51] L. Zhou, S. Wang, and M. Kaess, " π -lsam: Lidar smoothing and mapping with planes," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 5751–5757.
- [52] L. Zhou, D. Koppel, and M. Kaess, "Lidar slam with plane adjustment for indoor environment," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7073–7080, 2021.
- [53] P. Geneva, K. Eickenhoff, Y. Yang, and G. Huang, "Lips: Lidar-inertial 3d plane slam," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 123–130.
- [54] A. J. Trevor, J. G. Rogers, and H. I. Christensen, "Planar surface slam with 3d and 2d sensors," in *2012 IEEE International Conference on Robotics and Automation*. IEEE, 2012, pp. 3041–3048.
- [55] D. Strelow, "General and nested wiberg minimization: L 2 and maximum likelihood," in *Computer Vision—ECCV 2012: 12th European Conference on Computer Vision, Florence, Italy, October 7–13, 2012, Proceedings, Part VII 12*. Springer, 2012, pp. 195–207.
- [56] G. H. Golub and V. Pereyra, "The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate," *SIAM Journal on numerical analysis*, vol. 10, no. 2, pp. 413–432, 1973.
- [57] T. Wiberg, "Computation of principal components when data are missing," in *Proc. of Second Symp. Computational Statistics*, 1976, pp. 229–236.
- [58] A. Ruhe and P. Å. Wedin, "Algorithms for separable nonlinear least squares problems," *SIAM review*, vol. 22, no. 3, pp. 318–337, 1980.
- [59] Z. Liu, X. Liu, and F. Zhang, "Efficient and consistent bundle adjustment on lidar point clouds supplementary," 2022. [Online]. Available: <https://github.com/hku-mars/BALM/blob/master/Supplementary/Supplementary.pdf>
- [60] S. Agarwal, K. Mierle, and T. C. S. Team, "Ceres Solver," <https://github.com/ceres-solver/ceres-solver>, 3 2022.
- [61] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with applications to tracking and navigation: theory algorithms and software*. John Wiley & Sons, 2004.
- [62] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2016.
- [63] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.
- [64] M. Helmberger, K. Morin, N. Kumar, D. Wang, Y. Yue, G. Cioffi, and D. Scaramuzza, "The hilti slam challenge dataset," *arXiv preprint arXiv:2109.11316*, 2021.
- [65] T.-M. Nguyen, S. Yuan, M. Cao, Y. Lyu, T. H. Nguyen, and L. Xie, "Ntu viral: A visual-inertial-ranging-lidar dataset, from an aerial vehicle viewpoint," *The International Journal of Robotics Research*, p. 02783649211052312, 2021.
- [66] W. Wen, Y. Zhou, G. Zhang, S. Fahandezh-Saadi, X. Bai, W. Zhan, M. Tomizuka, and L.-T. Hsu, "Urbanloco: A full sensor suite dataset for mapping and localization in urban scenes," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 2310–2316.
- [67] C. Yuan, X. Liu, X. Hong, and F. Zhang, "Pixel-level extrinsic self calibration of high resolution lidar and camera in targetless environments," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7517–7524, 2021.
- [68] M. Camurri, L. Zhang, D. Wisth, and M. Fallon, "Hilti slam challenge submission: Vilens and slam," 2021. [Online]. Available: <https://hilti-challenge.com/submissions/VILENS%20and%20SLAM,%20Oxford%20Robotics%20Institute/report.pdf>
- [69] A. Filatov, A. Filatov, K. Krinkin, B. Chen, and D. Molodan, "2d slam quality evaluation methods," in *2017 21st Conference of Open Innovations Association (FRUCT)*. IEEE, 2017, pp. 120–126.
- [70] D. Droschel and S. Behnke, "Efficient continuous-time slam for 3d lidar-based online mapping," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 5000–5007.
- [71] C. H. Tong, P. Furgale, and T. D. Barfoot, "Gaussian process gauss-newton for non-parametric simultaneous localization and mapping," *The International Journal of Robotics Research*, vol. 32, no. 5, pp. 507–525, 2013.
- [72] C. Le Gentil, T. Vidal-Calleja, and S. Huang, "In2laama: Inertial lidar localization autocalibration and mapping," *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 275–290, 2020.

Efficient and Consistent Bundle Adjustment on Lidar Point Clouds (Supplementary)

Zheng Liu, Xiyuan Liu and Fu Zhang

Please note that equation numbers and section numbers from the main manuscript are labelled in this letter in **red**.

I. APPLICATIONS

A. Lidar-inertial odometry with sliding window optimization

A local bundle adjustment in a sliding window of keyframes has been widely used in visual odometry and proved to be very effective in lowering the odometry drift [1]–[4]. Similar idea could apply to the lidar-inertial odometry based on our BA method. To demonstrate the effectiveness, we design a lidar-inertial odometry system as shown in Fig. 12. The system is divided into two parts: the EKF-based front-end, which provides initial yet timely pose estimation (EKF odometry), and the BA-based back-end, which refines the pose estimation in a sliding window (Local mapping). The EKF design is similar to FAST-LIO2 [5] which compensates the motion distortion in the incoming lidar scans and performs EKF propagation and update on the state manifold. The sliding window optimization performs a local BA among the most recent 20 scans considering constraints of IMU preintegration [6] and constraints from plane features co-visible among all scans in the window (Section III). After the convergence of the local BA, points in the local window are built into a k -d tree to register the next incoming scan. Furthermore, the oldest scan is removed from window and its contained points are merged into the global point cloud map.

We compare our proposed system with one state-of-art lidar-inertial odometry, FAST-LIO2 [5], which performs incremental pairwise scan registration via GICP. The comparison is conducted on “*utbm*”, “*ucl*”, and “*ncl*” dataset that evaluated by FAST-LIO2, so the results of FAST-LIO2 are directly read from the original paper [7]. As can be seen in Table V, benefiting from the abundant multi-view constraints in the local sliding window, our system consistently outperforms FAST-LIO2 in terms of accuracy with considerable margins except the sequence “*nclt4*”. The improvements in odometry accuracy confirm the effectiveness of the local BA optimization. The improvement in accuracy comes with increased computation costs as shown in the last two columns of Table V, where for FAST-LIO2, we record the time of each scan-to-map registration and for our LIO system, the time consumption is divided into two parts: one is the front-end scan-to-local map registration and the back-end local BA optimization. As can be seen, our system has a little more time consumption in front-end than FAST-LIO2 because of the building of a k -d tree, which takes a constant time overhead, while FAST-LIO2 uses a more efficient incremental k -d tree structure. In addition, our

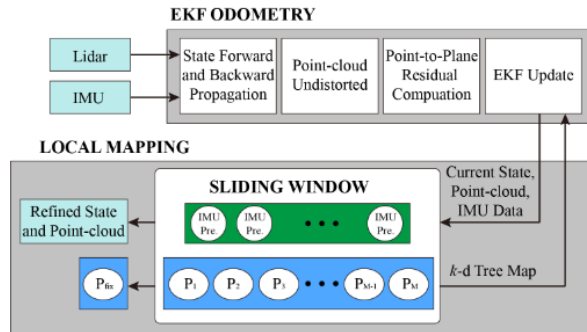


Fig. 12. Overview of the LIO system with BA and IMU preintegration

TABLE V
ATE AND TIME COST PER SCAN OF FAST-LIO2 AND LIO WITH BA

	ATE (m)		Time (ms)	
	FAST-LIO2	Local-BA	FAST-LIO2	Local-BA (Odom/BA Opt.)
utbm8	23.7	20.4	22.05	31.73/92.11
utbm9	45.9	39.1	25.44	33.42/95.43
utbm10	16.8	13.1	22.48	30.46/97.81
ucl4	1.31	1.15	20.14	19.53/69.89
nclt4	8.50	9.23	15.72	21.77/59.83
nclt5	6.65	6.25	16.60	26.13/61.23
nclt6	20.57	20.34	15.84	25.14/63.19
nclt7	6.58	5.69	16.87	23.18/62.53
nclt8	30.08	26.24	14.25	24.80/69.29
nclt9	5.56	5.07	13.65	22.98/67.36
nclt10	16.29	14.10	21.79	23.41/64.45
Average	16.54	14.61	18.62	25.68/73.01

system requires an average of 73 ms in the back-end local BA optimization, which ensures a 10 Hz running frequency for both the front-end and back-end. Since the back-end runs in parallel in a separate thread, the overall odometry latency of our system is only 7 ms more than that of FAST-LIO2.

B. Multiple-lidar Calibration

With the ability of concurrent optimization of multiple lidar poses, our BA method can be applied to multi-lidar extrinsic calibration. We consider the problem in [8], which aims to calibrate the extrinsic of multiple solid-state lidars shown in Fig. 13. Due to the very small FoV, these lidars have very small or even no FoV overlap. To create co-visible features, the vehicle is rotated for one cycle, during which a set of point cloud scans are collected by all lidars. Due to the rotation,

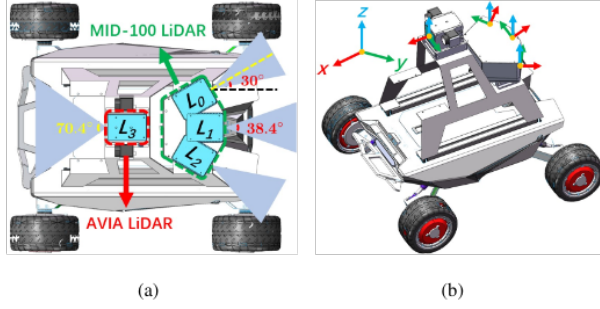


Fig. 13. The customized multi-sensor vehicle platform in [8]. The MID-100 lidar consists of three lidars: L_0 , L_1 and L_2 , with FoV overlap between adjacent lidars 8.4° . The AVIA lidar is denoted as L_3 . (a) The FoV coverage of each lidar sensor. (b) Each lidar sensor's orientation denoted in the right-handed coordinate system.

it further introduces unknown lidar poses, in addition to the extrinsic, to estimate.

To formulate the simultaneous localization and extrinsic calibration problem, we choose a reference lidar as the base and calibrate the extrinsic of the rest lidars relative to it. Denote ${}^G\mathbf{T}_j, j = 1, \dots, M_p$, the pose of the reference lidar at j -th scan in a rotation, ${}^R\mathbf{T}_k$ the extrinsic of the k -th lidar relative to the base lidar. Then, following (25), the cost item corresponding to the i -th plane feature is

$$c_i({}^G\mathbf{T}_j, {}^R\mathbf{T}_k) = \lambda_3 \left(\mathbf{A} \left(\sum_{j=1}^{M_p} {}^G\mathbf{T}_j \cdot \mathbf{C}_{f_{ij}}^r \cdot {}^G\mathbf{T}_j^T + \sum_{k \neq r} {}^G\mathbf{T}_j \cdot {}^R\mathbf{T}_k \cdot \mathbf{C}_{f_{ij}}^k \cdot ({}^G\mathbf{T}_j \cdot {}^R\mathbf{T}_k)^T \right) \right) \quad (57)$$

where $\mathbf{C}_{f_{ij}}^r, \mathbf{C}_{f_{ij}}^k$ are the point cluster of the reference lidar and the k -th lidar, respectively, observed on the i -th feature at the j -th scan. Following Theorem 4, the first and second order derivatives of (57) can be obtained by chain rules, whose details are omitted here due to limited space.

We demonstrate the advantage of the proposed BA approach in multi-lidar calibration with the latest state-of-the-art calibration methods based on ICP [9] and BALM [8]. The ICP-based method [9] optimizes the extrinsic parameter ${}^R\mathbf{T}_k$ and base lidar pose ${}^G\mathbf{T}_j$ by repeatedly registering the point cloud by each lidar at each scan to the rest lidar points until convergence, whereas in both BALM-based method [8] and our method, the extrinsic parameter ${}^R\mathbf{T}_k$ and base lidar pose ${}^G\mathbf{T}_j$ are optimized concurrently. In [9] the feature correspondence searching is conducted by a k -d tree data structure, while in [8] and our method, this is resolved by adaptive voxelization proposed in BALM [10]. To restrain the computation time, the point number in each voxel of [8] have been down-sampled to four, whereas in our method, all feature points are used.

We test these methods on two lidar setups in the system shown in Fig. 13: lidars with small field-of-view (FoV) overlap (MID-100 self calibration) and lidars without FoV overlap (AVIA and MID-100 calibration). Since the extrinsic between

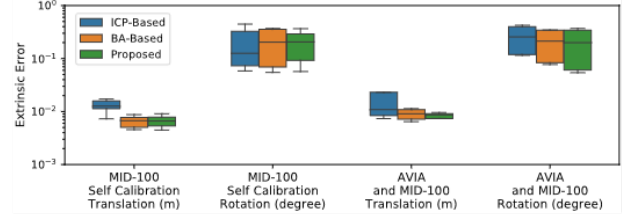


Fig. 14. Extrinsic calibration results of the ICP-based [9], BALM-based [8] and our proposed method in two experiment setups (with small or without FoV overlap).

the internal lidars of MID-100 (i.e., ${}^{L_1}\mathbf{T}, {}^{L_2}\mathbf{T}$) is known from the manufacturer's in-factory calibration, it is thus served as the ground truth in both setups. Moreover, in the MID-100 self calibration setup, the middle L_1 is chosen as the reference lidar to calibrate the extrinsic of adjacent lidars, i.e., ${}^{L_0}\mathbf{T}, {}^{L_2}\mathbf{T}$. This lidar setup is tested with data collected in both [9] and [8] under two scenes which contributes to eight calibration results. In the MID-100 and AVIA calibration setup, the L_3 is selected as the reference lidar to calibrate the extrinsic of each internal lidar of MID-100, i.e., ${}^{L_0}\mathbf{T}, {}^{L_1}\mathbf{T}$ and ${}^{L_2}\mathbf{T}$. We then calculate the relative pose ${}^{L_1}\mathbf{T}, {}^{L_2}\mathbf{T}$ from the calibrated results and compare them with the ground truth. This lidar setup is tested with data collected under two scenes in [8] since no AVIA lidar was used in [9], which contributes another four calibration results.

The total twelve independent calibration results are illustrated in Fig. 14 and Fig. 15. It is seen the our proposed method outperforms the ICP-based method [9] especially in translation. This is due to the concurrent optimization of all poses and extrinsic at the same time, which leads to full convergences within a few iterations. In contrast, the repetitive pairwise ICP registration leads to very slow convergence. The optimization did not fully converge after the maximum iteration number ($max_iter=40$), a phenomenon detailed in [8]. Furthermore, since all raw points on a feature are utilized in our method, the accuracy of our proposed work is also slightly improved when compared with the BALM-based method [8], which sample a few points on a feature to restrain the computation time. The averaged time consumption in each step of these methods have been summarized in Table VI. It is seen our proposed work and [8] has significantly shortened the calibration time in each step compared with the ICP-based method. This is due to the use of adaptive voxelization which saves a great amount of time in k -d tree build and nearest neighbor search used in [9]. Compared with [8], our proposed BA has further increased the computation speed due to the use of point cluster technique avoiding the enumerating of individual points in the BALM-based method [8].

C. Global BA on Large-Scale Dataset

In this section, we show that our proposed BA method could also be used to globally refine the quality of a large-scale lidar point cloud [11]. It is popular to use pose graph optimization (PGO) to increase the overall SLAM accuracy [12]. In PGO, the poses are optimized by minimizing the error between the

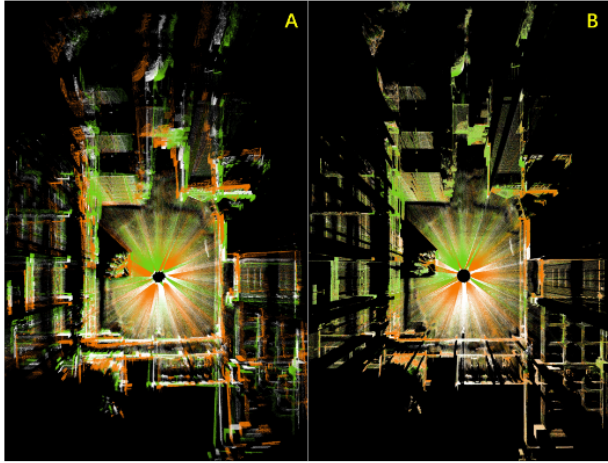


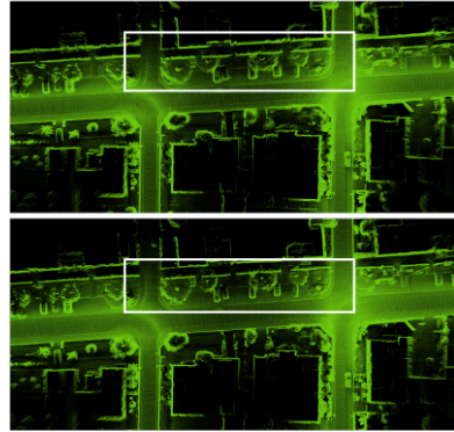
Fig. 15. The point cloud of MID-100 lidar in Scene-1 reconstructed before (A) and after (B) extrinsic optimization. The base lidar (L_1) point cloud is colored in white and the two lidars to be calibrated (L_0 , L_2) are colored in orange and green, respectively.

TABLE VI
AVERAGE TIME (SECOND) PER ITERATION FOR MULTI-LIDAR
CALIBRATION

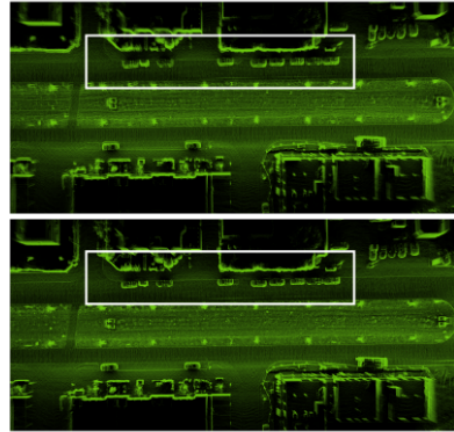
Method	Pose Optimization	Extrinsic Optimization	Joint Optimization
ICP-Based [9]	5.16	6.10	14.98
BALM-Based [8]	0.23	0.58	3.09
Proposed	0.19	0.20	0.80

relative transformation of these poses and that estimated by the odometry. One drawback of the PGO is that it cannot reinforce the map quality directly. A divergence in point cloud mapping would occur if the estimation of the relative poses near the loop is ill. We show that the mapping quality (and odometry accuracy) could be further improved with our BA method even after PGO is performed.

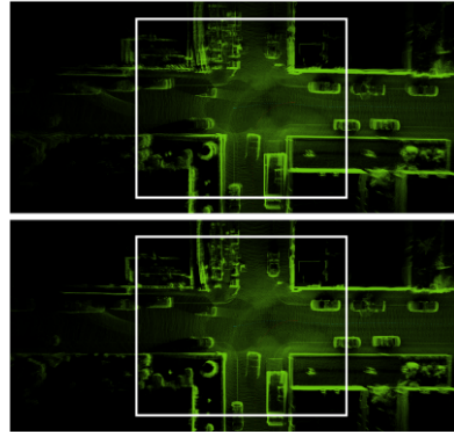
We choose to validate on KITTI dataset [13]. The initial lidar odometry is provided by the state-of-the-art SLAM algorithm MULS [12] with loop closure function enabled. We directly feed this odometry to our BA algorithm [11] (and all other BA methods under comparison) and concurrently optimize the entire poses. The optimization results are summarized in Table VII and Fig. 16. It is seen that even with loop closure function, some divergence still exist in the area near the loop of the point cloud due to ill estimations. With our proposed global BA refinement, the accuracy in odometry is further improved and the divergence in point cloud is eliminated. When compared to other BA methods, our method achieves higher accuracy while converges significantly faster as shown in Fig. 17. Moreover, we compare the results with one state-of-the-art lidar SLAM, CT-ICP [14] augmented with loop-closure [15]. The ATE of our method is lower than CT-ICP in most sequences and also the average value. For sequences that our method is outperformed by CT-ICP (i.e., sequence 02, 08, and 09), we compare the mapping quality, which is shown in Fig. 18. As can be seen, despite the larger ATE of our method, the



(a) Sequence 05



(b) Sequence 06



(c) Sequence 07

Fig. 16. Comparison of map consistency on KITTI dataset [13]. The image above of each subplot depicts the point cloud reconstructed by MULS [12] which is used as our initial value. The image blow of each subplot is the point cloud optimized by our proposed global BA method.

obtained map is more consistent, which is due to the direct optimization on the mapping consistency of our global BA.

TABLE VII
ABSOLUTE TRAJECTORY ERROR (RMSE,METERS) ON KITTI.

Sequence	00	01	02	03	04	05	06	07	08	09	10	Mean
MULLS	1.09	1.96	5.42	0.74	0.89	0.97	0.31	0.44	2.93	2.12	1.13	1.63
CT-ICP	1.68	2.25	4.06	0.67	0.67	0.76	0.34	0.40	2.52	0.91	0.83	1.40
EF	1.02	1.94	5.28	0.70	0.82	0.84	0.31	0.43	2.80	1.98	0.99	1.55
BALM	0.96	1.90	5.21	0.68	0.75	0.72	0.28	0.40	2.72	1.75	0.92	1.48
PA (inner)	0.86	1.84	5.08	0.58	0.64	0.66	0.23	0.31	2.63	1.50	0.80	1.37
BAREG	0.89	1.88	5.12	0.65	0.70	0.69	0.24	0.35	2.68	1.59	0.88	1.42
Our	0.84	1.83	5.06	0.57	0.64	0.62	0.21	0.30	2.59	1.48	0.78	1.34

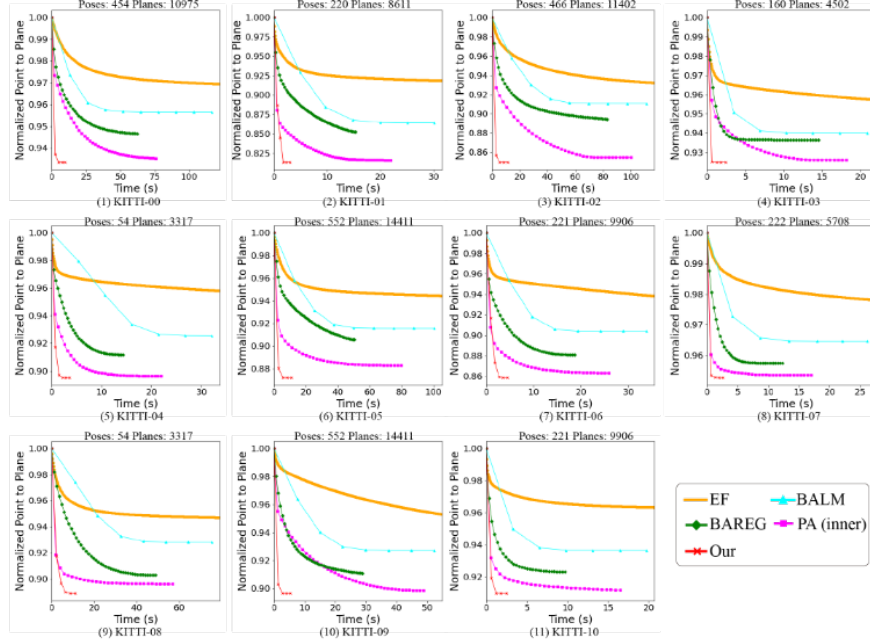


Fig. 17. Point-to-plane distance versus optimization time in KITTI dataset. All methods have the same initial pose (hence the same initial point-to-plane distance) and have their point-to-plane distance all normalized by the initial values.

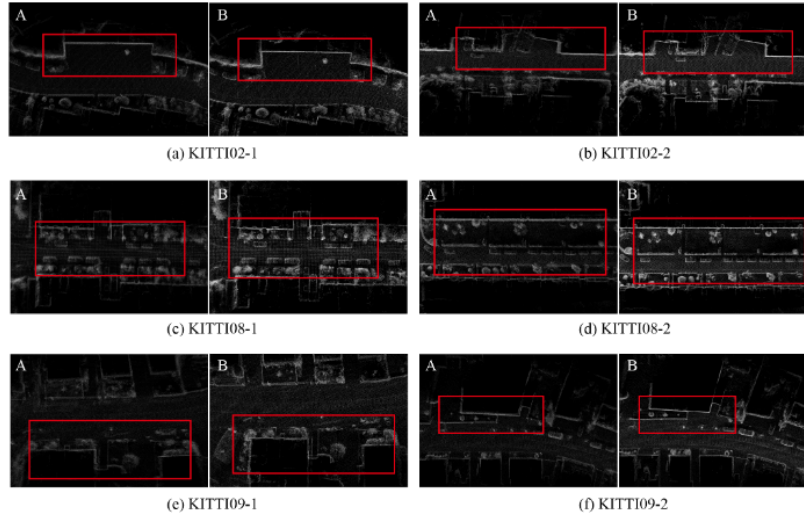


Fig. 18. Comparison of map consistency between CT-ICP(A) and our method(B) in KITTI sequence 02, 08 and 09.

II. LEMMAS

Lemma 1. For a scalar $x \in \mathbb{R}$ and a matrix $\mathbf{A} \in \mathbb{S}^{3 \times 3}$ which depends on x , we have the two following conclusions.

$$\frac{\partial \lambda_l(x)}{\partial x} = \mathbf{u}_l(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{u}_l(x) \quad (58)$$

$$\frac{\partial \mathbf{u}_l(x)}{\partial x} = \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k(x) \mathbf{u}_k(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{u}_l(x) \quad (59)$$

where λ_l ($l = 1, 2, 3$) denotes the l -th largest eigenvalue and $\mathbf{u}_l$ is the corresponding eigenvector.

Proof. Since the matrix $\mathbf{A}(x)$ is symmetric, its singular value decomposition is,

$$\mathbf{A}(x) = \mathbf{U}(x) \mathbf{\Lambda}(x) \mathbf{U}(x)^T \quad (60)$$

where $\mathbf{\Lambda}(x) = \text{diag}(\lambda_1(x), \lambda_2(x), \lambda_3(x))$ consists of all the eigenvalues and $\mathbf{U}(x) = [\mathbf{u}_1(x) \ \mathbf{u}_2(x) \ \mathbf{u}_3(x)]$ is an orthonormal matrix consisting of the eigenvectors. Therefore,

$$\mathbf{\Lambda}(x) = \mathbf{U}(x)^T \mathbf{A}(x) \mathbf{U}(x) \quad (61)$$

Both sides take the derivative of x ,

$$\begin{aligned} \frac{\partial \mathbf{\Lambda}(x)}{\partial x} &= \mathbf{U}(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{U}(x) + \mathbf{U}(x)^T \mathbf{A}(x) \frac{\partial \mathbf{U}(x)}{\partial x} \\ &\quad + \left(\frac{\partial \mathbf{U}(x)}{\partial x} \right)^T \mathbf{A}(x) \mathbf{U}(x) \end{aligned} \quad (62)$$

Since $\mathbf{U}(x)^T \mathbf{A}(x) = \mathbf{\Lambda}(x) \mathbf{U}(x)^T$ and $\mathbf{A}(x) \mathbf{U}(x) = \mathbf{U}(x) \mathbf{\Lambda}(x)$, the equation is

$$\begin{aligned} \frac{\partial \mathbf{\Lambda}(x)}{\partial x} &= \mathbf{U}(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{U}(x) + \underbrace{\mathbf{\Lambda}(x) \mathbf{U}(x)^T \frac{\partial \mathbf{U}(x)}{\partial x}}_{\mathbf{D}(x)} \\ &\quad + \underbrace{\left(\frac{\partial \mathbf{U}(x)}{\partial x} \right)^T \mathbf{U}(x) \mathbf{\Lambda}(x)}_{\mathbf{D}^T(x)} \end{aligned} \quad (63)$$

Denote $\mathbf{D}(x) \triangleq \mathbf{U}(x)^T \frac{\partial \mathbf{U}(x)}{\partial x}$. Since $\mathbf{U}(x) \mathbf{U}(x)^T = \mathbf{I}$, differentiating both sides with respect to x leads to,

$$\begin{aligned} \mathbf{U}(x)^T \frac{\partial \mathbf{U}(x)}{\partial x} + \left(\frac{\partial \mathbf{U}(x)}{\partial x} \right)^T \mathbf{U}(x) &= \mathbf{0} \\ \Rightarrow \mathbf{D}(x) + \mathbf{D}^T(x) &= \mathbf{0} \end{aligned}$$

It is seen that $\mathbf{D}(x)$ is a skew symmetric matrix whose diagonal elements are zeros. Moreover, since $\mathbf{\Lambda}(x)$ is diagonal, the last two items of the right side of (63) sum to zero on diagonal positions. Only considering the diagonal elements in (63) leads to

$$\frac{\partial \lambda_l(x)}{\partial x} = \mathbf{u}_l(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{u}_l(x), l \in \{1, 2, 3\} \quad (64)$$

which yields the first conclusion. Now we aims to prove the second one. In (63), $\frac{\partial \mathbf{\Lambda}(x)}{\partial x}$ is diagonal matrix and thus for the off-diagonal, k -th row, l -th column, element ($k \neq l$),

$$0 = \mathbf{u}_k(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{u}_l(x) + \lambda_k D_x^{k,l} - D_x^{k,l} \lambda_l \quad (65)$$

where $D_x^{k,l}$ is the k -th row, l -th column element in the skew symmetric $\mathbf{D}(x)$ and satisfy $D_x^{k,l} = -D_x^{l,k}$. From (65), we can solve $D_x^{k,l}$

$$D_x^{k,l} = \begin{cases} \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{u}_l(x), & k \neq l \\ 0, & k = l \end{cases} \quad (66)$$

Since $\mathbf{D}(x) \triangleq \mathbf{U}(x)^T \frac{\partial \mathbf{U}(x)}{\partial x}$, we have $\frac{\partial \mathbf{U}(x)}{\partial x} = \mathbf{U}(x) \mathbf{D}(x)$. Taking the l -th column on both sides leads to

$$\frac{\partial \mathbf{u}_l(x)}{\partial x} = \mathbf{U}(x) \mathbf{D}_x^{:,l}, \quad (67)$$

where $\mathbf{D}_x^{:,l} \in \mathbb{R}^3$ represents the l -th column of $\mathbf{D}(x)$. Finally, substituting $\mathbf{U}(x) = [\mathbf{u}_1(x) \ \mathbf{u}_2(x) \ \mathbf{u}_3(x)]$ and (66) into (67), we obtain

$$\frac{\partial \mathbf{u}_l(x)}{\partial x} = \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k(x) \mathbf{u}_k(x)^T \frac{\partial \mathbf{A}(x)}{\partial x} \mathbf{u}_l(x), \quad (68)$$

which yields the second conclusion. $\square$

Lemma 2. Given

- (1) Matrices $\mathbf{C}_j = \begin{bmatrix} \mathbf{P}_j & \mathbf{v}_j \\ \mathbf{v}_j^T & N_j \end{bmatrix} \in \mathbb{S}^{4 \times 4}, j = 1, \dots, M_p$;
- (2) Poses $\mathbf{T}_j \in SE(3), j = 1, \dots, M_p$;
- (3) A matrix $\mathbf{C} = \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} \triangleq \sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \in \mathbb{S}^{4 \times 4}$, which is the aggregation of $\mathbf{C}_j$, and a matrix function $\mathbf{A}(\mathbf{C}) \triangleq \frac{1}{N} \mathbf{P} - \frac{1}{N^2} \mathbf{v} \mathbf{v}^T \in \mathbb{S}^{3 \times 3}$;
- (4) Two constant vectors $\mathbf{u}_k, \mathbf{u}_l \in \mathbb{R}^3$,

then the first and second order derivatives of $\mathbf{u}_k^T \mathbf{A}(\mathbf{T}) \mathbf{u}_l$ w.r.t. $\mathbf{T}$ are:

$$\mathbf{g}_{kl} \triangleq \frac{\partial \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} = [\dots \ \mathbf{g}_{kl}^j \ \dots] \in \mathbb{R}^{1 \times 6M_p}, \quad (69)$$

$$\mathbf{Q}_{kl} \triangleq \frac{\partial^2 \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial (\delta \mathbf{T})^2} = \begin{bmatrix} \dots & \vdots & \dots \\ \dots & \mathbf{Q}_{kl}^{ij} & \dots \\ \dots & \vdots & \dots \end{bmatrix} \in \mathbb{R}^{6M_p \times 6M_p}, \quad (70)$$

where $\mathbf{g}_{kl}^j \in \mathbb{R}^{1 \times 6}, \mathbf{Q}_{kl}^{ij} \in \mathbb{R}^{6 \times 6}, \forall i, j \in \{1, \dots, M_p\}$, are block elements of $\mathbf{g}_{kl}$ and $\mathbf{Q}_{kl}$ defined as below

$$\begin{aligned} \mathbf{g}_{kl}^j &= \frac{1}{N} \mathbf{u}_l^T \mathbf{S}_p(\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_k^T \\ &\quad + \frac{1}{N} \mathbf{u}_k^T \mathbf{S}_p(\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \end{aligned} \quad (71)$$

$$\begin{aligned} \mathbf{Q}_{kl}^{ij} &= -\frac{2}{N^2} \mathbf{V}_k \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \mathbb{1}_{i=j} \cdot \left(\frac{2}{N} \mathbf{V}_k \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \right. \\ &\quad \left. + \begin{bmatrix} \mathbf{K}_{kl}^j & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \right) \end{aligned} \quad (72)$$

$$\begin{aligned} \mathbf{K}_{kl}^j &= \frac{1}{N} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_k] [\mathbf{u}_l] \\ &\quad + \frac{1}{N} [\mathbf{u}_k] [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l] \end{aligned} \quad (73)$$

where

$$\mathbf{V}_l = \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_l \end{bmatrix} \quad \mathbf{F} = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (74)$$

$$\mathbf{S}_\mathbf{P} = [\mathbf{I}_{3 \times 3} \quad \mathbf{0}_{3 \times 1}] \quad \mathbb{1}_{i=j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}. \quad (75)$$

Additionally, the block elements $\mathbf{g}_{kl}^j, \mathbf{Q}_{kl}^{ij}$ satisfy that $\forall \mathbf{u}_k, \mathbf{u}_l$,

$$\mathbf{g}_{kl}^j = \mathbf{0}_{1 \times 6}, \text{ if } \mathbf{C}_j = 0, \quad (76)$$

$$\mathbf{Q}_{kl}^{ij} = \mathbf{0}_{6 \times 6}, \text{ if } \mathbf{C}_i = 0 \text{ or } \mathbf{C}_j = 0. \quad (77)$$

Proof. Partition the matrix $\mathbf{C}$ as

$$\mathbf{C} = \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} = \sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T, \quad (78)$$

then

$$\mathbf{P} = \mathbf{S}_\mathbf{P} \mathbf{C} \mathbf{S}_\mathbf{P}^T = \sum_{j=1}^{M_p} (\mathbf{R}_j \mathbf{P}_j \mathbf{R}_j^T + \mathbf{R}_j \mathbf{v}_j \mathbf{t}_j^T + \mathbf{t}_j \mathbf{v}_j^T \mathbf{R}_j^T + N_j \mathbf{t}_j \mathbf{t}_j^T), \quad (79)$$

$$\mathbf{v} = \mathbf{S}_\mathbf{P} \mathbf{C} \mathbf{S}_\mathbf{V}^T = \sum_{j=1}^{M_p} (\mathbf{R}_j \mathbf{v}_j + N_j \mathbf{t}_j), \quad (80)$$

$$N = \sum_{j=1}^{M_p} N_j, \quad (81)$$

where

$$\mathbf{S}_\mathbf{P} = [\mathbf{I}_{3 \times 3} \quad \mathbf{0}_{3 \times 1}] \in \mathbb{R}^{3 \times 4}, \quad (82)$$

$$\mathbf{S}_\mathbf{V} = [\mathbf{0}_{1 \times 3} \quad 1] \in \mathbb{R}^{1 \times 4}. \quad (83)$$

Therefore,

$$\mathbf{A} \left(\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \right) = \frac{1}{N} \mathbf{P} - \frac{1}{N^2} \mathbf{v} \mathbf{v}^T \quad (84)$$

$$= \frac{1}{N} \mathbf{S}_\mathbf{P} \left(\mathbf{C} - \frac{1}{N} \mathbf{C} \mathbf{S}_\mathbf{V}^T \mathbf{S}_\mathbf{V} \mathbf{C}^T \right) \mathbf{S}_\mathbf{P}^T \quad (85)$$

$$= \frac{1}{N} \mathbf{S}_\mathbf{P} \left(\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T - \frac{1}{N} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{T}_i \mathbf{C}_i \mathbf{T}_i^T \mathbf{S}_\mathbf{V}^T \mathbf{S}_\mathbf{V} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \right) \mathbf{S}_\mathbf{P}^T. \quad (86)$$

Since $\mathbf{T}_i^T \mathbf{S}_\mathbf{V}^T \mathbf{S}_\mathbf{V} \mathbf{T}_j = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \triangleq \mathbf{F}$, we obtain

$$\mathbf{A} = \frac{1}{N} \mathbf{S}_\mathbf{P} \left(\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T - \frac{1}{N} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \right) \mathbf{S}_\mathbf{P}^T, \quad (87)$$

where we omitted the input argument of $\mathbf{A}$ for the sake of notation simplicity. Since $N = \sum_{j=1}^{M_p} N_j$ is a constant number that is irrelevant to the pose $\mathbf{T}$, perturbing the pose $\mathbf{T}$ (i.e., the input of $\mathbf{A}$) by $\delta \mathbf{T}$ yields

$$\begin{aligned} \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l &= \frac{1}{N} \mathbf{u}_k^T \mathbf{S}_\mathbf{P} \left(\sum_{j=1}^{M_p} (\mathbf{T}_j \boxplus \delta \mathbf{T}_j) \mathbf{C}_j (\mathbf{T}_j \boxplus \delta \mathbf{T}_j)^T \right. \\ &\quad \left. - \frac{1}{N} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} (\mathbf{T}_i \boxplus \delta \mathbf{T}_i) (\mathbf{C}_i \mathbf{F} \mathbf{C}_j) (\mathbf{T}_j \boxplus \delta \mathbf{T}_j)^T \right) \mathbf{S}_\mathbf{P}^T \mathbf{u}_l. \end{aligned} \quad (88)$$

Based on the definition of $\boxplus$ on $SE(3)$ in (27), we define,

$$\begin{aligned} \mathbf{w}_{jl}(\delta \mathbf{T}_j) &\triangleq (\mathbf{T}_j \boxplus \delta \mathbf{T}_j)^T \mathbf{S}_\mathbf{P}^T \mathbf{u}_l \\ &= \begin{bmatrix} \mathbf{R}_j^T \exp^T([\delta \phi_j]) \mathbf{u}_l \\ \mathbf{t}_j^T \exp^T([\delta \phi_j]) \mathbf{u}_l + \mathbf{u}_l^T \delta \mathbf{t}_j \end{bmatrix}. \end{aligned} \quad (89)$$

When $\delta \phi_j$ is small, which is indeed the case for the purpose of derivative computation, we have

$$\exp([\delta \phi_j]) \approx \mathbf{I} + [\delta \phi_j] + \frac{1}{2} [\delta \phi_j]^2 \quad (90)$$

Substituting (90) into $\mathbf{w}_{jl}(\delta \mathbf{T}_j)$, we obtain

$$\begin{aligned} \mathbf{w}_{jl}(\delta \mathbf{T}_j) &\approx \begin{bmatrix} \mathbf{R}_j^T (\mathbf{I} - [\delta \phi_j] + \frac{1}{2} [\delta \phi_j]^2) \mathbf{u}_l \\ \mathbf{t}_j^T (\mathbf{I} - [\delta \phi_j] + \frac{1}{2} [\delta \phi_j]^2) \mathbf{u}_l + \mathbf{u}_l^T \delta \mathbf{t}_j \end{bmatrix} \\ &\approx \underbrace{\begin{bmatrix} \mathbf{R}_j^T \mathbf{u}_l \\ \mathbf{t}_j^T \mathbf{u}_l \end{bmatrix}}_{\bar{\mathbf{w}}_{jl}} + \underbrace{\begin{bmatrix} \mathbf{R}_j^T [\mathbf{u}_l] & \mathbf{0}_{3 \times 3} \\ \mathbf{t}_j^T [\mathbf{u}_l] & \mathbf{u}_l^T \end{bmatrix}}_{\mathbf{J}_{\mathbf{w}_{jl}} \in \mathbb{R}^{4 \times 6}} \underbrace{\begin{bmatrix} \delta \phi_j \\ \delta \mathbf{t}_j \end{bmatrix}}_{\delta \mathbf{T}_j} + \underbrace{\begin{bmatrix} \frac{1}{2} \mathbf{R}_j^T [\delta \phi_j]^2 \mathbf{u}_l \\ \frac{1}{2} \mathbf{t}_j^T [\delta \phi_j]^2 \mathbf{u}_l \end{bmatrix}}_{\delta \Phi_{jl}} \end{aligned} \quad (91)$$

where $\bar{\mathbf{w}}_{jl}$, $\delta \Phi_{jl}$ and $\mathbf{J}_{\mathbf{w}_{jl}}$ can be simplified as

$$\bar{\mathbf{w}}_{jl} = (\mathbf{S}_\mathbf{P} \mathbf{T}_j)^T \mathbf{u}_l, \quad \delta \Phi_{jl} = \frac{1}{2} (\mathbf{S}_\mathbf{P} \mathbf{T}_j)^T [\delta \phi_j]^2 \mathbf{u}_l$$

$$\mathbf{J}_{\mathbf{w}_{jl}} = \mathbf{T}_j^T \mathbf{V}_l^T, \quad \text{where } \mathbf{V}_l = \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_l \end{bmatrix} \quad (92)$$

Substituting (91) into (88) and keeping terms up to the second order lead to

$$\begin{aligned} \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l &= \frac{1}{N} \sum_{j=1}^{M_p} \mathbf{w}_{jk}^T(\delta \mathbf{T}_j) \mathbf{C}_j \mathbf{w}_{jl}(\delta \mathbf{T}_j) \\ &\quad - \frac{1}{N^2} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{w}_{ik}^T(\delta \mathbf{T}_i) \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{w}_{jl}(\delta \mathbf{T}_j) \quad (93) \\ &= \frac{1}{N} \sum_{j=1}^{M_p} \left((\bar{\mathbf{w}}_{jk}^T \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jl}} + \bar{\mathbf{w}}_{jl}^T \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jk}}) \delta \mathbf{T}_j + \bar{\mathbf{w}}_{jk}^T \mathbf{C}_j \bar{\mathbf{w}}_{jl} \right. \\ &\quad \left. + \delta \mathbf{T}_j^T \mathbf{J}_{\mathbf{w}_{jk}}^T \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jl}} \delta \mathbf{T}_j + \underbrace{\bar{\mathbf{w}}_{jk}^T \mathbf{C}_j \delta \Phi_{jl} + \delta \Phi_{jk}^T \mathbf{C}_j \bar{\mathbf{w}}_{jl}}_{\frac{1}{2} \delta \mathbf{T}_j^T \mathbf{Y}_{kl}^j \delta \mathbf{T}_j} \right) \\ &\quad - \frac{1}{N^2} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \left(\delta \mathbf{T}_i^T \mathbf{J}_{\mathbf{w}_{ik}}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jl}} \delta \mathbf{T}_j \right. \\ &\quad \left. + \bar{\mathbf{w}}_{ik}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jl}} \delta \mathbf{T}_j + \delta \mathbf{T}_i^T \mathbf{J}_{\mathbf{w}_{ik}}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \bar{\mathbf{w}}_{jl} \right. \\ &\quad \left. + \bar{\mathbf{w}}_{ik}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \bar{\mathbf{w}}_{jl} + \underbrace{\bar{\mathbf{w}}_{ik}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \delta \Phi_{jl}}_{\frac{1}{2} \delta \mathbf{T}_j^T \mathbf{N}_{kl}^j \delta \mathbf{T}_j} + \underbrace{\delta \Phi_{ik}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \bar{\mathbf{w}}_{jl}}_{\frac{1}{2} \delta \mathbf{T}_i^T \mathbf{M}_{kl}^j \delta \mathbf{T}_i} \right). \end{aligned} \quad (94)$$

To compute $\mathbf{Y}_{kl}^j$ in $\frac{1}{2}\delta\mathbf{T}_j^T\mathbf{Y}_{kl}^j\delta\mathbf{T}_j$, note that $\mathbf{a}^T[\delta\phi]^2\mathbf{b} = \delta\phi^T[\mathbf{a}][\mathbf{b}]\delta\phi$, $\forall \mathbf{a}, \mathbf{b}, \delta\phi \in \mathbb{R}^3$, we have

$$\begin{aligned}\bar{\mathbf{w}}_{jk}^T\mathbf{C}_j\delta\Phi_{jl} &= \frac{1}{2}\mathbf{u}_k^T\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T[\delta\phi_j]^2\mathbf{u}_l \\ &= \frac{1}{2}\delta\phi_j^T[\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_k][\mathbf{u}_l]\delta\phi_j.\end{aligned}\quad (96)$$

Similarly,

$$\delta\Phi_{jk}^T\mathbf{C}_j\bar{\mathbf{w}}_{jl} = \frac{1}{2}\delta\phi_j^T[\mathbf{u}_k][\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_l]\delta\phi_j. \quad (97)$$

Summing (96) and (97) and extending the $\delta\phi$ into $\delta\mathbf{T}$:

$$\mathbf{Y}_{kl}^j = \begin{bmatrix} [\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_k][\mathbf{u}_l] + [\mathbf{u}_k][\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (98)$$

For $\frac{1}{2}\delta\mathbf{T}_j^T\mathbf{N}_{kl}^{ij}\delta\mathbf{T}_j$ and $\frac{1}{2}\delta\mathbf{T}_i^T\mathbf{M}_{kl}^{ij}\delta\mathbf{T}_i$,

$$\begin{aligned}\bar{\mathbf{w}}_{ik}^T\mathbf{C}_i\mathbf{F}\mathbf{C}_j\delta\Phi_{jl} &= \frac{1}{2}\mathbf{u}_k^T\mathbf{S}_\mathbf{P}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T[\delta\phi_j]^2\mathbf{u}_l \\ &= \frac{1}{2}\delta\phi_j^T[\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{F}\mathbf{C}_i\mathbf{T}_i^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_k][\mathbf{u}_l]\delta\phi_j \quad (99) \\ \delta\Phi_{ik}^T\mathbf{C}_i\mathbf{F}\mathbf{C}_j\bar{\mathbf{w}}_{jl} &= \frac{1}{2}\mathbf{u}_k^T[\delta\phi_i]^2\mathbf{S}_\mathbf{P}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_l \\ &= \frac{1}{2}\delta\phi_i^T[\mathbf{u}_k][\mathbf{S}_\mathbf{P}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_l]\delta\phi_i\end{aligned}\quad (100)$$

Thus, extending the $\delta\phi$ into $\delta\mathbf{T}$, we obtain

$$\mathbf{N}_{kl}^{ij} = \begin{bmatrix} [\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{F}\mathbf{C}_i\mathbf{T}_i^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_k][\mathbf{u}_l] & \mathbf{0}_{3\times 3} \\ \mathbf{0}_{3\times 3} & \mathbf{0}_{3\times 3} \end{bmatrix} \quad (101)$$

$$\mathbf{M}_{kl}^{ij} = \begin{bmatrix} [\mathbf{u}_k][\mathbf{S}_\mathbf{P}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{T}_j^T\mathbf{S}_\mathbf{P}^T\mathbf{u}_l] & \mathbf{0}_{3\times 3} \\ \mathbf{0}_{3\times 3} & \mathbf{0}_{3\times 3} \end{bmatrix} \quad (102)$$

It can be seen that (95) is quadratic w.r.t. $\delta\mathbf{T}$, so we cast it into the following standard form

$$\mathbf{u}_k^T\mathbf{A}(\delta\mathbf{T})\mathbf{u}_l = r_{kl} + \mathbf{g}_{kl} \cdot \delta\mathbf{T} + \frac{1}{2}\delta\mathbf{T}^T \cdot \mathbf{Q}_{kl} \cdot \delta\mathbf{T}, \quad (103)$$

where $\mathbf{g}_{kl}$ and $\mathbf{Q}_{kl}$ are partitioned as

$$\mathbf{g}_{kl} = [\cdots \quad \mathbf{g}_{kl}^j \quad \cdots] \in \mathbb{R}^{1\times 6M_p} \quad (104)$$

$$\mathbf{Q}_{kl} = \begin{bmatrix} \cdots & \vdots & \cdots \\ \cdots & \mathbf{Q}_{kl}^{ij} & \cdots \\ \cdots & \vdots & \cdots \end{bmatrix} \in \mathbb{R}^{6M_p \times 6M_p}. \quad (105)$$

For $\mathbf{g}_{kl}^j \in \mathbb{R}^{1\times 6}$, $\forall j \in \{1, \dots, M_p\}$, the j -th column block of $\mathbf{g}_{kl}$ in (104), it is

$$\begin{aligned}\mathbf{g}_{kl}^j &= \frac{1}{N}(\bar{\mathbf{w}}_{jk}^T\mathbf{C}_j\mathbf{J}_{\mathbf{w}_{jl}} + \bar{\mathbf{w}}_{jl}^T\mathbf{C}_j\mathbf{J}_{\mathbf{w}_{jk}}) \\ &\quad - \frac{1}{N^2}\sum_{i=1}^{M_p}(\bar{\mathbf{w}}_{ik}^T\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{J}_{\mathbf{w}_{jl}} + \bar{\mathbf{w}}_{il}^T\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{J}_{\mathbf{w}_{jk}}) \quad (106) \\ &= \frac{1}{N}(\mathbf{u}_k^T\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{V}_l^T + \mathbf{u}_l^T\mathbf{S}_\mathbf{P}\mathbf{T}_j\mathbf{C}_j\mathbf{T}_j^T\mathbf{V}_k^T) \\ &\quad - \frac{1}{N^2}\sum_{i=1}^{M_p}(\mathbf{u}_k^T\mathbf{S}_\mathbf{P}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{T}_j^T\mathbf{V}_l^T + \mathbf{u}_l^T\mathbf{S}_\mathbf{P}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{T}_j^T\mathbf{V}_k^T) \\ &= \frac{1}{N}\mathbf{u}_l^T\mathbf{S}_\mathbf{P}(\mathbf{T}_j\mathbf{C}_j - \frac{1}{N}\sum_{i=1}^{M_p}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j)\mathbf{T}_j^T\mathbf{V}_k^T \\ &\quad + \frac{1}{N}\mathbf{u}_k^T\mathbf{S}_\mathbf{P}(\mathbf{T}_j\mathbf{C}_j - \frac{1}{N}\sum_{i=1}^{M_p}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j)\mathbf{T}_j^T\mathbf{V}_l^T \quad (107)\end{aligned}$$

Since $\mathbf{v} = \sum_{j=1}^{M_p}(\mathbf{R}_j\mathbf{v}_j + N_j\mathbf{t}_j)$ from (80) and $\mathbf{F} = \begin{bmatrix} \mathbf{0}_{3\times 1} & \mathbf{0}_{3\times 1} \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix}$, we have

$$\begin{aligned}\sum_{i=1}^{M_p}\mathbf{T}_i\mathbf{C}_i\mathbf{F} &= \sum_{i=1}^{M_p}\begin{bmatrix} \mathbf{R}_i & \mathbf{t}_i \\ \mathbf{0} & 1 \end{bmatrix}\begin{bmatrix} \mathbf{P}_i & \mathbf{v}_i \\ \mathbf{v}_i^T & N_i \end{bmatrix}\begin{bmatrix} \mathbf{0}_{3\times 1} & \mathbf{0}_{3\times 1} \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix} \\ &= \sum_{i=1}^{M_p}\begin{bmatrix} \mathbf{0}_{3\times 1} & \mathbf{R}_i\mathbf{v}_i + N_i\mathbf{t}_i \\ \mathbf{0}_{1\times 3} & N_i \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{v} \\ \mathbf{0} & N_i \end{bmatrix} = \mathbf{C}\mathbf{F}. \quad (108)\end{aligned}$$

Hence, the part $(\mathbf{T}_j\mathbf{C}_j - \frac{1}{N}\sum_{i=1}^{M_p}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j)$ in (107) is

$$\begin{aligned}\mathbf{T}_j\mathbf{C}_j - \frac{1}{N}\sum_{i=1}^{M_p}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\mathbf{C}_j &= \mathbf{T}_j\mathbf{C}_j - \frac{1}{N}\underbrace{\left(\sum_{i=1}^{M_p}\mathbf{T}_i\mathbf{C}_i\mathbf{F}\right)}_{\mathbf{C}\mathbf{F}}\mathbf{C}_j \\ &= (\mathbf{T}_j - \frac{1}{N}\mathbf{C}\mathbf{F})\mathbf{C}_j.\end{aligned}\quad (109)$$

Therefore,

$$\begin{aligned}\mathbf{g}_{kl}^j &= \frac{1}{N}\mathbf{u}_l^T\mathbf{S}_\mathbf{P}(\mathbf{T}_j - \frac{1}{N}\mathbf{C}\mathbf{F})\mathbf{C}_j\mathbf{T}_j^T\mathbf{V}_k^T \\ &\quad + \frac{1}{N}\mathbf{u}_k^T\mathbf{S}_\mathbf{P}(\mathbf{T}_j - \frac{1}{N}\mathbf{C}\mathbf{F})\mathbf{C}_j\mathbf{T}_j^T\mathbf{V}_l^T,\end{aligned}\quad (110)$$

which yields the result in (71). Additionally, it is seen that if $\mathbf{C}_j = \mathbf{0}_{4\times 4}$, $\mathbf{g}_{kl}^j = \mathbf{0}_{1\times 6}$, $\forall \mathbf{u}_k, \mathbf{u}_l$.

For $\mathbf{Q}_{kl}^{ij} \in \mathbb{R}^{6\times 6}$, $\forall i, j \in \{1, \dots, M_p\}$, the i -th row, j -th column block of $\mathbf{Q}_{kl}$ in (105), it is

$$\begin{aligned}\mathbf{Q}_{kl}^{ij} &= -\frac{2}{N^2}\mathbf{J}_{\mathbf{w}_{ik}}^T\mathbf{C}_i\mathbf{F}\mathbf{C}_j\mathbf{J}_{\mathbf{w}_{jl}} + \mathbf{1}_{i=j} \cdot \left(\frac{2}{N}\mathbf{J}_{\mathbf{w}_{jk}}^T\mathbf{C}_j\mathbf{J}_{\mathbf{w}_{jl}}\right) \\ &\quad + \frac{1}{N}\mathbf{Y}_{kl}^j - \frac{1}{N^2}\sum_{\nu=1}^{M_p}(\mathbf{N}_{kl}^{\nu j} + \mathbf{M}_{kl}^{i\nu})\end{aligned}\quad (111)$$

where

$$\mathbb{1}_{i=j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad (112)$$

$$\mathbf{J}_{\mathbf{w}_{ik}}^T \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jl}} = \mathbf{V}_k \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \quad (113)$$

$$\mathbf{J}_{\mathbf{w}_{jk}}^T \mathbf{C}_j \mathbf{J}_{\mathbf{w}_{jl}} = \mathbf{V}_k \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \quad (114)$$

$$\sum_{\nu=1}^{M_p} (\mathbf{N}_{kl}^{\nu j} + \mathbf{M}_{kl}^{i\nu}) = \sum_{\nu=1}^{M_p} \begin{bmatrix} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j \mathbf{F} \mathbf{C}_\nu \mathbf{T}_\nu^T \mathbf{S}_p^T \mathbf{u}_k] [\mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} + \sum_{\nu=1}^{M_p} \begin{bmatrix} [\mathbf{u}_k] [\mathbf{S}_p \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_\nu \mathbf{T}_\nu^T \mathbf{S}_p^T \mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (115)$$

By using similar method in (108), we have

$$\sum_{\nu=1}^{M_p} \mathbf{F} \mathbf{C}_\nu \mathbf{T}_\nu^T = \mathbf{F} \mathbf{C} \quad (116)$$

and

$$\sum_{\nu=1}^{M_p} (\mathbf{N}_{kl}^{\nu j} + \mathbf{M}_{kl}^{i\nu}) = \begin{bmatrix} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j \mathbf{F} \mathbf{C}_p^T \mathbf{u}_k] [\mathbf{u}_l] + [\mathbf{u}_k] [\mathbf{S}_p \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_p^T \mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (117)$$

Thus, the matrix $\mathbf{Q}_{kl}^{ij}$ is

$$\mathbf{Q}_{kl}^{ij} = -\frac{2}{N^2} \mathbf{V}_k \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \mathbb{1}_{i=j} \cdot \left(\frac{2}{N} \mathbf{V}_k \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \begin{bmatrix} \mathbf{K}_{kl}^j & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \right) \quad (118)$$

$$\begin{aligned} \mathbf{K}_{kl}^j &= \frac{1}{N} \mathbf{Y}_{kl}^j - \frac{1}{N^2} \sum_{\nu=1}^{M_p} (\mathbf{N}_{kl}^{\nu j} + \mathbf{M}_{kl}^{i\nu}), \quad \text{with } i = j \\ &= \frac{1}{N} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_k] [\mathbf{u}_l] \\ &\quad + \frac{1}{N} [\mathbf{u}_k] [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l], \end{aligned} \quad (119)$$

which yields the results in (72) and (73). Additionally, it can be seen that if $\mathbf{C}_i = \mathbf{0}_{4 \times 4}$ or $\mathbf{C}_j = \mathbf{0}_{4 \times 4}$, we have $\mathbf{C}_i \mathbf{F} \mathbf{C}_j = \mathbf{0}_{4 \times 4}$, $\mathbf{K}_{kl}^j = \mathbf{0}_{3 \times 3}$ and then $\mathbf{Q}_{kl}^{ij} = \mathbf{0}_{6 \times 6}, \forall \mathbf{u}_k, \mathbf{u}_l$. $\square$

III. PROOF OF THEOREMS

A. Proof of formula (6) and (8)

Proof. The variable to be optimized is $\boldsymbol{\pi}_i = (\mathbf{n}_i, \mathbf{q}_i)$ and the cost function is

$$c_i = \min_{\boldsymbol{\pi}_i = (\mathbf{n}_i, \mathbf{q}_i)} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|\mathbf{h}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2^2 \right) \quad (120)$$

where $\mathbf{h}_i = \mathbf{n}_i$ for plane feature and $\mathbf{h}_i = (\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T)$ for edge feature. The dimensions of $\mathbf{h}_i$ may be different for these two features but it has no influence on following derivation.

$$c_i = \min_{\mathbf{n}_i} \min_{\mathbf{q}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|\mathbf{h}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i)\|_2^2 \right) \quad (121)$$

$$= \min_{\mathbf{n}_i} \left(\min_{\mathbf{q}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} - \mathbf{q}_i)^T \mathbf{h}_i \mathbf{h}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i) \right) \right).$$

As can be seen, the inner optimization on $\mathbf{q}_i$ is a standard quadratic optimization problem. So, the optimum $\mathbf{q}_i^*$ can be solved by setting the derivative to zero:

$$\begin{aligned} 2 \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \mathbf{h}_i \mathbf{h}_i^T (\mathbf{p}_{ijk} - \mathbf{q}_i) &= \mathbf{0} \implies \\ 2 \mathbf{h}_i \mathbf{h}_i^T \left(\sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \mathbf{p}_{ijk} - N_i \mathbf{q}_i \right) &= \mathbf{0} \end{aligned} \quad (122)$$

where $N_i = \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} 1 = \sum_{j=1}^{M_p} N_{ij}$. This equation does not lead to a unique solution of $\mathbf{q}_i$, one particular optimum solution is $\mathbf{q}_i^* = \frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \mathbf{p}_{ijk} = \bar{\mathbf{p}}_i$ as is defined in (7).

Now, substituting the optimum solution $\mathbf{q}_i^* = \bar{\mathbf{p}}_i$ into (121) leads to:

$$c_i = \min_{\mathbf{n}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|\mathbf{h}_i^T (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)\|_2^2 \right) \quad (123)$$

To solve for the optimal parameter $\mathbf{n}_i$ in the above optimization problem, we discuss the case of plane and edge features separately, as follows.

1) *Plane feature:* $\mathbf{h}_i = \mathbf{n}_i$.

$$\begin{aligned} c_i &= \min_{\|\mathbf{n}_i\|_2=1} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|\mathbf{n}_i^T (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)\|_2^2 \right) \\ &= \min_{\|\mathbf{n}_i\|_2=1} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{n}_i^T (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i) (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)^T \mathbf{n}_i) \right) \\ &= \min_{\mathbf{n}_i} \mathbf{n}_i^T \mathbf{A}_i \mathbf{n}_i, \end{aligned} \quad (124)$$

where $\mathbf{A}_i$ is defined in (7) and is a symmetric matrix. Performing Singular Value Decomposition (SVD) of $\mathbf{A}_i$

$$\mathbf{A}_i = \mathbf{U}_i \mathbf{\Lambda}_i \mathbf{U}_i^T \quad (125)$$

where

$$\mathbf{U}_i = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \mathbf{u}_3] \quad \mathbf{\Lambda}_i = \text{diag}(\lambda_1 \quad \lambda_2 \quad \lambda_3) \quad (126)$$

with $\lambda_1 \geq \lambda_2 \geq \lambda_3$ and $\mathbf{U}_i^T \mathbf{U}_i = \mathbf{I}$.

Denote $\mathbf{m} = \mathbf{U}_i \mathbf{n}_i = [m_1 \quad m_2 \quad m_3]^T$, $\|\mathbf{m}\|_2 = \sqrt{\mathbf{n}_i^T \mathbf{U}_i^T \mathbf{U}_i \mathbf{n}_i} = 1$, then (124) reduces to

$$\begin{aligned} c_i &= \min_{\|\mathbf{n}_i\|_2=1} (\mathbf{n}_i^T \mathbf{U}_i \mathbf{\Lambda}_i \mathbf{U}_i^T \mathbf{n}_i) = \min_{\|\mathbf{m}\|_2=1} (\mathbf{m}^T \mathbf{\Lambda}_i \mathbf{m}) \\ &= \min_{\|\mathbf{m}\|_2=1} (\lambda_1 m_1^2 + \lambda_2 m_2^2 + \lambda_3 m_3^2) \\ &\geq \min_{\|\mathbf{m}\|_2=1} (\lambda_3 m_1^2 + \lambda_3 m_2^2 + \lambda_3 m_3^2) = \lambda_3, \end{aligned} \quad (127)$$

where the minimum value λ_3 is reached when $m_3 = 1$, i.e., $\mathbf{m}^* = [0 \quad 0 \quad 1]^T$ and $\mathbf{n}_i^* = \mathbf{U}_i \mathbf{m}^* = \mathbf{u}_3$.

Therefore, the optimal cost is $\lambda_3(\mathbf{A}_i)$ and the optimum solution is $\mathbf{n}^* = \mathbf{u}_3(\mathbf{A}_i)$ and $\mathbf{q}^* = \bar{\mathbf{p}}_i$.

2) *Edge feature*: $\mathbf{h}_i = \mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T$, then $\mathbf{h}_i \mathbf{h}_i^T = \mathbf{h}_i$ and

$$\begin{aligned}
c_i &= \min_{\mathbf{n}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} \|(\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T)(\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)\|_2^2 \right) \\
&= \min_{\mathbf{n}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)^T (\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^T) (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i) \right) \\
&= \min_{\mathbf{n}_i} \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)^T (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i) \right. \\
&\quad \left. - \mathbf{n}_i^T \left(\frac{1}{N_i} \sum_{j=1}^{M_p} \sum_{k=1}^{N_{ij}} (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i)^T (\mathbf{p}_{ijk} - \bar{\mathbf{p}}_i) \right) \mathbf{n}_i \right) \\
&= \min_{\mathbf{n}_i} (\text{trace}(\mathbf{A}_i) - \mathbf{n}_i^T \mathbf{A}_i \mathbf{n}_i) \\
&= \lambda_1 + \lambda_2 + \lambda_3 - \underbrace{\max_{\mathbf{n}_i} \mathbf{n}_i^T \mathbf{A}_i \mathbf{n}_i}_{=\lambda_1, \text{ when } \mathbf{n}_i^* = \mathbf{u}_1} \\
&= \lambda_2 + \lambda_3.
\end{aligned} \tag{128}$$

Therefore, the optimal cost is $\lambda_2(\mathbf{A}_i) + \lambda_3(\mathbf{A}_i)$ and the optimum solution is $\mathbf{n}^* = \mathbf{u}_1(\mathbf{A}_i)$ and $\mathbf{q}^* = \bar{\mathbf{p}}_i$. $\square$

B. Proof of Theorem 1

For the point collections $\mathcal{C} = \{\mathbf{p}_k \in \mathbb{R}^3 | k = 1, \dots, n\}$, its point cluster is

$$\mathfrak{R}(\mathcal{C}) = \sum_{k=1}^n \begin{bmatrix} \mathbf{p}_k \\ 1 \end{bmatrix} [\mathbf{p}_k^T \ 1] \tag{129}$$

The rigid transformation of $\mathcal{C}$ by pose $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}_{3 \times 1} & 1 \end{bmatrix}$ is

$$\mathbf{T} \circ \mathcal{C} = \{\mathbf{R}\mathbf{p}_k + \mathbf{t} \in \mathbb{R}^3 | k = 1, \dots, n\} \tag{130}$$

whose point cluster is

$$\begin{aligned}
\mathfrak{R}(\mathbf{T} \circ \mathcal{C}) &= \sum_{k=1}^n \begin{bmatrix} \mathbf{R}\mathbf{p}_k + \mathbf{t} \\ 1 \end{bmatrix} [(\mathbf{R}\mathbf{p}_k + \mathbf{t})^T \ 1] \\
&= \sum_{k=1}^n \mathbf{T} \begin{bmatrix} \mathbf{p}_k \\ 1 \end{bmatrix} [\mathbf{p}_k^T \ 1] \mathbf{T}^T = \mathbf{T} \mathfrak{R}(\mathcal{C}) \mathbf{T}^T
\end{aligned} \tag{131}$$

which yields the solution. $\square$

C. Proof of Theorem 2

For two point collections $\mathcal{C}_1 = \{\mathbf{p}_k^1 \in \mathbb{R}^3 | k = 1, \dots, n_1\}$ and $\mathcal{C}_2 = \{\mathbf{p}_k^2 \in \mathbb{R}^3 | k = 1, \dots, n_2\}$ in the same reference frame, their point clusters are respectively

$$\mathfrak{R}(\mathcal{C}_l) = \sum_{k=1}^{n_l} \begin{bmatrix} \mathbf{p}_k^l \\ 1 \end{bmatrix} [(\mathbf{p}_k^l)^T \ 1], \quad l = 1, 2 \tag{132}$$

The merge of $\mathcal{C}_1$ and $\mathcal{C}_2$ is

$$\mathcal{C}_1 \oplus \mathcal{C}_2 = \{\mathbf{p}_k^l \in \mathbb{R}^3 | l = 1, 2; k = 1, \dots, n_l\} \tag{133}$$

whose point cluster is

$$\begin{aligned}
\mathfrak{R}(\mathcal{C}_1 \oplus \mathcal{C}_2) &= \sum_{k=1}^{n_1} \begin{bmatrix} \mathbf{p}_k^1 \\ 1 \end{bmatrix} [(\mathbf{p}_k^1)^T \ 1] + \sum_{k=1}^{n_2} \begin{bmatrix} \mathbf{p}_k^2 \\ 1 \end{bmatrix} [(\mathbf{p}_k^2)^T \ 1] \\
&= \mathfrak{R}(\mathcal{C}_1) + \mathfrak{R}(\mathcal{C}_2)
\end{aligned} \tag{134}$$

which yields the solution. $\square$

D. Proof of Theorem 3

Let $\mathbf{T}_0 = \begin{bmatrix} \mathbf{R}_0 & \mathbf{t}_0 \\ 0 & 1 \end{bmatrix}$ and $\bar{\mathbf{C}} = \mathbf{T}_0 \mathbf{C} \mathbf{T}_0^T = \begin{bmatrix} \bar{\mathbf{P}} & \bar{\mathbf{v}} \\ \bar{\mathbf{v}}^T & N \end{bmatrix}$, then

$$\bar{\mathbf{P}} = \mathbf{R}_0 \mathbf{P}_0 \mathbf{R}_0^T + \mathbf{R}_0 \mathbf{v}_0 \mathbf{t}_0^T + \mathbf{t}_0 \mathbf{v}_0^T \mathbf{R}_0^T + N \mathbf{t}_0 \mathbf{t}_0^T, \tag{135}$$

$$\bar{\mathbf{v}} = \mathbf{R}_0 \mathbf{v}_0 + N \mathbf{t}_0, \tag{136}$$

$$\mathbf{A}(\mathbf{T}_0 \mathbf{C} \mathbf{T}_0^T) = \frac{1}{N} \bar{\mathbf{P}} - \frac{1}{N^2} \bar{\mathbf{v}} \bar{\mathbf{v}}^T = \mathbf{R}_0 \mathbf{A}(\mathbf{C}) \mathbf{R}_0^T. \tag{137}$$

Since $\mathbf{A}(\mathbf{T}_0 \mathbf{C} \mathbf{T}_0^T)$ and $\mathbf{A}(\mathbf{C})$ are similar by transformation $\mathbf{R}_0$, they have the same eigenvalue. $\square$

E. Proof of Theorem 4

Denote λ_l the l -th largest eigenvalue of $\mathbf{A}$ and $\mathbf{u}_l$ the corresponding vector, i.e., $\lambda_l \mathbf{u}_l = \mathbf{A} \mathbf{u}_l$. Since $\mathbf{A}$ is symmetric, $\mathbf{u}_l$ is an orthonormal vector. Multiplying both sides of $\lambda_l \mathbf{u}_l = \mathbf{A} \mathbf{u}_l$ by $\mathbf{u}_l^T$ leads to

$$\lambda_l = \mathbf{u}_l^T \mathbf{A} \mathbf{u}_l. \tag{138}$$

Note that in the above equation, λ_l , $\mathbf{u}_l$ and $\mathbf{A}_l$ all depend on the pose $\mathbf{T}$. To avoid any confusion, we write them as explicit functions of $\mathbf{T}$:

$$\lambda_l(\mathbf{T}) = \mathbf{u}_l^T(\mathbf{T}) \mathbf{A}(\mathbf{T}) \mathbf{u}_l(\mathbf{T}). \tag{139}$$

Parameterizing the pose $\mathbf{T}$ by $\delta \mathbf{T}$ leads to

$$\lambda_l(\delta \mathbf{T}) = \mathbf{u}_l^T(\delta \mathbf{T}) \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l(\delta \mathbf{T}). \tag{140}$$

From the first conclusion of Lemma 1 (i.e., (58)), we know that for a vector $\mathbf{x} = [x_1 \ \dots \ x_m] \in \mathbb{R}^m$ that the matrix $\mathbf{A}$ depends on, we have

$$\begin{aligned}
\frac{\partial \lambda_l(\mathbf{x})}{\partial x_i} &= \frac{\partial (\mathbf{u}_l^T(\mathbf{x}) \mathbf{A}(\mathbf{x}) \mathbf{u}_l(\mathbf{x}))}{\partial x_i} = \mathbf{u}_l^T(\mathbf{x}) \frac{\partial \mathbf{A}(\mathbf{x})}{\partial x_i} \mathbf{u}_l(\mathbf{x}), \\
&\quad \forall i = 1, \dots, m, x_i \in \mathbb{R}
\end{aligned} \tag{141}$$

Directly applying this result to the entire vector $\mathbf{x}$ leads to the notation of $\frac{\partial \mathbf{A}(\mathbf{x})}{\partial \mathbf{x}}$, which is a tensor. To avoid it, we fix the vector $\mathbf{u}_l(\mathbf{x})$ at its current value and lump it with the matrix $\mathbf{A}(\mathbf{x})$ within the derivative, i.e.,

$$\mathbf{u}_l^T(\mathbf{x}) \frac{\partial \mathbf{A}(\mathbf{x})}{\partial x_i} \mathbf{u}_l(\mathbf{x}) := \frac{\partial \mathbf{u}_l^T \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial x_i} \tag{142}$$

where on the right hand side, $\mathbf{u}_l$ is fixed (so we remove its argument $\mathbf{x}$) and the derivative is only applied on the component $\mathbf{A}(\mathbf{x})$ (so we keep its argument $\mathbf{x}$). If applying (142) to the entire vector $\mathbf{x} \in \mathbb{R}^m$, the result would be $\frac{\partial \mathbf{u}_l^T \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial \mathbf{x}}$, which is now a row vector of dimension m .

Since the input parameter is the poses parameterized by $\delta \mathbf{T}$, setting $\mathbf{x}$ to $\delta \mathbf{T}$ in (141) and applying the notation trick in (142) lead to

$$\frac{\partial \lambda_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} = \frac{\partial \mathbf{u}_l^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}}. \quad (143)$$

Recalling Lemma 2 with $k = l$, we obtain:

$$\mathbf{u}_l^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l = \frac{1}{2} \delta \mathbf{T}^T \cdot \mathbf{Q}_{ll} \cdot \delta \mathbf{T} + \mathbf{g}_{ll} \cdot \delta \mathbf{T} + r_{ll}. \quad (144)$$

where $\mathbf{g}_{ll}$ and $\mathbf{Q}_{ll}$ are defined in (69) and (70) with $k = l$, respectively.

Therefore, the first order derivative of $\lambda_l(\mathbf{T})$ w.r.t. $\mathbf{T}$ is

$$\mathbf{J}_l \triangleq \frac{\partial \lambda_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} = \frac{\partial \mathbf{u}_l^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} = \mathbf{g}_{ll}, \quad (145)$$

which is the result of (30) in Theorem 4.

Next, we derive the second order derivative of $\lambda_l(\mathbf{T})$ w.r.t. $\mathbf{T}$. From (141), we have,

$$\frac{\partial \lambda_l(\mathbf{x})}{\partial x_i} = \mathbf{u}_l^T(\mathbf{x}) \frac{\partial \mathbf{A}(\mathbf{x})}{\partial x_i} \mathbf{u}_l(\mathbf{x}), \forall x_i \in \mathbb{R}, \quad (146)$$

Differentiating it w.r.t. the second parameter $x_j \in \mathbb{R}$ leads to

$$\begin{aligned} \frac{\partial^2 \lambda_l(\mathbf{x})}{\partial x_j \partial x_i} &= \frac{\partial}{\partial x_j} \left(\mathbf{u}_l^T(\mathbf{x}) \frac{\partial \mathbf{A}(\mathbf{x})}{\partial x_i} \mathbf{u}_l(\mathbf{x}) \right) \\ &= \left(\frac{\partial \mathbf{u}_l(\mathbf{x})}{\partial x_j} \right)^T \left(\frac{\partial \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial x_i} \right) + \frac{\partial}{\partial x_j} \left(\frac{\partial \mathbf{u}_l^T \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial x_i} \right) \\ &\quad + \left(\frac{\partial \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial x_i} \right)^T \left(\frac{\partial \mathbf{u}_l(\mathbf{x})}{\partial x_j} \right) \end{aligned} \quad (147)$$

Applying the above results to each elements x_i, x_j leads to

$$\begin{aligned} \frac{\partial^2 \lambda_l(\mathbf{x})}{\partial \mathbf{x}^2} &= \left(\frac{\partial \mathbf{u}_l(\mathbf{x})}{\partial \mathbf{x}} \right)^T \left(\frac{\partial \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial \mathbf{x}} \right) \\ &\quad + \frac{\partial}{\partial \mathbf{x}} \left(\frac{\partial \mathbf{u}_l^T \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial \mathbf{x}} \right) + \left(\frac{\partial \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial \mathbf{x}} \right)^T \left(\frac{\partial \mathbf{u}_l(\mathbf{x})}{\partial \mathbf{x}} \right). \end{aligned} \quad (148)$$

To compute $\frac{\partial \mathbf{u}_l(\mathbf{x})}{\partial \mathbf{x}}$, we apply the second conclusion in Lemma 1 (i.e., (59)) to all components of $\mathbf{x}$ and use the notation trick similar to (142):

$$\frac{\partial \mathbf{u}_l(\mathbf{x})}{\partial \mathbf{x}} = \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k \mathbf{u}_k^T \frac{\partial \mathbf{A}(\mathbf{x}) \mathbf{u}_l}{\partial \mathbf{x}} \quad (149)$$

Now, the input parameter is the pose vector parameterized by $\delta \mathbf{T}$, substituting $\mathbf{x} = \delta \mathbf{T}$ into (148) leads to

$$\begin{aligned} \frac{\partial^2 \lambda_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}^2} &= \underbrace{\left(\frac{\partial \mathbf{u}_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} \right)^T \left(\frac{\partial \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} \right)}_{\mathbf{D}_l^T} \\ &\quad + \underbrace{\frac{\partial^2 \mathbf{u}_l^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}^2}}_{\mathbf{Q}_{ll}} + \underbrace{\left(\frac{\partial \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} \right)^T \left(\frac{\partial \mathbf{u}_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} \right)}_{\mathbf{D}_l}, \end{aligned} \quad (150)$$

where term $\mathbf{Q}_{ll}$ is from (144), which is further from (70) with $k = l$. To obtain the term $\mathbf{D}_l$, we substitute $\mathbf{x} = \delta \mathbf{T}$ into (149):

$$\frac{\partial \mathbf{u}_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} = \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k \mathbf{u}_k^T \frac{\partial \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} \quad (151)$$

$$= \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k \frac{\partial \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}}. \quad (152)$$

From Lemma 2, we have

$$\frac{\partial \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} = \mathbf{g}_{kl} \quad (153)$$

with $\mathbf{g}_{kl}$ defined in (69). Hence,

$$\frac{\partial \mathbf{u}_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} = \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k \mathbf{g}_{kl} \quad (154)$$

and

$$\mathbf{D}_l = \left(\frac{\partial \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} \right)^T \left(\frac{\partial \mathbf{u}_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}} \right) \quad (155)$$

$$= \left(\frac{\partial \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} \right)^T \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{u}_k \mathbf{g}_{kl} \quad (156)$$

$$= \sum_{k=1, k \neq l}^3 \left(\frac{\partial \mathbf{u}_k^T \mathbf{A}(\delta \mathbf{T}) \mathbf{u}_l}{\partial \delta \mathbf{T}} \right)^T \frac{1}{\lambda_l - \lambda_k} \mathbf{g}_{kl} \quad (157)$$

$$= \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_l - \lambda_k} \mathbf{g}_{kl}^T \mathbf{g}_{kl}. \quad (158)$$

Therefore, the second order derivative of $\lambda_l(\mathbf{T})$ w.r.t. $\mathbf{T}$ is

$$\begin{aligned} \mathbf{H}_l &\triangleq \frac{\partial^2 \lambda_l(\delta \mathbf{T})}{\partial \delta \mathbf{T}^2} = \mathbf{D}_l^T + \mathbf{Q}_{ll} + \mathbf{D}_l \\ &= \mathbf{W}_l + \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \mathbf{g}_{kl}^T \mathbf{g}_{kl}. \end{aligned} \quad (159)$$

where $\mathbf{W}_l = \mathbf{Q}_{ll}$ defined in (70) with $k = l$. (159) gives the result of (31) in Theorem 4. $\square$

F. Proof of Corollary 4.1

First, we show that

$$\mathbf{J} \delta \mathbf{T} = 0, \delta \mathbf{T}^T \mathbf{H} \delta \mathbf{T} = 0, \delta \mathbf{T} = \begin{bmatrix} \mathbf{w} \\ \vdots \\ \mathbf{w} \end{bmatrix}, \forall \mathbf{w} \in \mathbb{R}^6 \quad (160)$$

From (30) in Theorem 4, we can obtain

$$\mathbf{J}_l \delta \mathbf{T} = \mathbf{g}_{ll} \delta \mathbf{T} = \sum_{j=1}^{M_p} \mathbf{g}_{ll}^j \mathbf{w} \quad (161)$$

where $\mathbf{g}_{ll}^j$ is defined in (71) with $k = l$:

$$\mathbf{g}_{ll}^j = \frac{2}{N} \mathbf{u}_l^T \mathbf{S}_p(\mathbf{T}_j - \frac{1}{N} \mathbf{C} \mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \quad (162)$$

$$\mathbf{V}_l = \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_l \end{bmatrix} \quad \mathbf{F} = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (163)$$

$$\mathbf{S}_p = [\mathbf{I}_{3 \times 3} \quad \mathbf{0}_{3 \times 1}] \quad \mathbf{C} = \sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \quad (164)$$

Thus,

$$\begin{aligned} \sum_{j=1}^{M_p} \mathbf{g}_{il}^j \mathbf{w} &= \frac{2}{N} \sum_{j=1}^{M_p} \mathbf{u}_l^T \mathbf{S}_p (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \mathbf{w} \\ &= \frac{2}{N} \mathbf{u}_l^T \mathbf{S}_p \left(\sum_{j=1}^{M_p} (\mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T - \frac{1}{N} \mathbf{C}\mathbf{F} \mathbf{C}_j \mathbf{T}_j^T) \right) \mathbf{V}_l^T \mathbf{w}. \end{aligned}$$

Since $\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \triangleq \mathbf{C}$ and $\sum_{j=1}^{M_p} \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T = \mathbf{F}\mathbf{C}$ from equation (108), we have

$$\sum_{j=1}^{M_p} \mathbf{g}_{il}^j \mathbf{w} = \frac{2}{N} \mathbf{u}_l^T \mathbf{S}_p (\mathbf{C} - \frac{1}{N} \mathbf{C}\mathbf{F}\mathbf{C}) \mathbf{V}_l^T \mathbf{w}.$$

Partition $\mathbf{C} = \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix}$ and recalling $\mathbf{A} = \frac{1}{N} \mathbf{P} - \frac{1}{N^2} \mathbf{v}\mathbf{v}^T$,

$$\begin{aligned} \mathbf{C} - \frac{1}{N} \mathbf{C}\mathbf{F}\mathbf{C} &= \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} - \frac{1}{N} \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{P} & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} - \begin{bmatrix} \frac{1}{N} \mathbf{v}\mathbf{v}^T & \mathbf{v} \\ \mathbf{v}^T & N \end{bmatrix} = \begin{bmatrix} N\mathbf{A} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 0 \end{bmatrix} \quad (165) \end{aligned}$$

Thus,

$$\sum_{j=1}^{M_p} \mathbf{g}_{il}^j \mathbf{w} = \frac{2}{N} \mathbf{u}_l^T \mathbf{S}_p \begin{bmatrix} N\mathbf{A} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 0 \end{bmatrix} \mathbf{V}_l^T \mathbf{w} \quad (166)$$

$$= 2 [\mathbf{u}_l^T \mathbf{A} \quad \mathbf{0}_{3 \times 1}] \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_l \end{bmatrix}^T \mathbf{w} \quad (167)$$

$$= 2 [\mathbf{u}_l^T \mathbf{A} [\mathbf{u}_l] \quad \mathbf{0}_{3 \times 1}] \mathbf{w} \quad (168)$$

Since $\mathbf{A}\mathbf{u}_l = \lambda_l \mathbf{u}_l$,

$$\mathbf{u}_l^T \mathbf{A} [\mathbf{u}_l] = \lambda_l \mathbf{u}_l^T [\mathbf{u}_l] = \mathbf{0}_{1 \times 3}. \quad (169)$$

Therefore,

$$\mathbf{J}_l \delta \mathbf{T} = \sum_{j=1}^{M_p} \mathbf{g}_{il}^j \mathbf{w} = 0. \quad (170)$$

For the proof of $\delta \mathbf{T}^T \mathbf{H}_l \delta \mathbf{T} = 0$, from (31) in Theorem 4,

$$\begin{aligned} \delta \mathbf{T}^T \mathbf{H}_l \delta \mathbf{T} &= \delta \mathbf{T}^T (\mathbf{W}_l + \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \mathbf{g}_{kl}^T \mathbf{g}_{kl}) \delta \mathbf{T} \\ &= \mathbf{w}^T \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \left(\mathbf{W}_l^{ij} + \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} (\mathbf{g}_{kl}^i)^T \mathbf{g}_{kl}^j \right) \mathbf{w} \quad (171) \end{aligned}$$

where

$$\begin{aligned} \mathbf{W}_l^{ij} &= -\frac{2}{N^2} \mathbf{V}_l \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \mathbb{1}_{i=j} \\ &\quad \cdot \left(\frac{2}{N} \mathbf{V}_l \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \begin{bmatrix} \mathbf{K}_l^j & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \right), \quad (172) \end{aligned}$$

$$\begin{aligned} \mathbf{K}_l^j &= \frac{1}{N} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l] [\mathbf{u}_l] \\ &\quad + \frac{1}{N} [\mathbf{u}_l] [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l], \quad (173) \end{aligned}$$

$$\begin{aligned} \mathbf{g}_{kl}^j &= \frac{1}{N} \mathbf{u}_l^T \mathbf{S}_p (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_k^T \\ &\quad + \frac{1}{N} \mathbf{u}_k^T \mathbf{S}_p (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F}) \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T, \quad (174) \end{aligned}$$

We will divide (171) into two parts to discuss. For the first part,

$$\begin{aligned} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{W}_l^{ij} &= -\frac{2}{N^2} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{V}_l \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T \\ &\quad + \frac{2}{N} \sum_{j=1}^{M_p} \mathbf{V}_l \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \mathbf{V}_l^T + \begin{bmatrix} \sum_{j=1}^{M_p} \mathbf{K}_l^j & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \quad (175) \end{aligned}$$

Since $\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \triangleq \mathbf{C}$, $\sum_{j=1}^{M_p} \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T = \mathbf{F}\mathbf{C}$ and $\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{F} = \mathbf{C}\mathbf{F}$ from (108), we have

$$\begin{aligned} \sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T &= \sum_{i=1}^{M_p} \mathbf{T}_i \mathbf{C}_i \sum_{j=1}^{M_p} \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T = \sum_{i=1}^{M_p} \mathbf{T}_i \mathbf{C}_i \mathbf{F} \mathbf{C} = \mathbf{C}\mathbf{F}\mathbf{C}, \quad (176) \end{aligned}$$

and

$$\begin{aligned} \sum_{j=1}^{M_p} \mathbf{K}_l^j &= \frac{1}{N} \sum_{j=1}^{M_p} [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l] [\mathbf{u}_l] \\ &\quad + \frac{1}{N} \sum_{j=1}^{M_p} [\mathbf{u}_l] [\mathbf{S}_p \mathbf{T}_j \mathbf{C}_j (\mathbf{T}_j - \frac{1}{N} \mathbf{C}\mathbf{F})^T \mathbf{S}_p^T \mathbf{u}_l] \\ &= \frac{1}{N} [\mathbf{S}_p (\mathbf{C} - \frac{1}{N} \mathbf{C}\mathbf{F}\mathbf{C}) \mathbf{S}_p^T \mathbf{u}_l] [\mathbf{u}_l] \\ &\quad + \frac{1}{N} [\mathbf{u}_l] [\mathbf{S}_p (\mathbf{C} - \frac{1}{N} \mathbf{C}\mathbf{F}\mathbf{C}) \mathbf{S}_p^T \mathbf{u}_l]. \quad (177) \end{aligned}$$

Then, from (165) and $\mathbf{A}\mathbf{u}_l = \lambda_l \mathbf{u}_l$,

$$\sum_{j=1}^{M_p} \mathbf{K}_l^j = [\mathbf{A}\mathbf{u}_l] [\mathbf{u}_l] + [\mathbf{u}_l] [\mathbf{A}\mathbf{u}_l] = 2\lambda_l [\mathbf{u}_l]^2. \quad (178)$$

Now, substituting the results in (176) and (178) into (175):

$$\sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \mathbf{w}_l^{ij} = -\frac{2}{N^2} \mathbf{V}_l \mathbf{C} \mathbf{F} \mathbf{C} \mathbf{V}_l^T + \frac{2}{N} \mathbf{V}_l \mathbf{C} \mathbf{V}_l^T + \begin{bmatrix} 2\lambda_l [\mathbf{u}_l]^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (179)$$

$$= \frac{2}{N} \mathbf{V}_l \left(\mathbf{C} - \frac{1}{N} \mathbf{C} \mathbf{F} \mathbf{C} \right) \mathbf{V}_l^T + \begin{bmatrix} 2\lambda_l [\mathbf{u}_l]^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (180)$$

$$\begin{aligned} &= 2\mathbf{V}_l \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{V}_l^T + \begin{bmatrix} 2\lambda_l [\mathbf{u}_l]^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \\ &= 2 \begin{bmatrix} \lambda_l [\mathbf{u}_l]^2 - [\mathbf{u}_l] \mathbf{A} [\mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}. \end{aligned} \quad (181)$$

For the second part in (171),

$$\begin{aligned} &\sum_{i=1}^{M_p} \sum_{j=1}^{M_p} \left(\sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} (\mathbf{g}_{kl}^i)^T \mathbf{g}_{kl}^j \right) \\ &= \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \left(\sum_{i=1}^{M_p} \mathbf{g}_{kl}^i \right)^T \left(\sum_{j=1}^{M_p} \mathbf{g}_{kl}^j \right) \end{aligned} \quad (182)$$

where

$$\begin{aligned} \sum_{j=1}^{M_p} \mathbf{g}_{kl}^j &= \frac{1}{N} \mathbf{u}_l^T \mathbf{S}_p \sum_{j=1}^{M_p} (\mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T - \frac{1}{N} \mathbf{C} \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T) \mathbf{V}_k^T \\ &\quad + \frac{1}{N} \mathbf{u}_k^T \mathbf{S}_p \sum_{j=1}^{M_p} (\mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T - \frac{1}{N} \mathbf{C} \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T) \mathbf{V}_l^T. \end{aligned} \quad (183)$$

Again, since $\sum_{j=1}^{M_p} \mathbf{T}_j \mathbf{C}_j \mathbf{T}_j^T \triangleq \mathbf{C}$ and $\sum_{j=1}^{M_p} \mathbf{F} \mathbf{C}_j \mathbf{T}_j^T = \mathbf{F} \mathbf{C}$ from (108), we have

$$\begin{aligned} \sum_{j=1}^{M_p} \mathbf{g}_{kl}^j &= \frac{1}{N} \mathbf{u}_l^T \mathbf{S}_p \left(\mathbf{C} - \frac{1}{N} \mathbf{C} \mathbf{F} \mathbf{C} \right) \mathbf{V}_k^T \\ &\quad + \frac{1}{N} \mathbf{u}_k^T \mathbf{S}_p \left(\mathbf{C} - \frac{1}{N} \mathbf{C} \mathbf{F} \mathbf{C} \right) \mathbf{V}_l^T \end{aligned} \quad (184)$$

Further, from (165), $\mathbf{A} \mathbf{u}_l = \lambda_l \mathbf{u}_l$, and $\mathbf{A} \mathbf{u}_k = \lambda_k \mathbf{u}_k$, we have

$$\begin{aligned} \sum_{j=1}^{M_p} \mathbf{g}_{kl}^j &= \mathbf{u}_l^T [\mathbf{A} \quad \mathbf{0}_{3 \times 1}] \mathbf{V}_k^T + \mathbf{u}_k^T [\mathbf{A} \quad \mathbf{0}_{3 \times 1}] \mathbf{V}_l^T \\ &= [\mathbf{u}_l^T \mathbf{A} [\mathbf{u}_k] \quad \mathbf{0}_{3 \times 1}] + [\mathbf{u}_k^T \mathbf{A} [\mathbf{u}_l] \quad \mathbf{0}_{3 \times 1}] \\ &= [\lambda_l \mathbf{u}_l^T [\mathbf{u}_k] + \lambda_k \mathbf{u}_k^T [\mathbf{u}_l] \quad \mathbf{0}_{3 \times 1}] \end{aligned} \quad (185)$$

Therefore,

$$\begin{aligned} &\sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \left(\sum_{i=1}^{M_p} \mathbf{g}_{kl}^i \right)^T \left(\sum_{j=1}^{M_p} \mathbf{g}_{kl}^j \right) \\ &= \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \begin{bmatrix} \mathbf{r}_{kl} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 0 \end{bmatrix} \end{aligned} \quad (186)$$

where due to $[\mathbf{u}_k] \mathbf{u}_l = -[\mathbf{u}_l] \mathbf{u}_k$ and $\mathbf{u}_k^T [\mathbf{u}_l] = -\mathbf{u}_l^T [\mathbf{u}_k]$

$$\begin{aligned} \mathbf{r}_{kl} &= (\lambda_l \mathbf{u}_l^T [\mathbf{u}_k] + \lambda_k \mathbf{u}_k^T [\mathbf{u}_l])^T (\lambda_l \mathbf{u}_l^T [\mathbf{u}_k] + \lambda_k \mathbf{u}_k^T [\mathbf{u}_l]) \\ &= -\lambda_l^2 [\mathbf{u}_k] \mathbf{u}_l \mathbf{u}_l^T [\mathbf{u}_k] - \lambda_l \lambda_k [\mathbf{u}_k] \mathbf{u}_l \mathbf{u}_k^T [\mathbf{u}_l] \\ &\quad - \lambda_k^2 [\mathbf{u}_l] \mathbf{u}_k \mathbf{u}_k^T [\mathbf{u}_l] - \lambda_l \lambda_k [\mathbf{u}_l] \mathbf{u}_k \mathbf{u}_l^T [\mathbf{u}_k] \\ &= -(\lambda_l - \lambda_k)^2 [\mathbf{u}_l] \mathbf{u}_k \mathbf{u}_k^T [\mathbf{u}_l]. \end{aligned} \quad (187)$$

Thus,

$$\begin{aligned} &\sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} \left(\sum_{i=1}^{M_p} \mathbf{g}_{kl}^i \right)^T \left(\sum_{j=1}^{M_p} \mathbf{g}_{kl}^j \right) \\ &= 2 \sum_{k=1, k \neq l}^3 \begin{bmatrix} (\lambda_k - \lambda_l) [\mathbf{u}_l] \mathbf{u}_k \mathbf{u}_k^T [\mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & 0 \end{bmatrix} \end{aligned} \quad (188)$$

Now, from (181) and (188), the equation (171) is turned into

$$\delta \mathbf{T}^T \mathbf{H}_l \delta \mathbf{T} = \mathbf{w}^T \begin{bmatrix} 2\mathbf{L}_l & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 0 \end{bmatrix} \mathbf{w}, \quad (189)$$

where

$$\begin{aligned} \mathbf{L}_l &= \lambda_l [\mathbf{u}_l]^2 - [\mathbf{u}_l] \mathbf{A} [\mathbf{u}_l] + \sum_{k=1, k \neq l}^3 (\lambda_k - \lambda_l) [\mathbf{u}_l] \mathbf{u}_k \mathbf{u}_k^T [\mathbf{u}_l] \\ &= \lambda_l [\mathbf{u}_l]^2 - [\mathbf{u}_l] \mathbf{A} [\mathbf{u}_l] \\ &\quad + [\mathbf{u}_l] \left(\sum_{k=1, k \neq l}^3 \lambda_k \mathbf{u}_k \mathbf{u}_k^T \right) [\mathbf{u}_l] - \lambda_l [\mathbf{u}_l] \left(\sum_{k=1, k \neq l}^3 \mathbf{u}_k \mathbf{u}_k^T \right) [\mathbf{u}_l]. \end{aligned} \quad (190)$$

Since $\mathbf{u}_k$ ($k = 1, 2, 3$) is the eigenvector (with eigenvalue λ_k) of matrix $\mathbf{A}$, which is symmetric, we have the following two conditions from the singular value decomposition of $\mathbf{A}$:

$$\mathbf{A} = \sum_{k=1}^3 \lambda_k \mathbf{u}_k \mathbf{u}_k^T, \quad \mathbf{I} = \sum_{k=1}^3 \mathbf{u}_k \mathbf{u}_k^T, \quad (191)$$

which imply

$$\sum_{k=1, k \neq l}^3 \lambda_k \mathbf{u}_k \mathbf{u}_k^T = \mathbf{A} - \lambda_l \mathbf{u}_l \mathbf{u}_l^T, \quad \sum_{k=1, k \neq l}^3 \mathbf{u}_k \mathbf{u}_k^T = \mathbf{I} - \mathbf{u}_l \mathbf{u}_l^T. \quad (192)$$

Substituting the above results into $\mathbf{L}_l$ in (190):

$$\begin{aligned} \mathbf{L}_l &= \lambda_l [\mathbf{u}_l]^2 - [\mathbf{u}_l] \mathbf{A} [\mathbf{u}_l] \\ &\quad + [\mathbf{u}_l] (\mathbf{A} - \lambda_l \mathbf{u}_l \mathbf{u}_l^T) [\mathbf{u}_l] - \lambda_l [\mathbf{u}_l] (\mathbf{I} - \mathbf{u}_l \mathbf{u}_l^T) [\mathbf{u}_l] \\ &= \lambda_l [\mathbf{u}_l]^2 - [\mathbf{u}_l] \mathbf{A} [\mathbf{u}_l] + [\mathbf{u}_l] \mathbf{A} [\mathbf{u}_l] - \lambda_l [\mathbf{u}_l]^2 = \mathbf{0}. \end{aligned} \quad (193)$$

As a result,

$$\delta \mathbf{T}^T \mathbf{H} \delta \mathbf{T} = \mathbf{w}^T \begin{bmatrix} 2\mathbf{L}_l & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 0 \end{bmatrix} \mathbf{w} = 0. \quad (194)$$

Finally, if $\mathbf{C}_j = \mathbf{0}$, we have $\mathbf{g}_{kl}^j = \mathbf{0}_{1 \times 6}$ from (76) of Lemma 2, hence $\mathbf{J}^j = \mathbf{g}_{ll}^j = \mathbf{0}$, the result (40) of Corollary 4.1; If $\mathbf{C}_i = \mathbf{0}$ or $\mathbf{C}_j = \mathbf{0}$, we have $\mathbf{Q}_{kl}^{ij} = \mathbf{0}_{6 \times 6}$ from (77) of Lemma 2, hence $\mathbf{W}_l^{ij} = \mathbf{Q}_{ll}^{ij} = \mathbf{0}_{6 \times 6}$ and $\mathbf{H}_l^{ij} = \mathbf{W}_l^{ij} + \sum_{k=1, k \neq l}^3 \frac{2}{\lambda_l - \lambda_k} (\mathbf{g}_{kl}^j)^T \mathbf{g}_{kl}^j = \mathbf{0}$, the result (41) of Corollary 4.1. $\square$

G. Derivation of pose covariance

The quantity $\delta \mathbf{C}_{f_{ij}}$ can be obtained by substituting (44) into the definition of $\mathbf{C}_{f_{ij}}^{\text{gt}}$ in (45) and retaining only the first order items:

$$\delta \mathbf{C}_{f_{ij}} = \begin{bmatrix} \delta \mathbf{P}_{f_{ij}} & \delta \mathbf{v}_{f_{ij}} \\ \delta \mathbf{v}_{f_{ij}}^T & 0 \end{bmatrix}, \quad \text{where} \quad (195)$$

$$\delta \mathbf{P}_{f_{ij}} = \sum_{k=1}^{N_{ij}} (\mathbf{p}_{f_{ijk}} \delta \mathbf{p}_{f_{ijk}}^T + \delta \mathbf{p}_{f_{ijk}} \mathbf{p}_{f_{ijk}}^T), \quad (196)$$

$$\delta \mathbf{v}_{f_{ij}} = \sum_{k=1}^{N_{ij}} \delta \mathbf{p}_{f_{ijk}}. \quad (197)$$

To derive $\frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f$ without involving any tensor, we parameterize the matrix $\mathbf{C}_{f_{ij}}$ by a column vector $\mathbf{c}_{f_{ij}}$, which consists of the independent elements in $\mathbf{C}_{f_{ij}}$:

$$\mathbf{c}_{f_{ij}} = \text{vec}(\mathbf{C}_{f_{ij}}) \triangleq \begin{bmatrix} \mathbf{e}_1^T \mathbf{C}_{f_{ij}} \mathbf{e}_1 & \mathbf{e}_1^T \mathbf{C}_{f_{ij}} \mathbf{e}_2 & \mathbf{e}_1^T \mathbf{C}_{f_{ij}} \mathbf{e}_3 \\ \mathbf{e}_2^T \mathbf{C}_{f_{ij}} \mathbf{e}_2 & \mathbf{e}_2^T \mathbf{C}_{f_{ij}} \mathbf{e}_3 & \mathbf{e}_3^T \mathbf{C}_{f_{ij}} \mathbf{e}_3 \\ \mathbf{e}_1^T \mathbf{C}_{f_{ij}} \mathbf{e}_4 & \mathbf{e}_2^T \mathbf{C}_{f_{ij}} \mathbf{e}_4 & \mathbf{e}_3^T \mathbf{C}_{f_{ij}} \mathbf{e}_4 \end{bmatrix}^T \in \mathbb{R}^9 \quad (198)$$

where $\text{vec}(\cdot) : \mathbb{S}^{4 \times 4} \mapsto \mathbb{R}^9$ maps a symmetric matrix to its column vector representation, $\mathbf{e}_l \in \mathbb{R}^4$ ($l \in \{1, 2, 3, 4\}$) is a vector with all zero elements except for the l -th element being one. Note that the constant N in the 4-th row, 4-th column of $\mathbf{C}_{f_{ij}}$ is not contained in $\mathbf{c}_{f_{ij}}$ since it is a constant number independent of the noise. Correspondingly, noises in $\mathbf{C}_{f_{ij}}$ becomes the noise of $\mathbf{c}_{f_{ij}}$ as below:

$$\begin{aligned} \delta \mathbf{c}_{f_{ij}} &= \begin{bmatrix} \mathbf{e}_1^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_1 & \mathbf{e}_1^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_2 & \mathbf{e}_1^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_3 \\ \mathbf{e}_2^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_2 & \mathbf{e}_2^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_3 & \mathbf{e}_3^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_3 \\ \mathbf{e}_1^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_4 & \mathbf{e}_2^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_4 & \mathbf{e}_3^T \delta \mathbf{C}_{f_{ij}} \mathbf{e}_4 \end{bmatrix}^T \\ &= \sum_{k=1}^{N_{ij}} \begin{bmatrix} 2\mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{11} \mathbf{S}_P^T \delta \mathbf{p}_{f_{ijk}} \\ \mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{12} \mathbf{S}_P^T \delta \mathbf{p}_{f_{ijk}} \\ \mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{13} \mathbf{S}_P^T \delta \mathbf{p}_{f_{ijk}} \\ 2\mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{22} \mathbf{S}_P^T \delta \mathbf{p}_{f_{ijk}} \\ \mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{23} \mathbf{S}_P^T \delta \mathbf{p}_{f_{ijk}} \\ 2\mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{33} \mathbf{S}_P^T \delta \mathbf{p}_{f_{ijk}} \\ \delta \mathbf{p}_{f_{ijk}} \end{bmatrix} = \sum_{k=1}^{N_{ij}} \mathbf{B}_{f_{ijk}} \delta \mathbf{p}_{f_{ijk}}, \quad (199) \end{aligned}$$

where $\mathbf{S}_P = [\mathbf{I}_{3 \times 3}, \mathbf{0}_{3 \times 1}]$, $\mathbf{E}_{kl} = \mathbf{e}_k \mathbf{e}_l^T + \mathbf{e}_l \mathbf{e}_k^T \in \mathbb{S}^{4 \times 4}$, $k, l \in \{1, 2, 3, 4\}$, and

$$\mathbf{B}_{f_{ijk}} = \begin{bmatrix} 2\mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{11} \mathbf{S}_P^T \\ \mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{12} \mathbf{S}_P^T \\ \mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{13} \mathbf{S}_P^T \\ 2\mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{22} \mathbf{S}_P^T \\ \mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{23} \mathbf{S}_P^T \\ 2\mathbf{p}_{f_{ijk}}^T \mathbf{S}_P \mathbf{E}_{33} \mathbf{S}_P^T \\ \mathbf{I}_{3 \times 3} \end{bmatrix} \in \mathbb{R}^{9 \times 3}. \quad (201)$$

With the column representation of each $\mathbf{C}_{f_{ij}}$ contained in $\mathbf{C}_f$, $\frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f$ can now be computed as

$$\frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f = \sum_{i=1}^{M_f} \sum_{j=1}^{M_p} \frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_{f_{ij}})}{\partial \mathbf{c}_{f_{ij}}} \delta \mathbf{c}_{f_{ij}}. \quad (202)$$

To derive the quantity $\frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_{f_{ij}})}{\partial \mathbf{c}_{f_{ij}}}$, we give two lemmas which are useful for subsequent derivations.

Lemma 3. For $\mathbf{w} \in \mathbb{R}^4$, $\mathbf{C} \in \mathbb{S}^{4 \times 4}$ and its vector form $\mathbf{c} = \text{vec}(\mathbf{C})$, we have

$$\frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{c}} = \mathbf{g}_1(\mathbf{w}) \in \mathbb{R}^{4 \times 9},$$

where

$$\mathbf{g}_1(\mathbf{w}) = \begin{bmatrix} \mathbf{E}_{11} \mathbf{w} & \mathbf{E}_{12} \mathbf{w} & \mathbf{E}_{13} \mathbf{w} & \mathbf{E}_{22} \mathbf{w} & \mathbf{E}_{23} \mathbf{w} \\ & \mathbf{E}_{33} \mathbf{w} & \mathbf{E}_{14} \mathbf{w} & \mathbf{E}_{24} \mathbf{w} & \mathbf{E}_{34} \mathbf{w} \end{bmatrix},$$

where $\mathbf{E}_{kl} \in \mathbb{S}^{4 \times 4}$, $k, l \in \{1, 2, 3, 4\}$, is

$$\mathbf{E}_{kl} = \begin{cases} \mathbf{e}_k \mathbf{e}_l^T + \mathbf{e}_l \mathbf{e}_k^T & (k \neq l) \\ \mathbf{e}_l \mathbf{e}_l^T & (k = l) \end{cases} \quad (203)$$

where $\mathbf{e}_l \in \mathbb{R}^4$ ($l \in \{1, 2, 3, 4\}$) is a vector with all zero elements except for the l -th element being one.

Proof. For the k -th row, l -th column element of $\mathbf{C}$, denoted by $\mathbf{C}_{k,l}$, we have

$$\frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{k,l}} = \mathbf{E}_{kl} \mathbf{w} \in \mathbb{R}^4.$$

Hence,

$$\begin{aligned} \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{c}} &= \begin{bmatrix} \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{1,1}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{1,2}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{1,3}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{2,2}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{2,3}} \\ & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{3,3}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{1,4}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{2,4}} & \frac{\partial \mathbf{C} \mathbf{w}}{\partial \mathbf{C}_{3,4}} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{E}_{11} \mathbf{w} & \mathbf{E}_{12} \mathbf{w} & \mathbf{E}_{13} \mathbf{w} & \mathbf{E}_{22} \mathbf{w} & \mathbf{E}_{23} \mathbf{w} \\ & \mathbf{E}_{33} \mathbf{w} & \mathbf{E}_{14} \mathbf{w} & \mathbf{E}_{24} \mathbf{w} & \mathbf{E}_{34} \mathbf{w} \end{bmatrix}. \end{aligned}$$

□

Lemma 4. For $\mathbf{w} \in \mathbb{R}^4$, $\mathbf{u}_l \in \mathbb{R}^3$ and

$$\mathbf{V}_l = \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_l \end{bmatrix} \in \mathbb{R}^{6 \times 4}$$

we have

$$\frac{\partial \mathbf{V}_l \mathbf{w}}{\partial \mathbf{u}_l} = \mathbf{g}_2(\mathbf{w}) = \begin{bmatrix} [\mathbf{w}_{1:3}] \\ w_4 \mathbf{I}_{3 \times 3} \end{bmatrix} \in \mathbb{R}^{6 \times 3}.$$

where $\mathbf{w}_{1:3}$ represents the first three elements in $\mathbf{w}$ and w_4 is the 4-th element of $\mathbf{w}$.

Proof.

$$\mathbf{V}_l \mathbf{w} = \begin{bmatrix} -[\mathbf{u}_l] & \mathbf{0} \\ \mathbf{0} & \mathbf{u}_l \end{bmatrix} \begin{bmatrix} \mathbf{w}_{1:3} \\ w_4 \end{bmatrix} = \begin{bmatrix} [\mathbf{w}_{1:3}] \mathbf{u}_l \\ w_4 \mathbf{u}_l \end{bmatrix} = \begin{bmatrix} [\mathbf{w}_{1:3}] \\ w_4 \mathbf{I}_{3 \times 3} \end{bmatrix} \mathbf{u}_l$$

Thus,

$$\frac{\partial \mathbf{V}_l \mathbf{w}}{\partial \mathbf{u}_l} = \begin{bmatrix} [\mathbf{w}_{1:3}] \\ w_4 \mathbf{I}_{3 \times 3} \end{bmatrix} = \mathbf{g}_2(\mathbf{w})$$

□

With these two lemmas, next we will continue to derive $\frac{\partial \mathbf{J}^T(\mathbf{T}^*, \mathbf{C}_{f_{ij}})}{\partial \mathbf{c}_{f_{ij}}}$ in (202). Theorem 4 gives the Jacobian for one cost item while the required Jacobian consists of items from all features, to distinguish the different cost item, we add a subscript ν to (30) to denote the ν -th item and replace $\mathbf{C}_j$

with the actual point cluster notation $\mathbf{C}_{f_{\nu j}}$ corresponding to the ν -th item, leading to:

$$\mathbf{J} = \sum_{\nu=1}^{M_f} \mathbf{J}_{\nu} \quad (204)$$

$$\mathbf{J}_{\nu} = [\dots \quad \mathbf{J}_{\nu}^p \quad \dots] \in \mathbb{R}^{1 \times 6M_p} \quad (205)$$

where $\mathbf{J}_{\nu}^p \in \mathbb{R}^{1 \times 6}$ is the p -th column block of $\mathbf{J}_{\nu}$ as shown in (32) of Theorem 4 (with $j = p$):

$$\mathbf{J}_{\nu}^p = \frac{2}{N_{\nu}} \mathbf{u}_{\nu l}^T \mathbf{S}_{\mathbf{p}} \left(\mathbf{T}_{\mathbf{p}} - \frac{1}{N_{\nu}} \mathbf{C}_{\nu} \mathbf{F} \right) \mathbf{C}_{f_{\nu p}} \mathbf{T}_{\mathbf{p}}^T \mathbf{V}_{\nu l}^T \in \mathbb{R}^{1 \times 6}.$$

where $\mathbf{u}_{\nu l}$ is the eigenvector associated to the l -th largest eigenvalue of the covariance matrix of the ν -th feature (or cost item), N_{ν} is the total number of points of the ν -th feature, $\mathbf{C}_{\nu}$ is the aggregation of all point clusters of the ν -th feature, and $\mathbf{C}_{f_{\nu p}}$ is the point cluster contributed by the p -th pose to the ν -th feature.

The total Jacobian is hence

$$\mathbf{J} = \sum_{\nu=1}^{M_f} \mathbf{J}_{\nu} = [\dots \quad \mathbf{J}^p \quad \dots] \in \mathbb{R}^{1 \times 6M_p}, \quad (206)$$

$$\begin{aligned} \mathbf{J}^p &= \sum_{\nu=1}^{M_f} \mathbf{J}_{\nu}^p \in \mathbb{R}^{1 \times 6}, \\ &= \sum_{\nu=1}^{M_f} \left(\frac{2}{N_{\nu}} \mathbf{u}_{\nu l}^T \mathbf{S}_{\mathbf{p}} \left(\mathbf{T}_{\mathbf{p}} - \frac{1}{N_{\nu}} \mathbf{C}_{\nu} \mathbf{F} \right) \mathbf{C}_{f_{\nu p}} \mathbf{T}_{\mathbf{p}}^T \mathbf{V}_{\nu l}^T \right). \end{aligned} \quad (207)$$

Next, we calculate the partial derivative $\frac{\partial(\mathbf{J}^p)^T}{\partial \mathbf{c}_{f_{ij}}}$. Note that in the summation of (207), only the i -th summation term (i.e., $\nu = i$) is related to $\mathbf{C}_{f_{ij}}$ (hence $\mathbf{c}_{f_{ij}}$), hence,

$$\frac{\partial(\mathbf{J}^p)^T}{\partial \mathbf{c}_{f_{ij}}} = \frac{\partial}{\partial \mathbf{c}_{f_{ij}}} \left(\frac{2}{N_i} \mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} \left(\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i \right) \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il} \right). \quad (208)$$

Since $\mathbf{C}_i = \sum_{\nu=1}^{M_p} \mathbf{T}_{\nu} \mathbf{C}_{f_{i\nu}} \mathbf{T}_{\nu}^T$, $\mathbf{u}_{il}$ is the eigenvector associated to the l -th largest eigenvalue of matrix $\mathbf{A}(\mathbf{C}_i)$, and $\mathbf{V}_{il} = \begin{bmatrix} -[\mathbf{u}_{il}] & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{u}_{il} \end{bmatrix}$, the derivative with respect to $\mathbf{c}_{f_{ij}}$ on the right hand side of (208) consists of four terms respectively from $\mathbf{V}_{il}$, $\mathbf{C}_{f_{ip}}$ (only when $p = j$), $\mathbf{C}_i$, and $\mathbf{u}_{il}$. Combining with Lemma 3 and Lemma 4, we have

$$\begin{aligned} \mathbf{L}_{ij}^p &\triangleq \frac{\partial(\mathbf{J}^p)^T}{\partial \mathbf{c}_{f_{ij}}} \in \mathbb{R}^{6 \times 9} \\ &= \frac{2}{N_i} \left(\mathbf{g}_2(\mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} (\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i) \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \right. \\ &\quad + \mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} (\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i) \mathbf{S}_{\mathbf{p}}^T \frac{\partial \mathbf{u}_{il}}{\partial \mathbf{c}_{f_{ij}}} \\ &\quad - \frac{2}{N_i^2} \mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} \mathbf{F} \mathbf{T}_j \mathbf{g}_1 (\mathbf{T}_j^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \\ &\quad \left. + \frac{2}{N_i} \mathbb{1}_{p=j} \cdot \left(\mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{g}_1 \left((\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i) \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il} \right) \right) \right) \end{aligned} \quad (210)$$

Only the term $\frac{\partial \mathbf{u}_{il}}{\partial \mathbf{c}_{f_{ij}}}$ is unknown in this formula. To compute it, we apply the second conclusion in Lemma 1 (i.e., (59)) to

all components of $\mathbf{c}_{f_{ij}}$ and use the notation trick similar to (142):

$$\frac{\partial \mathbf{u}_{il}}{\partial \mathbf{c}_{f_{ij}}} = \sum_{k=1, k \neq l}^3 \frac{1}{\lambda_{il} - \lambda_{ik}} \mathbf{u}_{ik} \frac{\partial (\mathbf{u}_{ik}^T \mathbf{A}(\mathbf{C}_i) \mathbf{u}_{il})}{\partial \mathbf{c}_{f_{ij}}} \quad (211)$$

From (87) of Lemma 2,

$$\begin{aligned} \mathbf{u}_{ik}^T \mathbf{A}(\mathbf{C}_i) \mathbf{u}_{il} &= \frac{1}{N_i} \mathbf{u}_{ik}^T \mathbf{S}_{\mathbf{p}} \\ &\left(\sum_{\mu=1}^{M_p} \mathbf{T}_{\mu} \mathbf{C}_{f_{i\mu}} \mathbf{T}_{\mu}^T - \frac{1}{N_i} \sum_{\mu=1}^{M_p} \sum_{\nu=1}^{M_p} \mathbf{T}_{\mu} \mathbf{C}_{f_{i\mu}} \mathbf{F} \mathbf{C}_{f_{i\nu}} \mathbf{T}_{\nu}^T \right) \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il} \end{aligned}$$

Denote

$$\begin{aligned} \mathbf{G}_{kl}^{ij} &\triangleq \frac{\partial \mathbf{u}_{ik}^T \mathbf{A}(\mathbf{C}_i) \mathbf{u}_{il}}{\partial \mathbf{c}_{f_{ij}}} = \frac{1}{N_i} \mathbf{u}_{ik}^T \mathbf{S}_{\mathbf{p}} \mathbf{T}_j \mathbf{g}_1 (\mathbf{T}_j^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \\ &\quad - \frac{1}{N_i^2} \mathbf{u}_{ik}^T \mathbf{S}_{\mathbf{p}} \sum_{\mu=1}^{M_p} \mathbf{T}_{\mu} \mathbf{C}_{f_{i\mu}} \mathbf{F} \mathbf{g}_1 (\mathbf{T}_j^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \\ &\quad - \frac{1}{N_i^2} \mathbf{u}_{ik}^T \mathbf{S}_{\mathbf{p}} \mathbf{T}_j \mathbf{g}_1 \left(\sum_{\nu=1}^{M_p} \mathbf{F} \mathbf{C}_{f_{i\nu}} \mathbf{T}_{\nu}^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il} \right) \in \mathbb{R}^{1 \times 9} \\ &= \frac{1}{N_i} \mathbf{u}_{ik}^T \mathbf{S}_{\mathbf{p}} \left(\mathbf{T}_j \mathbf{g}_1 (\mathbf{T}_j^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) - \frac{1}{N_i} \mathbf{C}_i \mathbf{F} \mathbf{g}_1 (\mathbf{T}_j^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \right. \\ &\quad \left. - \frac{1}{N_i} \mathbf{T}_j \mathbf{g}_1 (\mathbf{F} \mathbf{C}_i \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \right) \end{aligned} \quad (212)$$

Substitute it into $\mathbf{L}_{ij}^p$:

$$\begin{aligned} \mathbf{L}_{ij}^p &= \frac{2}{N_i} \left(\mathbf{g}_2(\mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} (\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i) \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) + \right. \\ &\quad \mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} (\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i) \mathbf{S}_{\mathbf{p}}^T \left(\sum_{k=1, k \neq l}^3 \frac{\mathbf{u}_{ik} \mathbf{G}_{kl}^{ij}}{\lambda_{il} - \lambda_{ik}} \right) \\ &\quad - \frac{2}{N_i^2} \mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{C}_{f_{ip}} \mathbf{F} \mathbf{T}_j \mathbf{g}_1 (\mathbf{T}_j^T \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il}) \\ &\quad \left. + \frac{2}{N_i} \mathbb{1}_{p=j} \cdot \left(\mathbf{V}_{il} \mathbf{T}_{\mathbf{p}} \mathbf{g}_1 \left((\mathbf{T}_{\mathbf{p}}^T - \frac{1}{N_i} \mathbf{F} \mathbf{C}_i) \mathbf{S}_{\mathbf{p}}^T \mathbf{u}_{il} \right) \right) \right). \end{aligned} \quad (213)$$

Since $\mathbf{J} = [\dots \quad \mathbf{J}^p \quad \dots]$ and (202), we obtain

$$\frac{\partial \mathbf{J}^T}{\partial \mathbf{c}_{f_{ij}}} = \begin{bmatrix} \vdots \\ \frac{\partial(\mathbf{J}^p)^T}{\partial \mathbf{c}_{f_{ij}}} \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ \mathbf{L}_{ij}^p \\ \vdots \end{bmatrix} \triangleq \mathbf{L}_{ij} \in \mathbb{R}^{6M_p \times 9}, \quad (214)$$

$$\frac{\partial \mathbf{J}^T (\mathbf{T}^*, \mathbf{C}_f)}{\partial \mathbf{C}_f} \delta \mathbf{C}_f = \sum_{i=1}^{M_f} \sum_{j=1}^{M_p} \mathbf{L}_{ij} \delta \mathbf{c}_{f_{ij}}. \quad (215)$$

Finally, from (52) of the main texts, we have

$$\delta \mathbf{T}^* = \mathbf{H}^{-1} \left(\sum_{i=1}^{M_f} \sum_{j=1}^{M_p} \mathbf{L}_{ij} \delta \mathbf{c}_{f_{ij}} \right) \in \mathbb{R}^{6M_p}, \quad (216)$$

$$\Sigma_{\delta \mathbf{T}^*} = \mathbf{H}^{-1} \left(\sum_{i=1}^{M_f} \sum_{j=1}^{M_p} \mathbf{L}_{ij} \Sigma_{\mathbf{c}_{f_{ij}}} \mathbf{L}_{ij}^T \right) \mathbf{H}^{-1}, \quad (217)$$

where

$$\Sigma_{c_{f_{ij}}} = \sum_{k=1}^{N_{ij}} \mathbf{B}_{f_{ijk}} \Sigma_{\mathbf{p}_{f_{ijk}}} \mathbf{B}_{f_{ijk}}^T, \quad (218)$$

which is obtained from (200) and can be computed beforehand without enumerating each raw point in the run time.

IV. TIME COMPLEXITY ANALYSIS

In this section, we analyze the time complexity of the proposed BA solver in Algorithm 1 and compare it with other similar BA methods, including BALM [32] and Plane Adjustment [34]. The computation time of Algorithm 1 mainly consists of the evaluation of the Jacobian $\mathbf{J}$ and Hessian matrix $\mathbf{H}$ on Line 6, and the solving of the linear equation on Line 9. For the evaluation of the Jacobian $\mathbf{J}$, according to (42), it consists of M_f items, each item $\mathbf{J}_i$ requires to evaluate M_p block elements, and each block $\mathbf{g}_{il}^j$ requires a constant computation time according to (34). Therefore, the time complexity for the evaluation of $\mathbf{J}$ is $O(M_f M_p)$. Similarly, for the Hessian matrix, according to (42), it consists of M_f items, each item $\mathbf{H}_i$ requires to evaluate M_p^2 block elements, and each block $\mathbf{H}^{ij}$ requires a constant computation time according to (35). Therefore, the time complexity for the evaluation of $\mathbf{H}$ is $O(M_f M_p^2)$. On Line 9, the linear equation has a dimension of $6M_p$, solving the linear equation requires an inversion of the Hessian matrix which contributes a time complexity of $O(M_p^3)$. As a result, the overall time complexity of Algorithm 1 is $O(M_f M_p + M_f M_p^2 + M_p^3)$.

Our previous work BALM [32] also eliminated the feature parameters from the optimization, leading to an optimization similar to (9) whose dimension of Jacobian and Hessian are also $6M_p$. To solve the cost function, BALM [32] adopted a second order solver similar to Algorithm 1, where the linear solver on Line 9 has the same time complexity $O(M_p^3)$. However, when deriving the Jacobian and Hessian matrix of the cost function, the chain rule was used where the Jacobian and Hessian is the multiplication of the cost w.r.t. each point of a feature and the derivative of the point w.r.t. scan poses (see Section III. C of [32]). As a consequence, for each feature, the evaluation of Jacobian has complexity of $O(NM_p)$ and the Hessian has $O(N^2 M_p^2)$, where N is the average number of points on each feature. Therefore, the time complexity including the evaluation of all features' Jacobian and Hessian matrices and the linear solver are $O(NM_f M_p + N^2 M_f M_p^2 + M_p^3)$.

The plane adjustment method in [34] is a direct mimic of the visual bundle adjustment, which does not eliminate the feature parameters but optimizes them along with the poses in each iteration. The resultant linear equation at each iteration is in the form of $\mathbf{J}^T \mathbf{J} \delta \mathbf{x} = \mathbf{b}$, where $\mathbf{J}$ is the Jacobian of the cost function w.r.t. both the pose and feature parameters. Due to a reduced residual and Jacobian technique similar to our point cluster method, the evaluation of $\mathbf{J}$ does not need to enumerate each point of a feature, so the time complexity of computing $\mathbf{J}$ for all M_f features is $O(M_f M_p + M_f)$ by noticing the inherent sparsity. Let $\mathbf{H} = \mathbf{J}^T \mathbf{J} = \begin{bmatrix} \mathbf{H}_{11} & \mathbf{H}_{12} \\ \mathbf{H}_{12}^T & \mathbf{H}_{22} \end{bmatrix}$, $\delta \mathbf{x} =$

Algorithm 1: LM optimization

Input: Initial poses $\mathbf{T}$;
Point cluster in the local frame $\mathbf{C}_{f_{ij}}$;
1 $\mu = 0.01$, $\nu = 2$, $j = 0$;
2 **repeat**
3 $j = j + 1$;
4 $\mathbf{J} = \mathbf{0}_{1 \times 6M_p}$, $\mathbf{H} = \mathbf{0}_{6M_p \times 6M_p}$;
5 **foreach** $i \in \{1, \dots, M_f\}$ **do**
6 Compute $\mathbf{J}_i$ and $\mathbf{H}_i$ from (30) and (31);
7 $\mathbf{J} = \mathbf{J} + \mathbf{J}_i$; $\mathbf{H} = \mathbf{H} + \mathbf{H}_i$
8 **end**
9 Solve $(\mathbf{H} + \mu \mathbf{I}) \Delta \mathbf{T} = -\mathbf{J}^T$;
10 $\mathbf{T}' = \mathbf{T} \boxplus \Delta \mathbf{T}$;
11 Compute current cost $c = c(\mathbf{T})$ and the new cost
 $c' = c(\mathbf{T}')$ from (22);
12 $\rho = (c - c') / (\frac{1}{2} \Delta \mathbf{T} \cdot (\mu \Delta \mathbf{T} - \mathbf{J}^T))$;
13 **if** $\rho > 0$ **then**
14 $\mathbf{T} = \mathbf{T}'$;
15 $\mu = \mu * \max(\frac{1}{3}, 1 - (2\rho - 1)^3)$; $\nu = 2$;
16 **else**
17 $\mu = \mu * \nu$; $\nu = 2 * \nu$;
18 **end**
19 **until** $\|\Delta \mathbf{T}\| < \epsilon$ **or** $j \geq j_{max}$;
Output: Final optimized states $\mathbf{T}$;

$\begin{bmatrix} \delta \mathbf{T} \\ \delta \boldsymbol{\pi} \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}$, where $\delta \mathbf{T} \in \mathbb{R}^{6M_p}$ is the pose component and $\delta \boldsymbol{\pi} \in \mathbb{R}^{3M_f}$ is feature component. The plane adjustment in [34] further used a Schur complement technique similar to visual bundle adjustment, leading to the optimization of the pose vector only: $(\mathbf{H}_{11} - \mathbf{H}_{12} \mathbf{H}_{22}^{-1} \mathbf{H}_{21}) \delta \mathbf{T} = \mathbf{b}_1 - \mathbf{H}_{12} \mathbf{H}_{22}^{-1} \mathbf{b}_2$. Since $\mathbf{H}_{22}$ is block diagonal, its inverse has a time complexity of $O(M_f)$. As a consequence, constructing this linear equation would require computing $\mathbf{H}_{12} \mathbf{H}_{22}^{-1} \mathbf{H}_{21}$ and $\mathbf{H}_{12} \mathbf{H}_{22}^{-1} \mathbf{b}_2$, which have time complexity of $O(M_p^2 M_f)$, and solving this linear equation has a complexity of $O(M_p^3)$. As a consequence, the overall time complexity is $O(M_f + M_f M_p + M_f M_p^2 + M_p^3)$.

In summary, BALM [32] has a complexity $O(NM_f M_p + N^2 M_f M_p^2 + M_p^3)$, which is linear to M_f , the number of feature, quadratic to N , the number of point, and cubic to M_p , the number of pose. The plane adjustment [34] has a complexity $O(M_f + M_f M_p + M_f M_p^2 + M_p^3)$, which is linear to M_f , irrelevant to N , and cubic to M_p . Our proposed method has a complexity of $O(M_f M_p + M_f M_p^2 + M_p^3)$, which is similar to [34] (i.e., linear to M_f , irrelevant to N , and cubic to M_p) but has less operations.

REFERENCES

- [1] A. I. Mourikis, S. I. Roumeliotis *et al.*, "A multi-state constraint kalman filter for vision-aided inertial navigation." in *ICRA*, vol. 2, 2007, p. 6.
- [2] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual-inertial odometry using nonlinear optimization," *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015.
- [3] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

- [4] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, “Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [5] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “Fast-lio2: Fast direct lidar-inertial odometry,” *IEEE Transactions on Robotics*, pp. 1–21, 2022.
- [6] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, “On-manifold preintegration for real-time visual-inertial odometry,” *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2016.
- [7] W. Xu and F. Zhang, “Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.
- [8] X. Liu, C. Yuan, and F. Zhang, “Targetless extrinsic calibration of multiple small fov lidars and cameras using adaptive voxelization,” *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–12, 2022.
- [9] X. Liu and F. Zhang, “Extrinsic calibration of multiple lidars of small fov in targetless environments,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2036–2043, 2021.
- [10] Z. Liu and F. Zhang, “Balm: Bundle adjustment for lidar mapping,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3184–3191, 2021.
- [11] X. Liu, Z. Liu, F. Kong, and F. Zhang, “Large-scale lidar consistent mapping using hierarchical lidar bundle adjustment,” *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1523–1530, 2023.
- [12] Y. Pan, P. Xiao, Y. He, Z. Shao, and Z. Li, “Mulls: Versatile lidar slam via multi-metric linear least square,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11 633–11 640.
- [13] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the kitti vision benchmark suite,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- [14] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, “Ct-icp: Real-time elastic lidar odometry with loop closure,” *arXiv preprint arXiv:2109.12979*, 2021.
- [15] G. Kim and A. Kim, “Scan context: Egocentric spatial descriptor for place recognition within 3d point cloud map,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 4802–4809.